

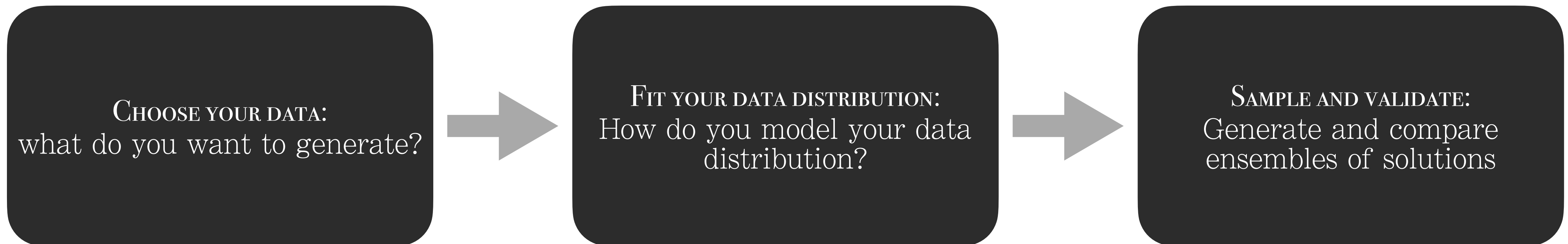
Introduction to Generative modelling

AstroAI Asian Network Summer School, Taipei 2026

Daniela Breitman, University of Tokyo

Lecture in one slide:

GENERATIVE MODELLING is a probabilistic procedure for creating new data samples given existing data samples.



Outline

PART I

Generative modelling
basics

- Intro to generation
- Simplest generative model: Gaussian
- Gaussian mixture model (GMM)
- Mixture density network (MDN)
- Training procedure

PART II

Image generation

- Variational autoencoder (VAE)
- Generative adversarial network (GAN)
- Diffusion
- Applications

HANDS-ON

Galaxy survey:
Spectroscopy vs
photometry

- SDSS galaxy survey
- Photometry vs spectroscopy
- Notebook setup
- Kaggle

By the time of the hands-on, you should be able to..

Build a simple generative model!

- Explain “generation” as sampling from a fitted probability model
- Build a mixture density network (MDN)
- Train MDN with negative log likelihood (NLL)
- Validate network performance

Observations



Learn about the Universe

Bayes' Theorem

POSTERIOR

Updated knowledge on θ
provided an observation d

WHAT WE WANT

LIKELIHOOD

Probability of reproducing
observation d with model
parameters θ

FORWARD MODEL

$$P(\theta | d) = \frac{P(d | \theta) \pi(\theta)}{P(d)}$$

PRIOR

Current knowledge on
model parameters θ

EVIDENCE

Bayes' factor:

$$B_{01} = \frac{P(d | \mathcal{M}_0)}{P(d | \mathcal{M}_1)}$$

WHAT WE WANT

Bayes' Theorem

POSTERIOR

Updated knowledge on θ
provided an observation d

WHAT WE WANT

LIKELIHOOD

Probability of reproducing
observation d with model
parameters θ

FORWARD MODEL

$\sim 10^5 - 10^6$ evaluations

$$P(\theta | d) = \frac{P(d | \theta) \pi(\theta)}{P(d)}$$

PRIOR

Current knowledge on
model parameters θ

EVIDENCE

Bayes' factor:

$$B_{01} = \frac{P(d | \mathcal{M}_0)}{P(d | \mathcal{M}_1)}$$

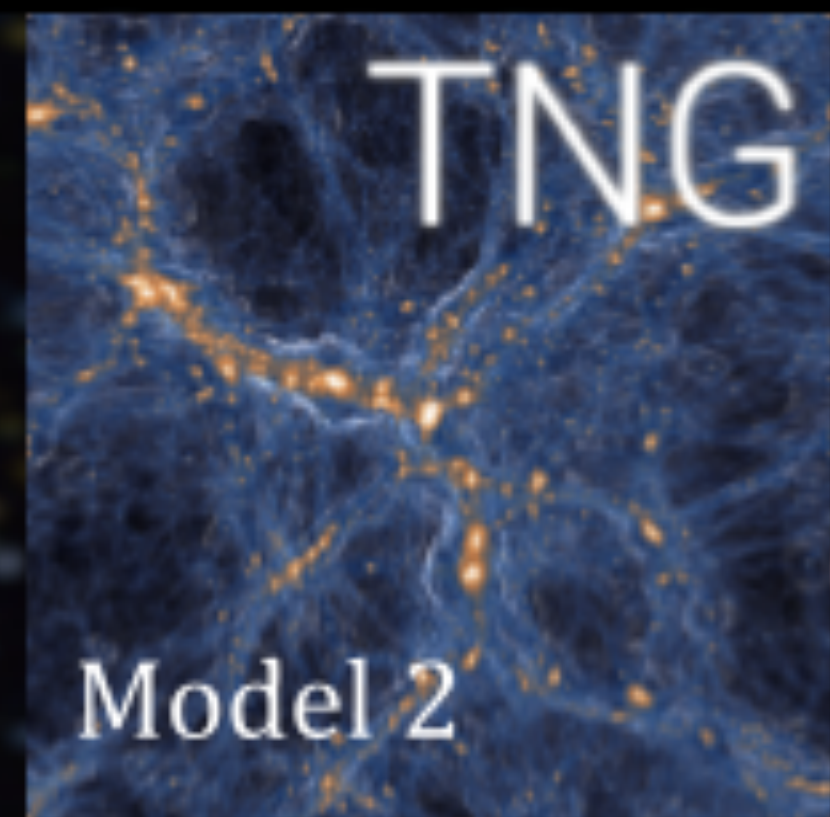
WHAT WE WANT



Model 1



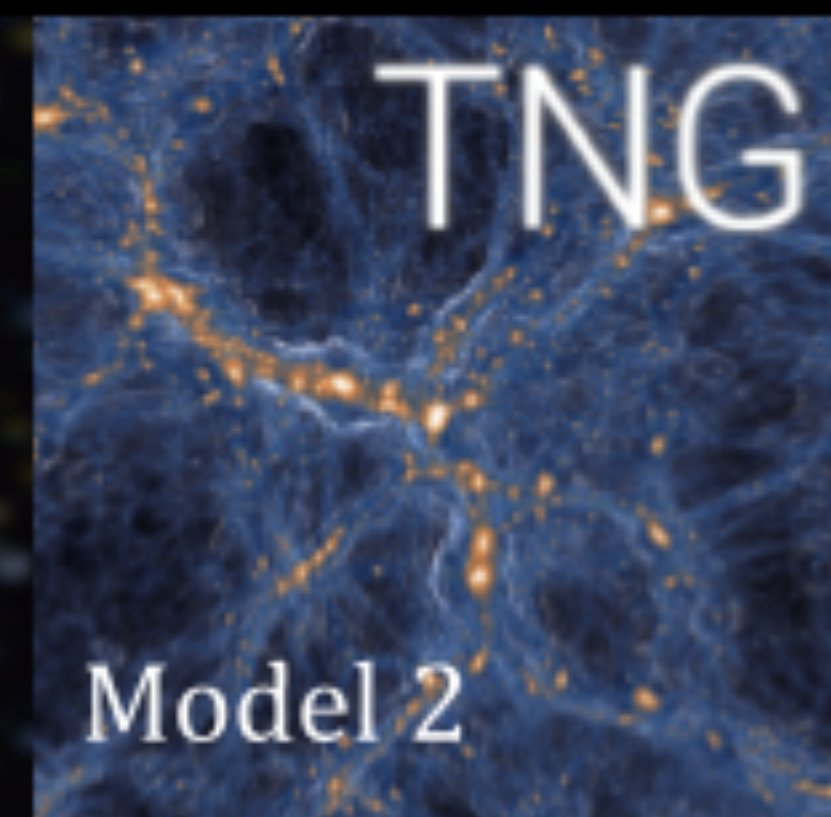
Model 1



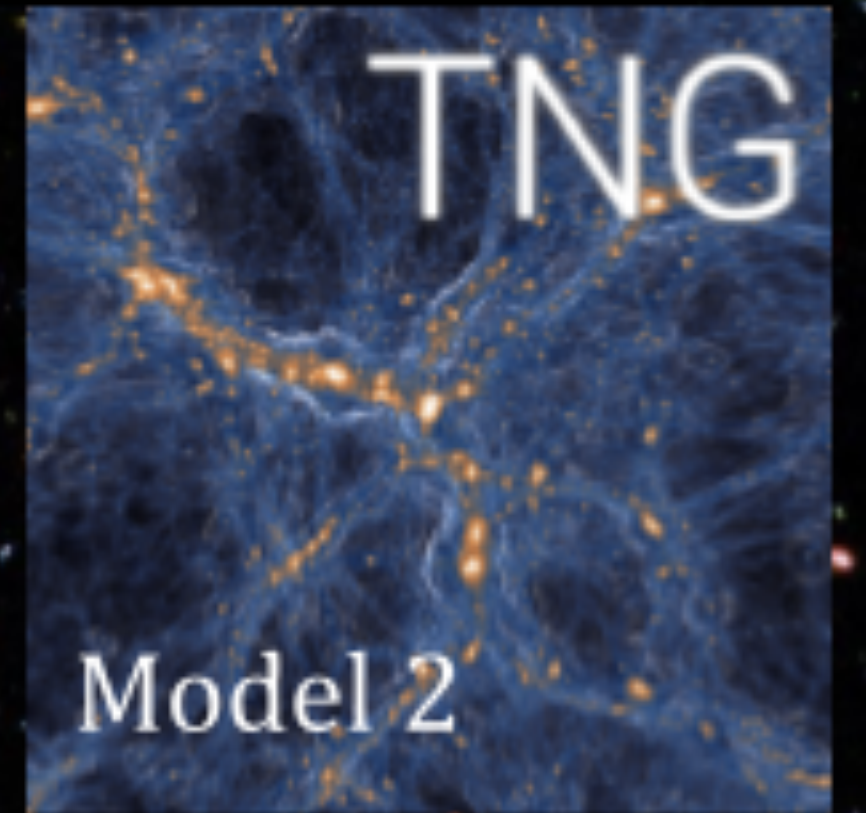
Model 2

Bayesian model selection:

Bayes factor $B_{21} \gg 1 \Rightarrow$ M2 more supported by the data than M1



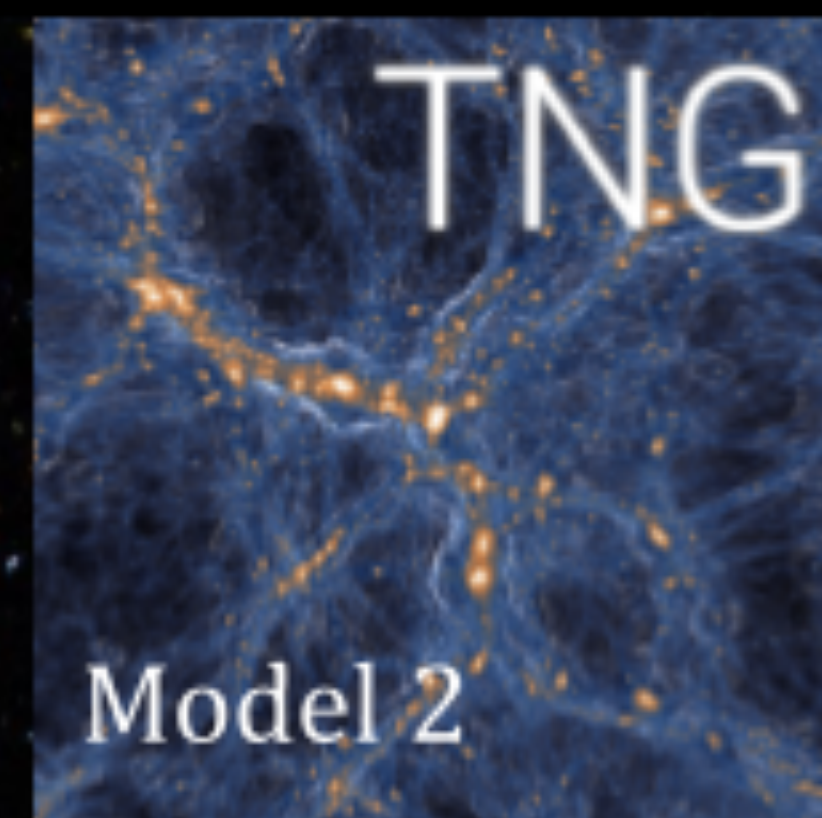
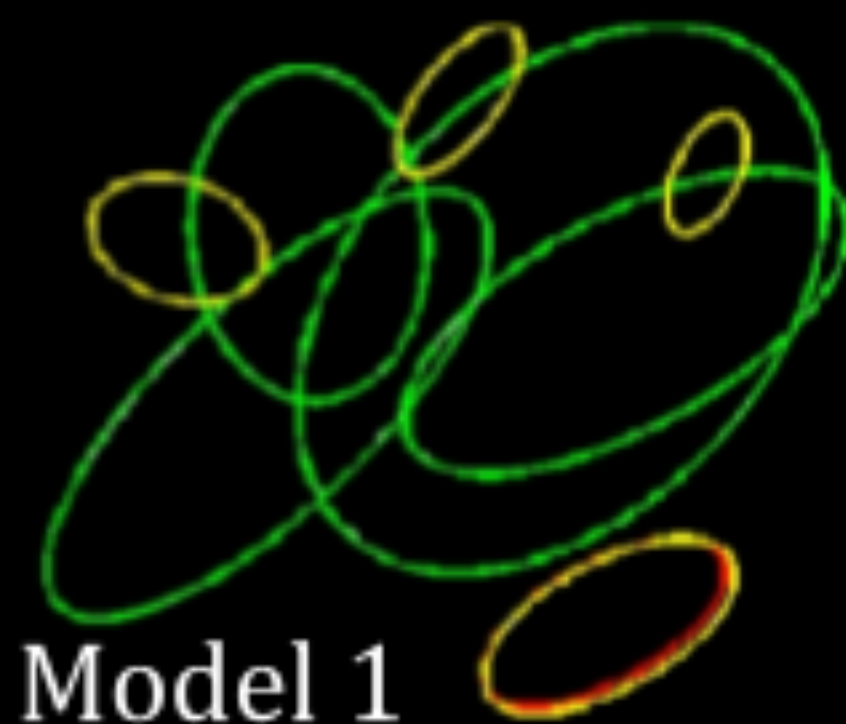
Improved observations



Improved observations

Bayesian model selection:

Bayes factor $B_{21} \gg 1 \Rightarrow$ M2 more supported by the data than M1

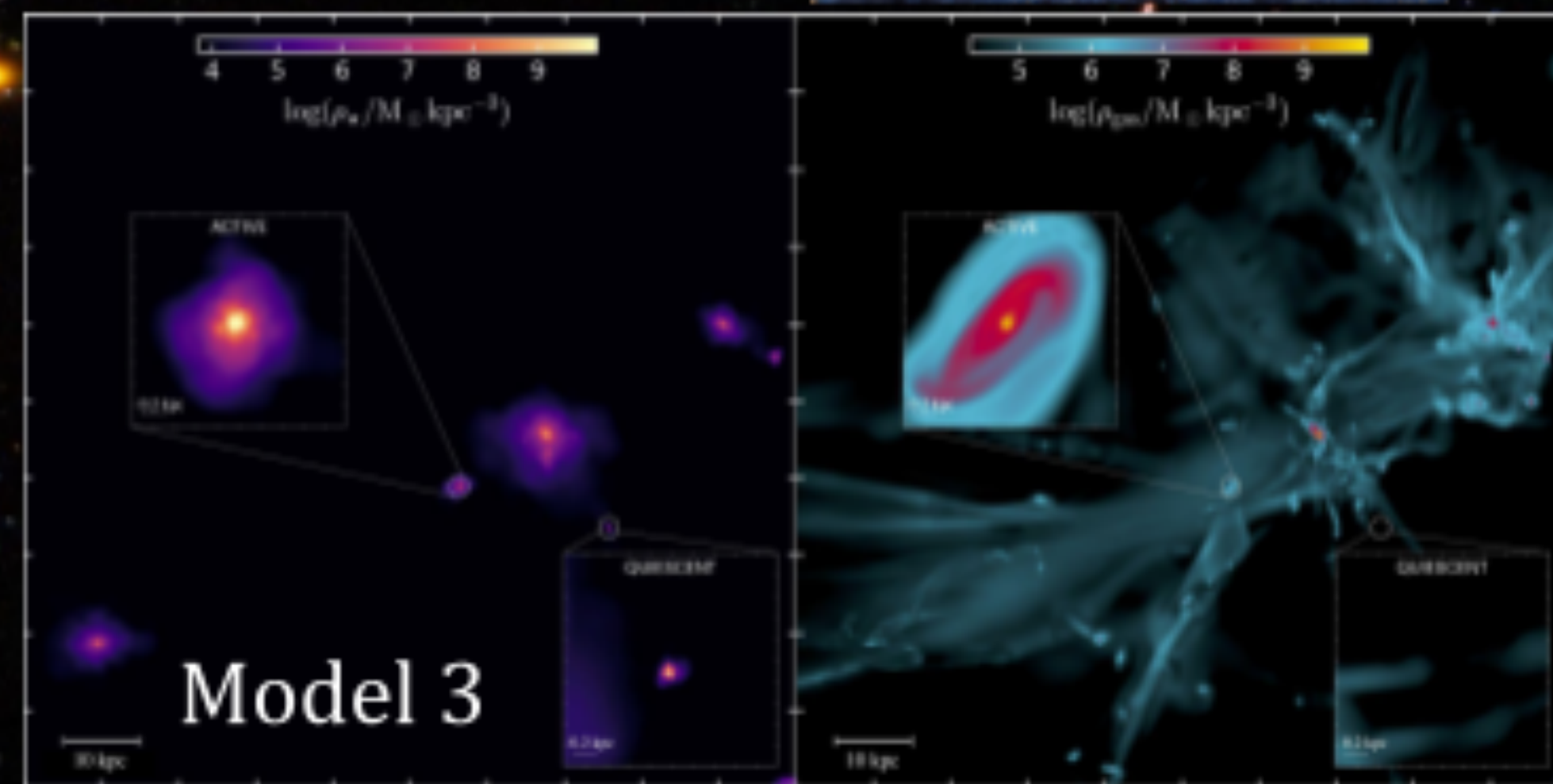
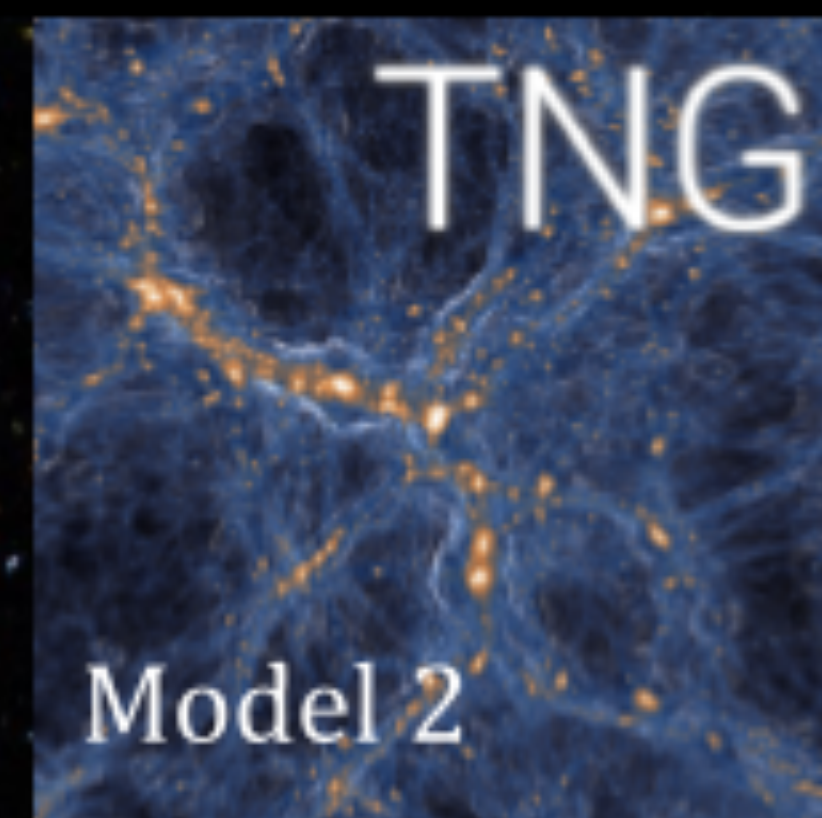
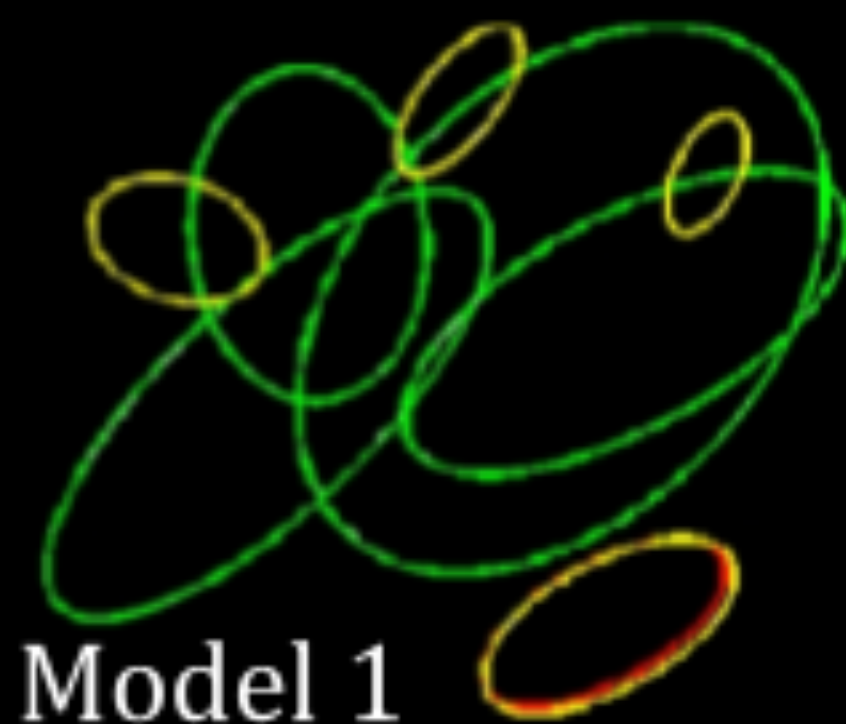


Improved observations

Bayesian model selection:

Bayes factor $B_{21} \gg 1 \Rightarrow$ M2 more supported by the data than M1

Bayes factor $B_{31} \gg \gg 1 \Rightarrow$ M3 more supported by the data than M1 and M2



Improved observations

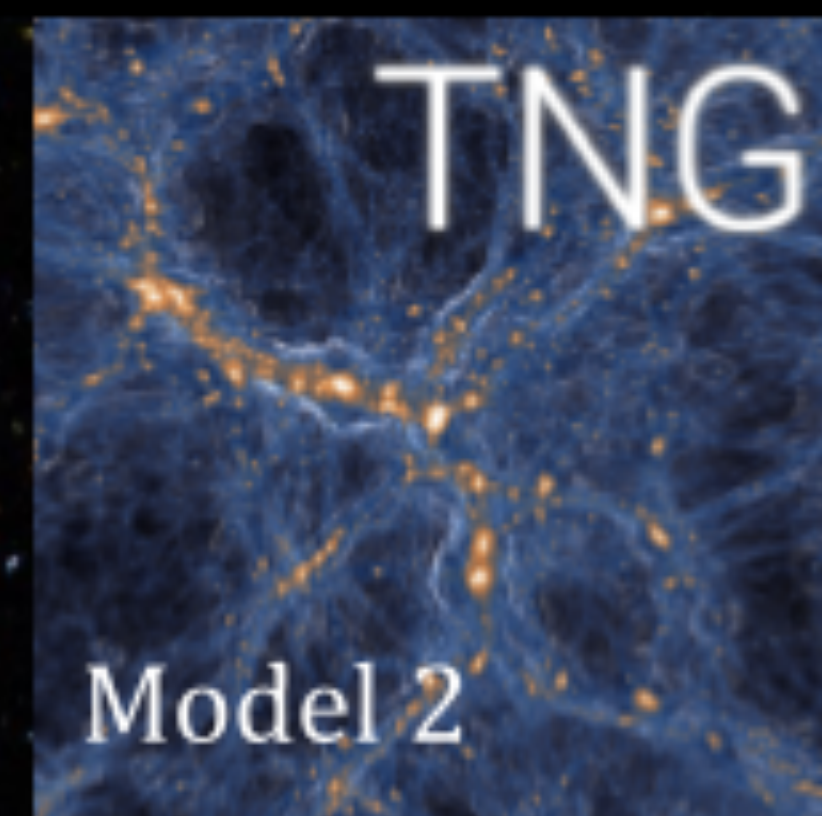
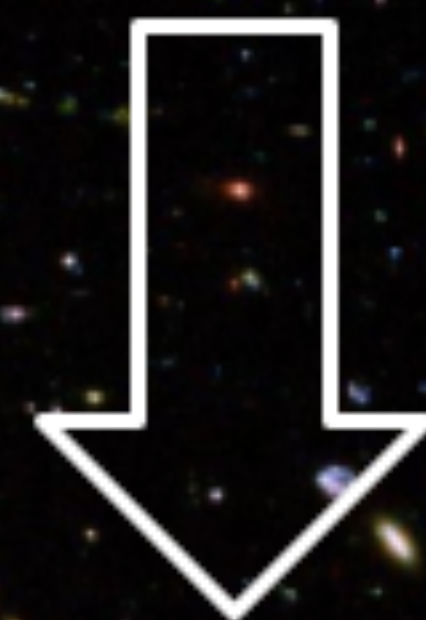
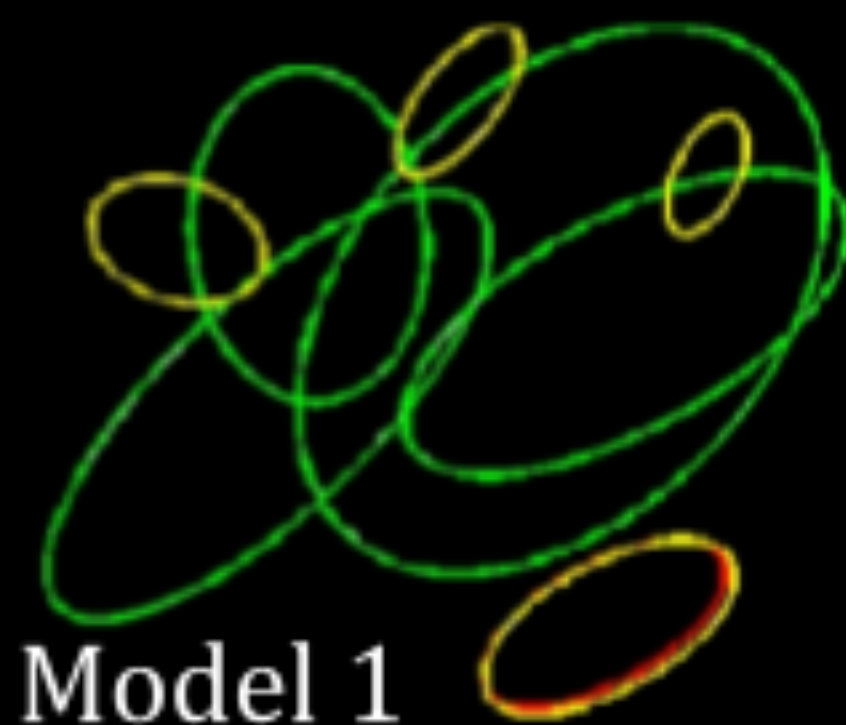
Bayesian model selection:

Bayes factor $B_{21} \gg 1 \Rightarrow$ M2 more supported by the data than M1

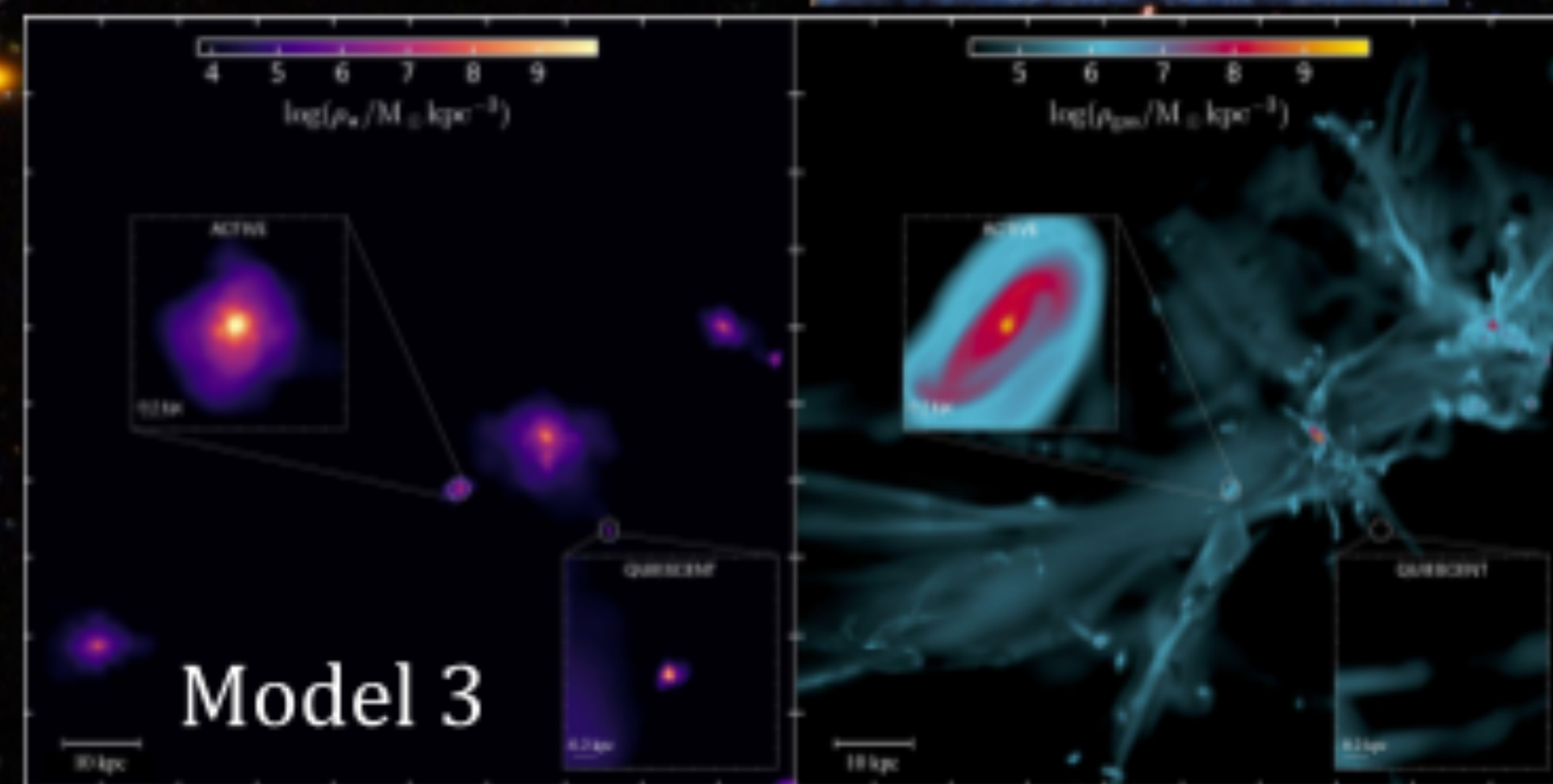
Bayes factor $B_{31} \gg \gg 1 \Rightarrow$ M3 more supported by the data than M1 and M2

In this example, the new data prefers the more complex model M3.

Simple



Complex



Theory: $\mathcal{M}_1$,
 $\mathcal{M}_2$, $\mathcal{M}_3$, etc.

Observations d

Theory: $\mathcal{M}_1$,
 $\mathcal{M}_2$, $\mathcal{M}_3$, etc.

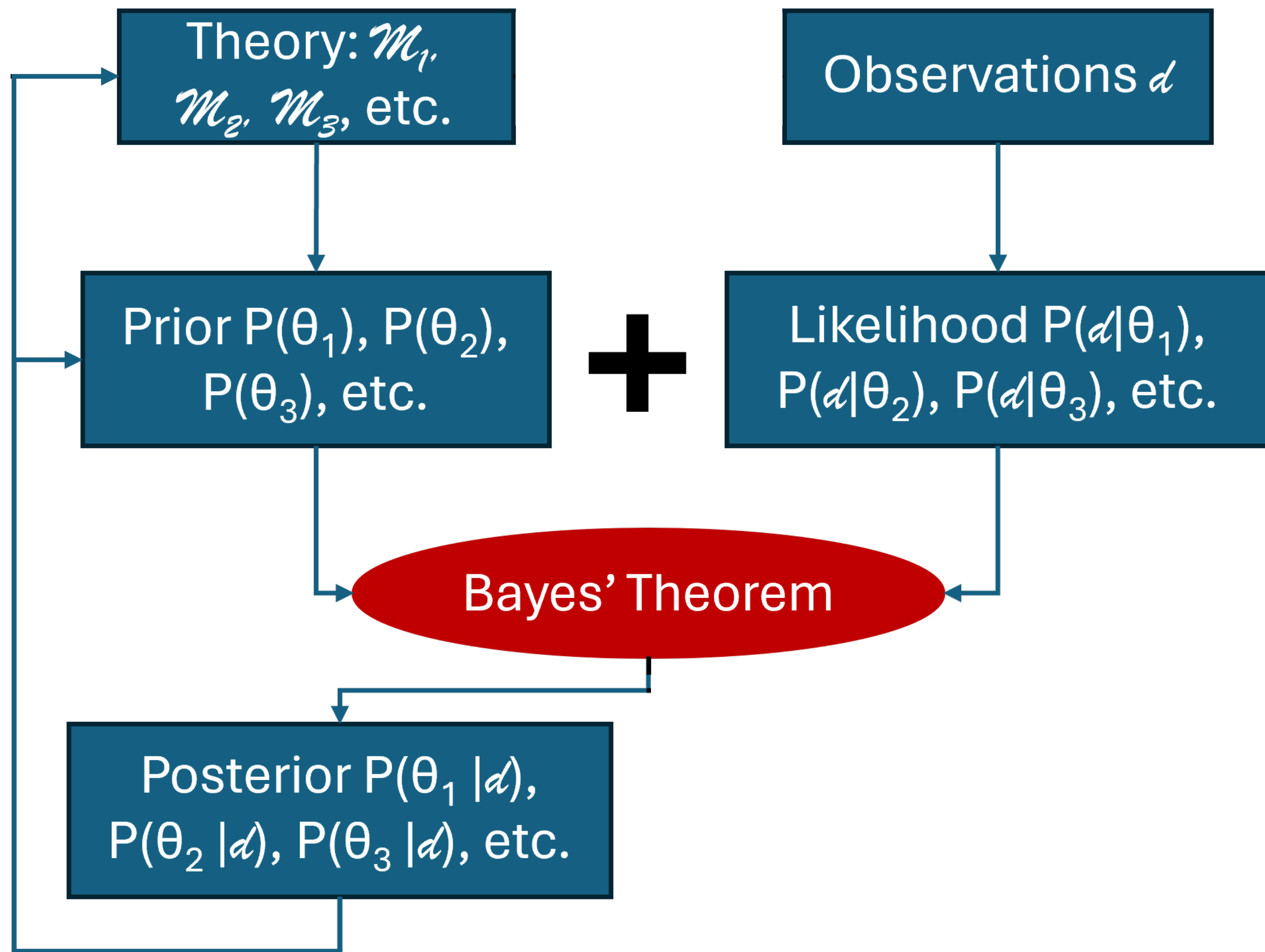
Observations d

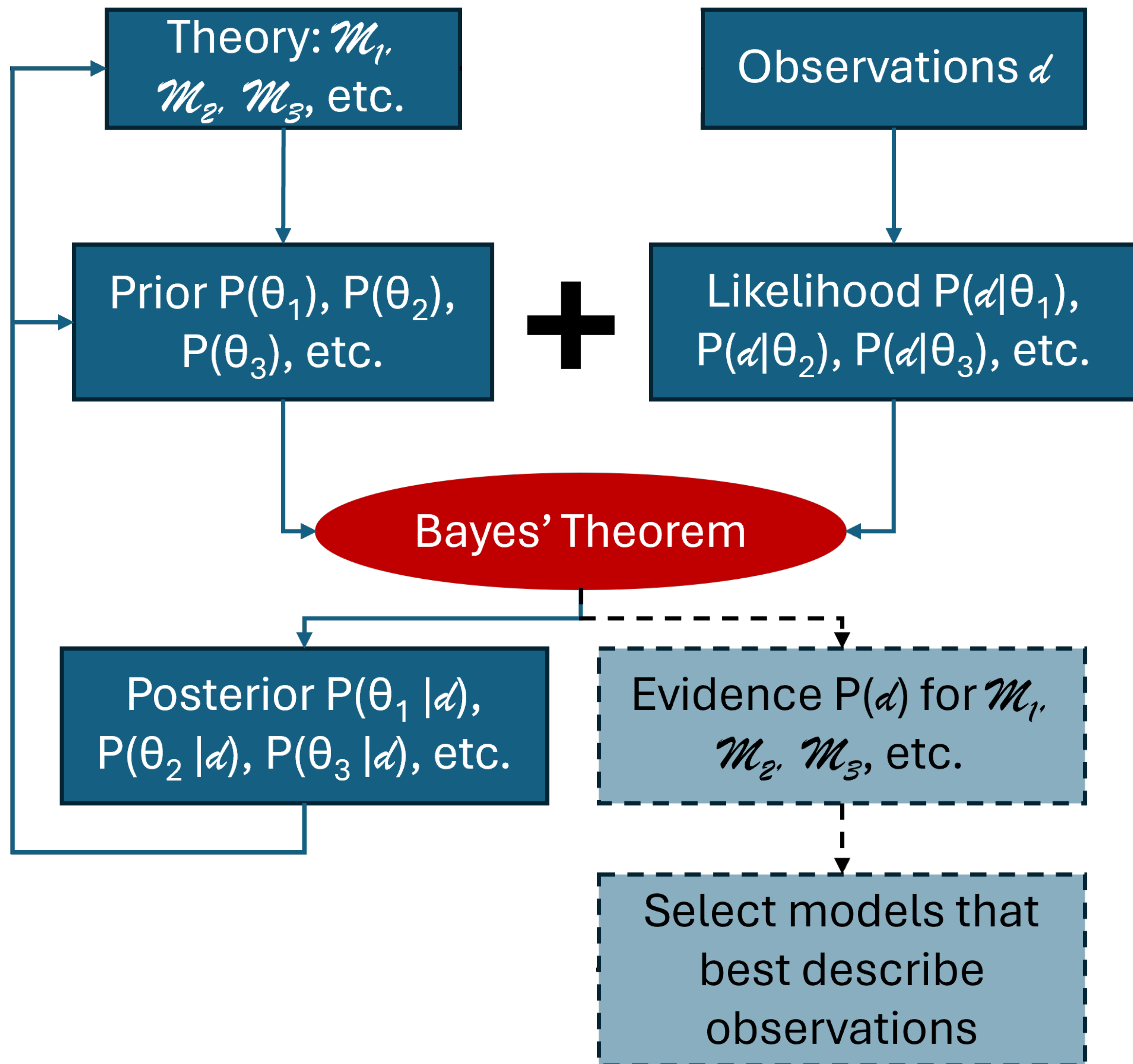
Prior $P(\theta_1)$, $P(\theta_2)$,
 $P(\theta_3)$, etc.

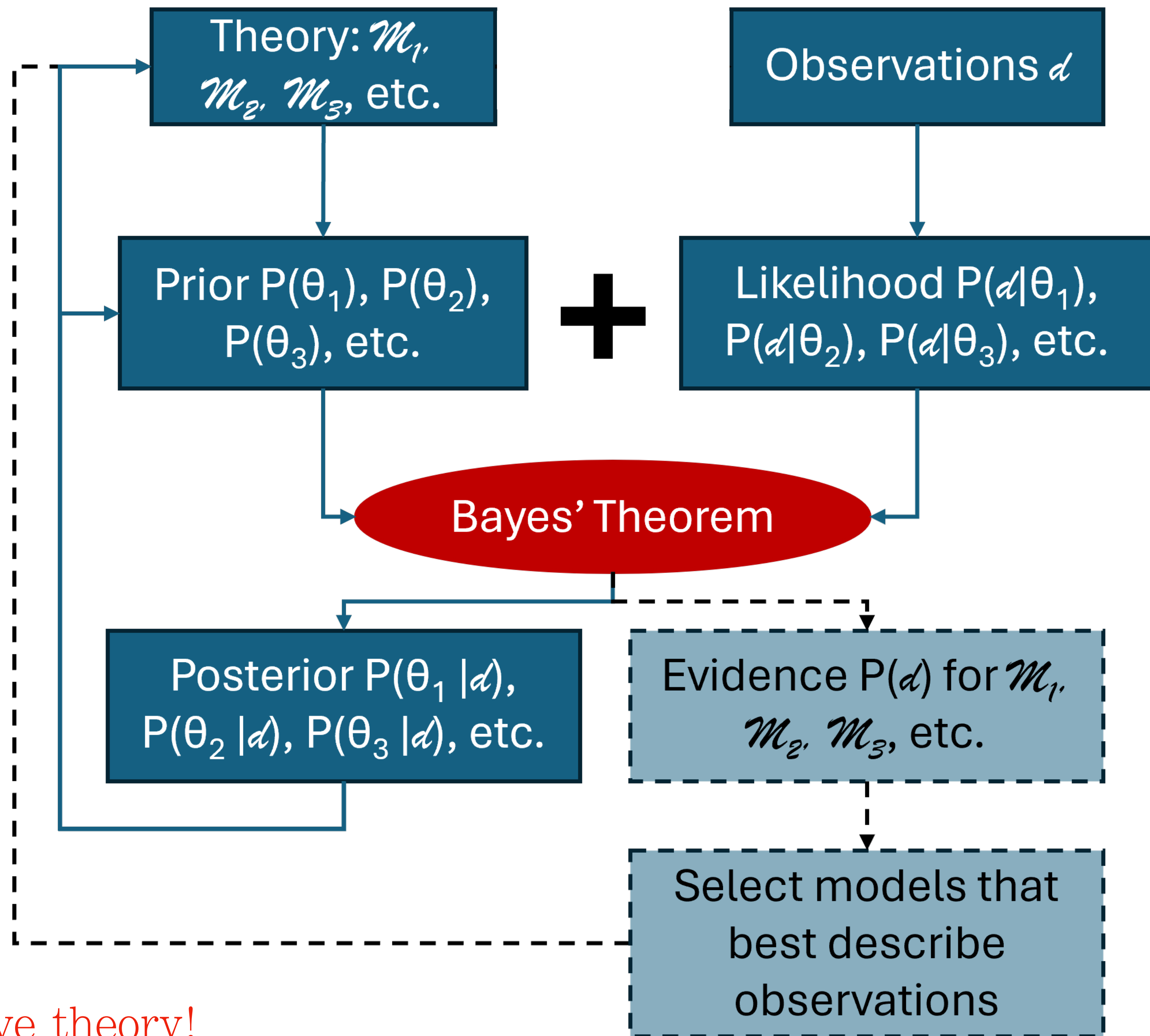
+

Likelihood $P(d|\theta_1)$,
 $P(d|\theta_2)$, $P(d|\theta_3)$, etc.

Bayes' Theorem







Observations drive theory!

Bayesian inference techniques must keep up with the ever improving quality of observations

- Expensive simulations such as N-body hydro sims, RT, etc. are not suitable for traditional Bayesian inference
- Semi-analytical models: trade some accuracy for speed
 - Can perform inference!!

Bayesian inference techniques must keep up with the ever improving quality of observations

- Expensive simulations such as N-body hydro sims, RT, etc. are not suitable for traditional Bayesian inference
- Semi-analytical models: trade some accuracy for speed
 - Can perform inference!!
 - Too slow for the fast pace of data releases $\rightarrow \sim 10^6$ CPUhrs 😞

Bayesian inference techniques must keep up with the ever improving quality of observations

- Expensive simulations such as N-body hydro sims, RT, etc. are not suitable for traditional Bayesian inference
- Semi-analytical models: trade some accuracy for speed
 - Can perform inference!!
 - Too slow for the fast pace of data releases $\rightarrow \sim 10^6$ CPUhrs 😞
 - Machine learning: pay an upfront simulation and training cost for **FAST, AMORTISED PREDICTIONS**, while introducing emulator approximation error $\rightarrow \sim$ GPUhr 😄

Discriminative vs generative

What do we learn?

Discriminative: $x \rightarrow y$ or $p(y | x)$

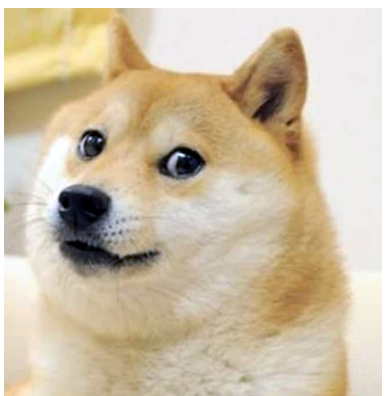
- Predict a target from given input
- Classification or regression
- Example:



$\rightarrow$ 'dog'

Generative: $p(x)$, $p(x, y)$, $p(x | c)$

- Represent how the data is distributed or produced
- Can draw new samples

- Example:  , ...,  $\sim p(x | \text{'dog'})$

Discriminative vs generative

What do we learn?

Discriminative: $x \rightarrow y$ or $p(y | x)$

- Predict a target from given input
- Classification or regression
- Example:



$\rightarrow$ 'dog'

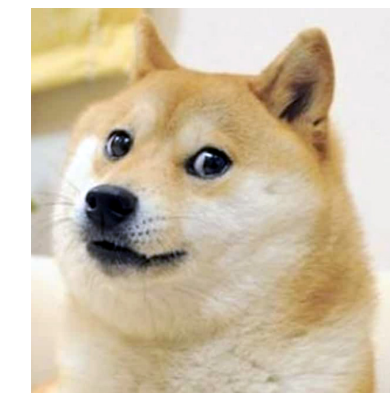
Generative: $p(x)$, $p(x, y)$, $p(x | c)$

- Represent how the data is distributed or produced
- Can draw new samples

- Example:



, ...,



$\sim p(x | \text{'dog'})$

Takeaway: a generative model REPRESENTS A PROBABILITY DISTRIBUTION and provides a PROCEDURE FOR SAMPLING FROM IT.

Is a classifier a generative model?

A classifier outputs the probability that the input belongs to a certain class.

Does that make it a generative model?



Deterministic vs stochastic forward modelling

A different question: what happens when we repeat the model at fixed conditions?

Point prediction: $x = f(\theta)$

same input $\rightarrow$ same output

Appropriate when:

- Mean prediction is sufficient
- Negligible stochasticity
- Uncertainty is neglected or added separately

Probabilistic prediction: $z \sim p(z), x = g(\theta, z)$

$(\theta, z_1, z_2, z_3) \rightarrow x_1, x_2, x_3$

Common sources of stochasticity:

- Cosmological simulation initial conditions
- Instrument or measurement noise
- Physics

The simplest generative model - Gaussian distribution

$$\text{In 1D: } \mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \text{ where } \left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$$

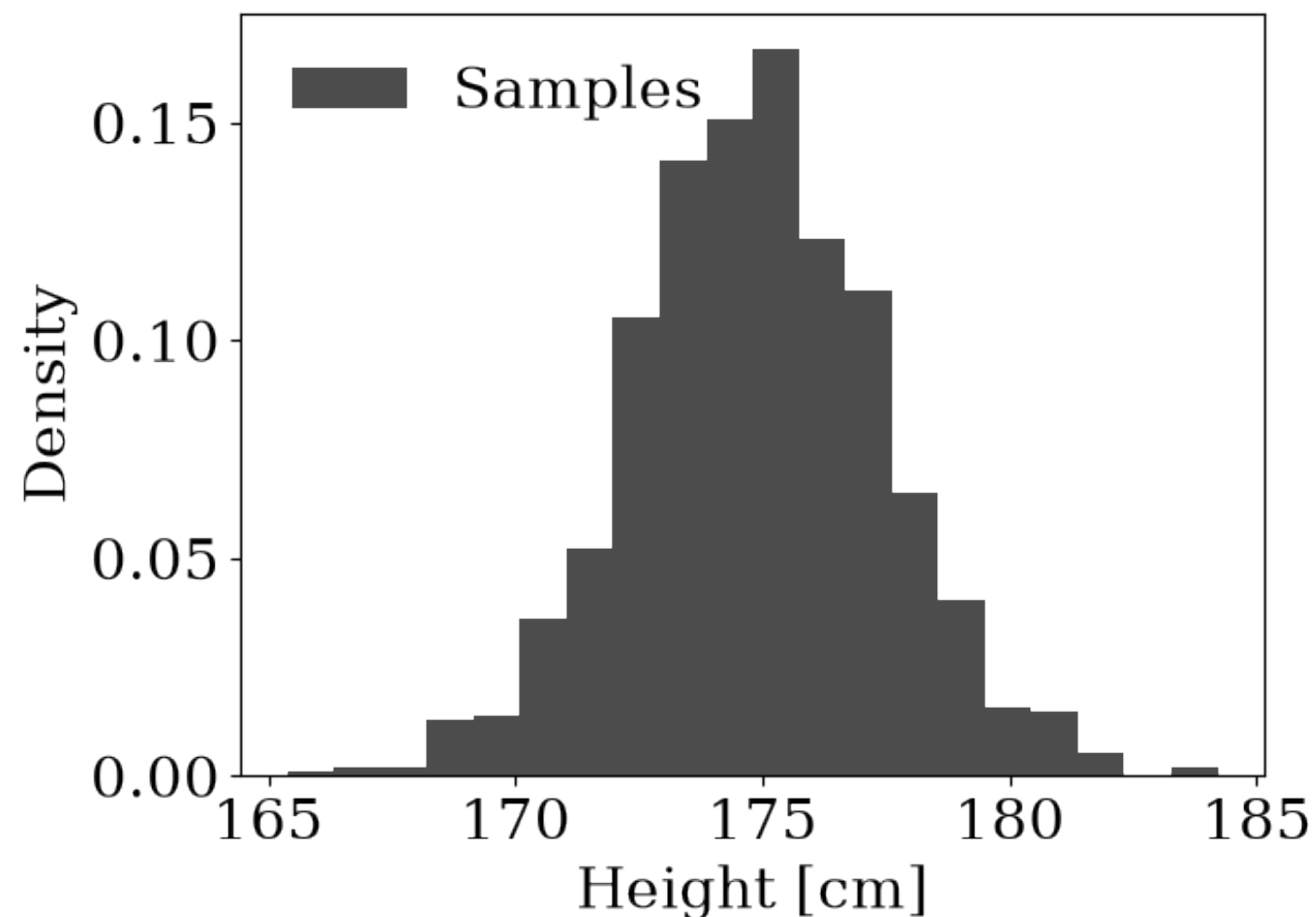
The simplest generative model - Gaussian distribution

$$\text{In 1D: } \mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \text{ where } \left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$$

The simplest generative model - Gaussian distribution

Example: height of people

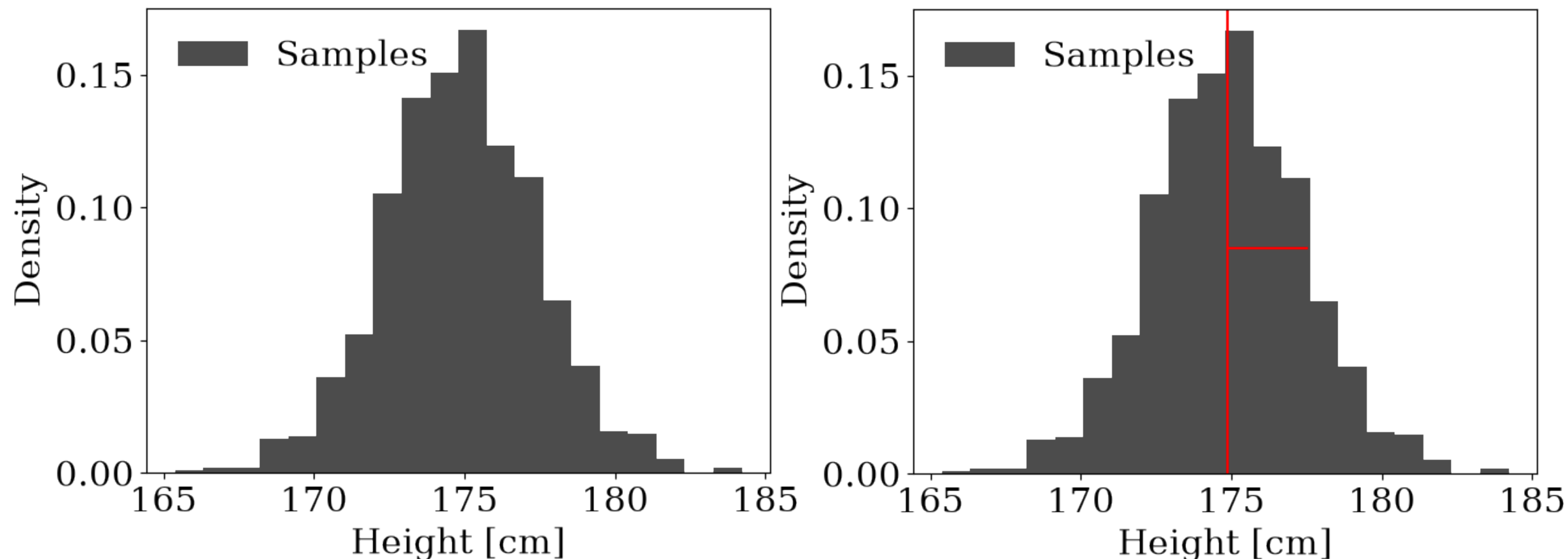
In 1D: $\mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$, where $\left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$



The simplest generative model - Gaussian distribution

Example: height of people

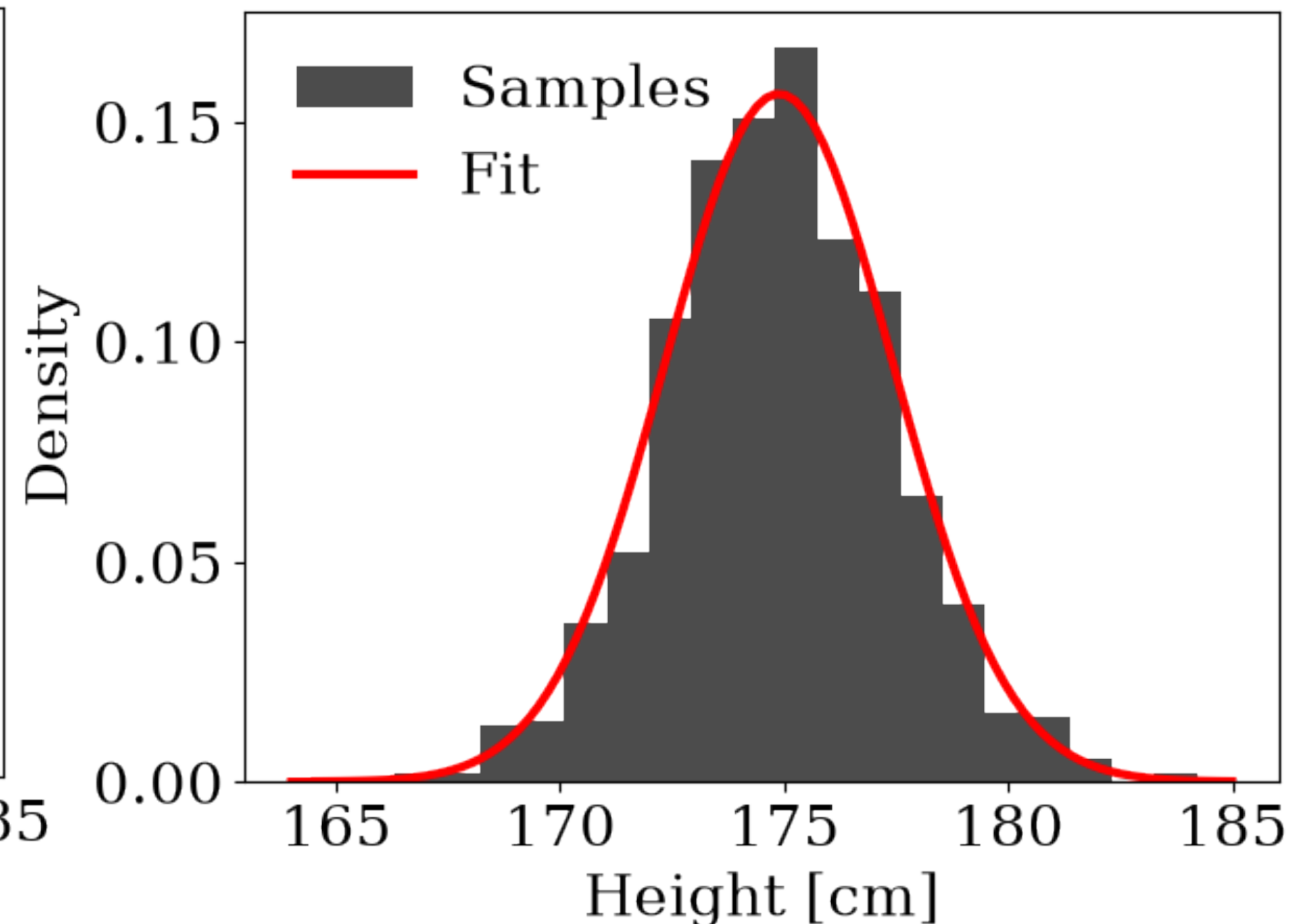
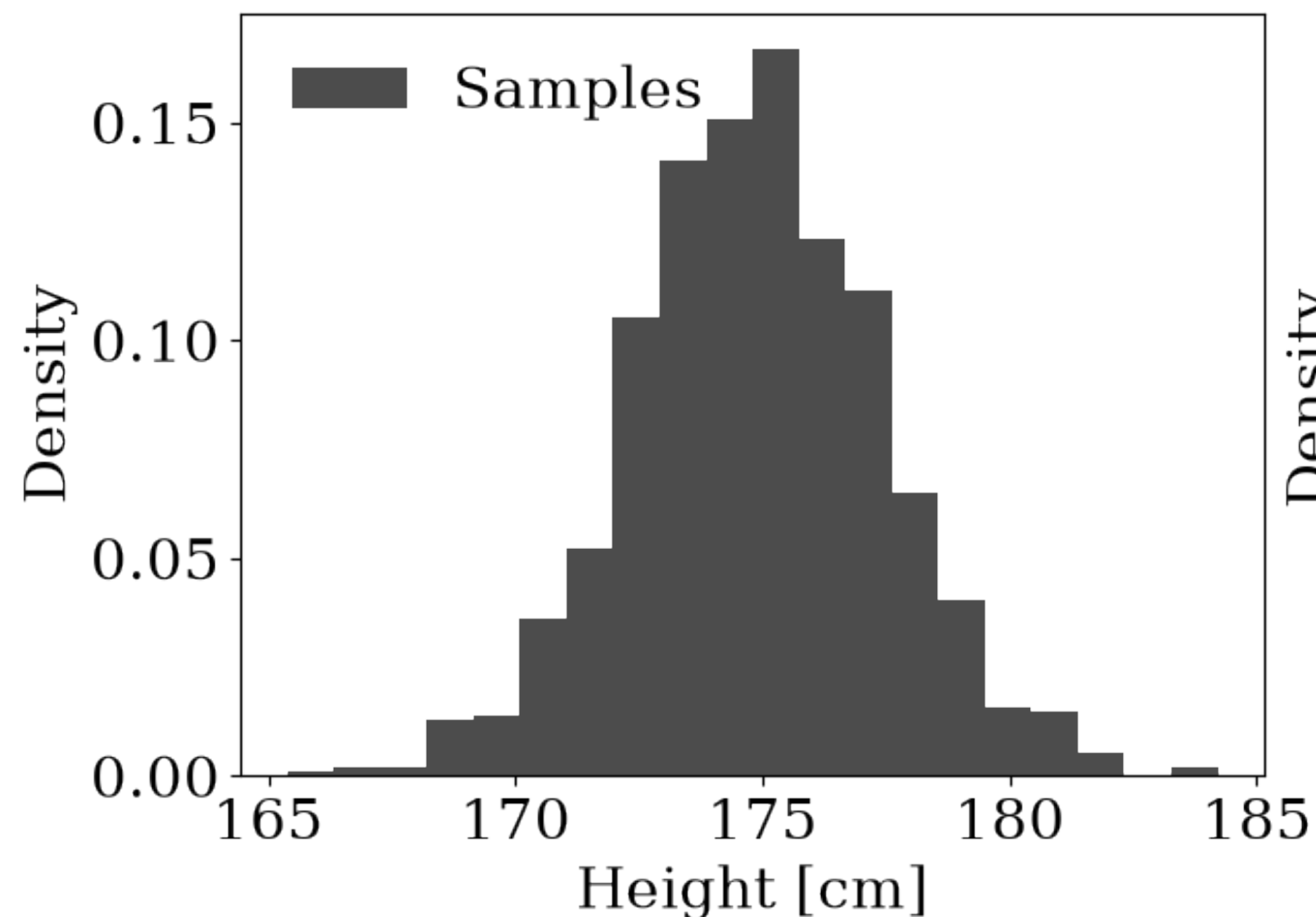
In 1D: $\mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$, where $\left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$



The simplest generative model - Gaussian distribution

Example: height of people

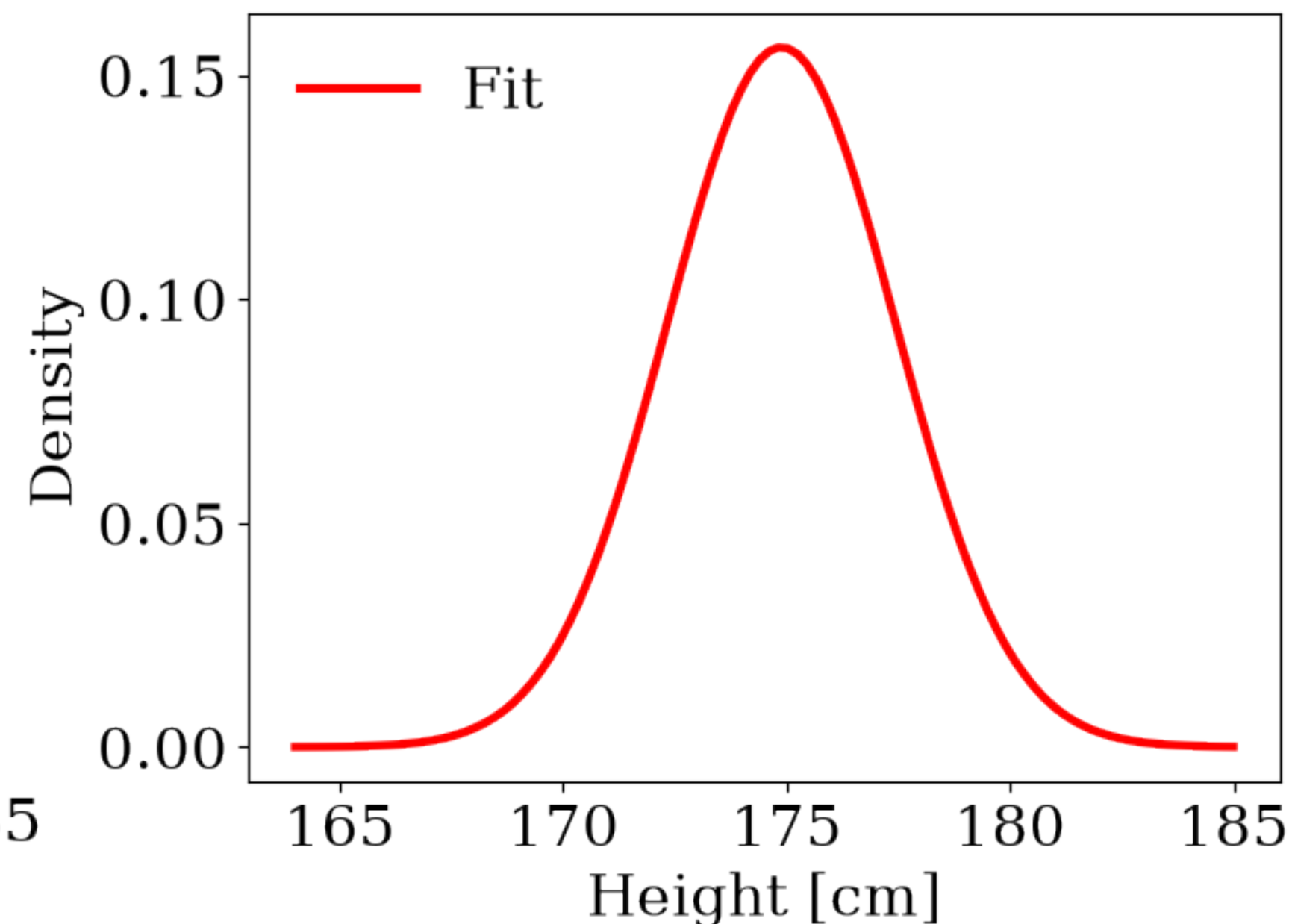
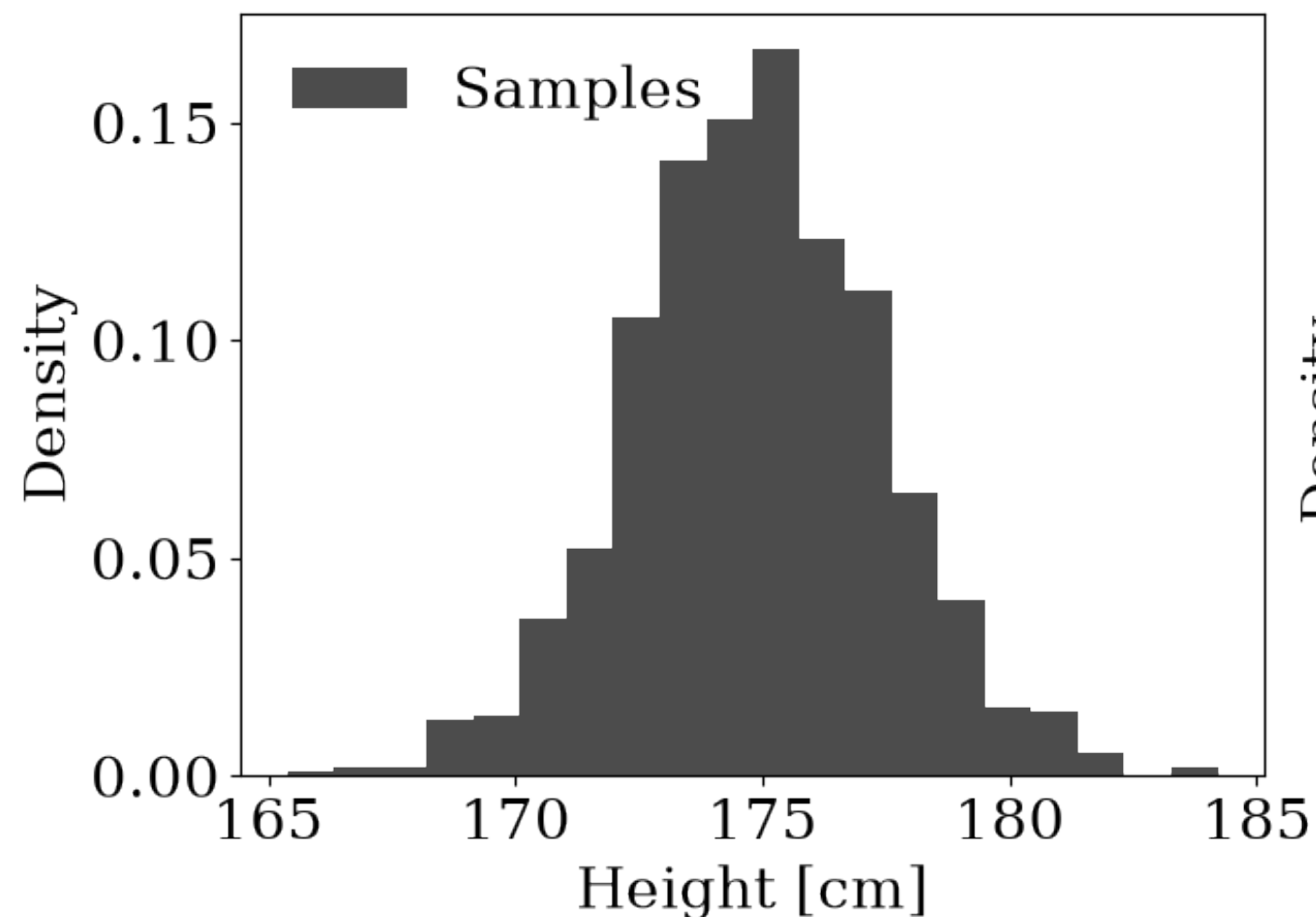
In 1D: $\mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$, where $\left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$



The simplest generative model - Gaussian distribution

Example: height of people

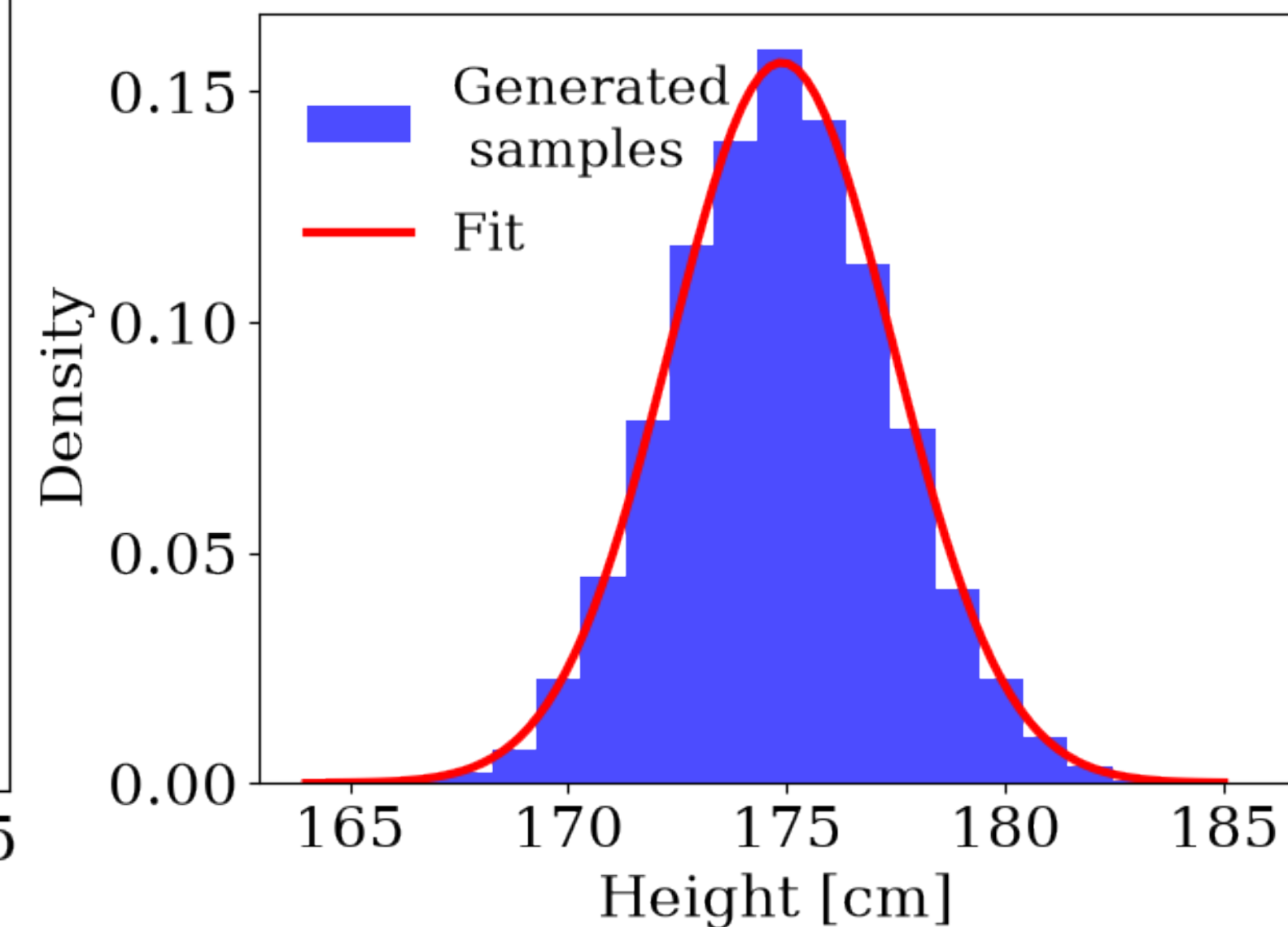
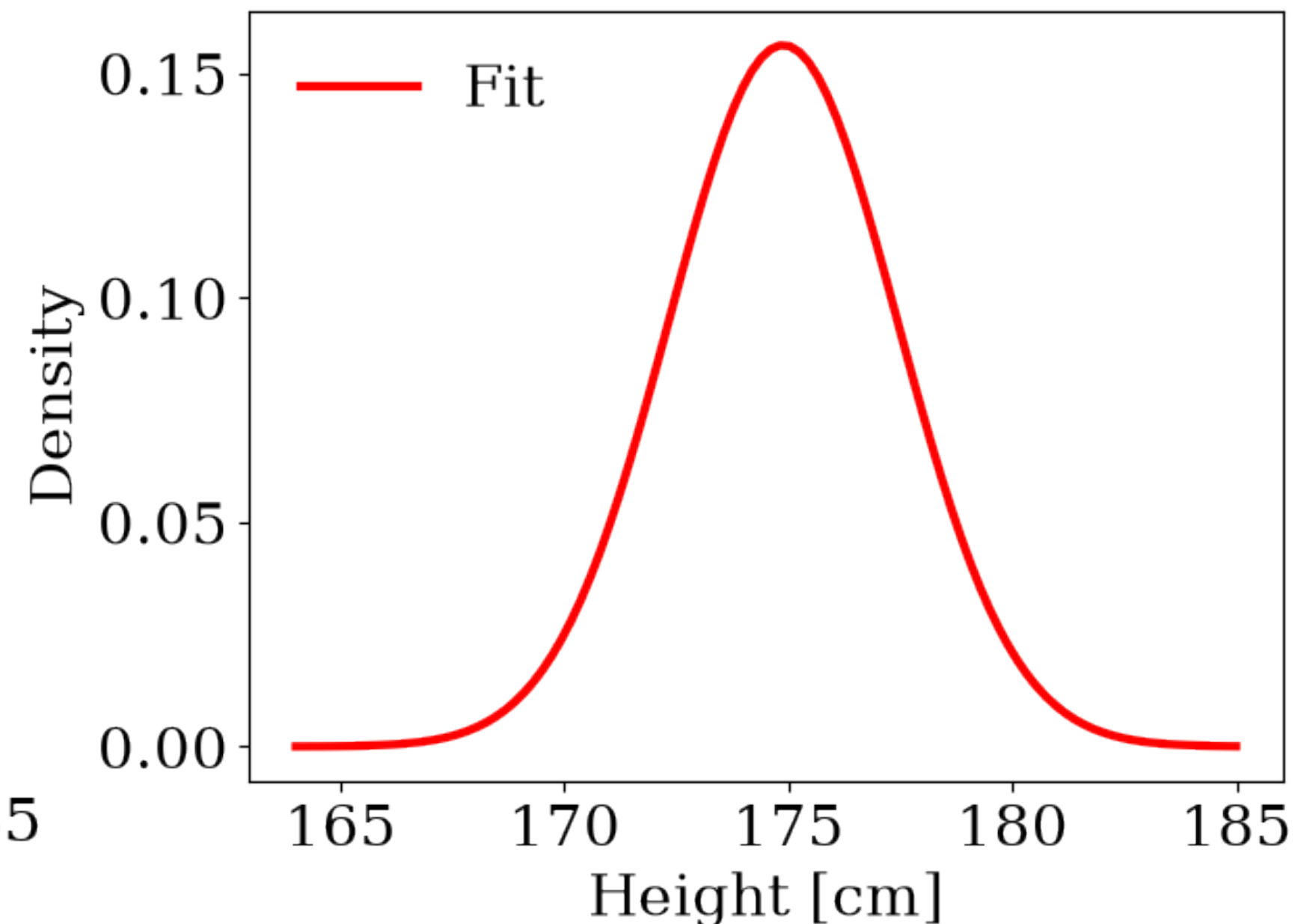
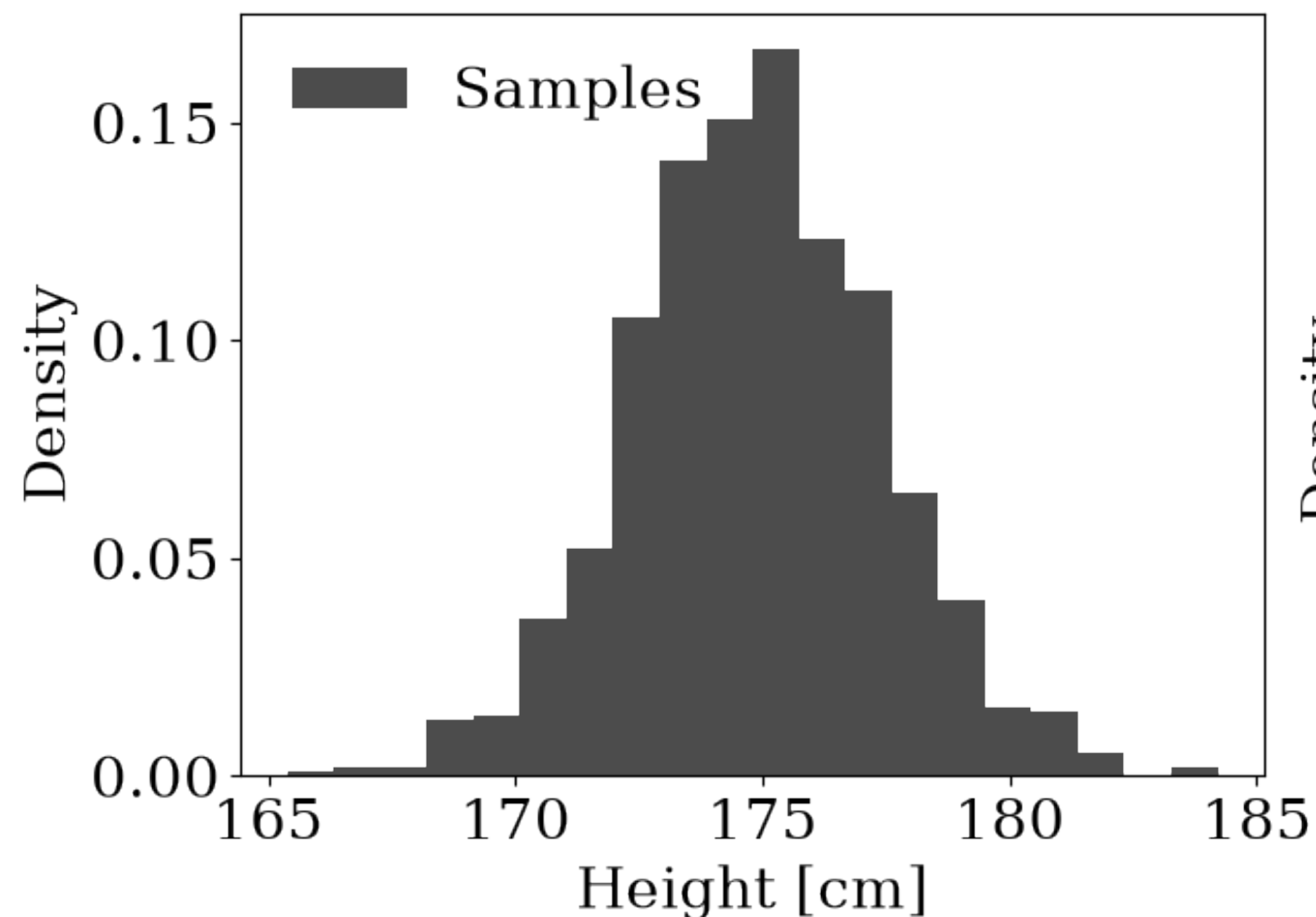
In 1D: $\mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$, where $\left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$



The simplest generative model - Gaussian distribution

Example: height of people

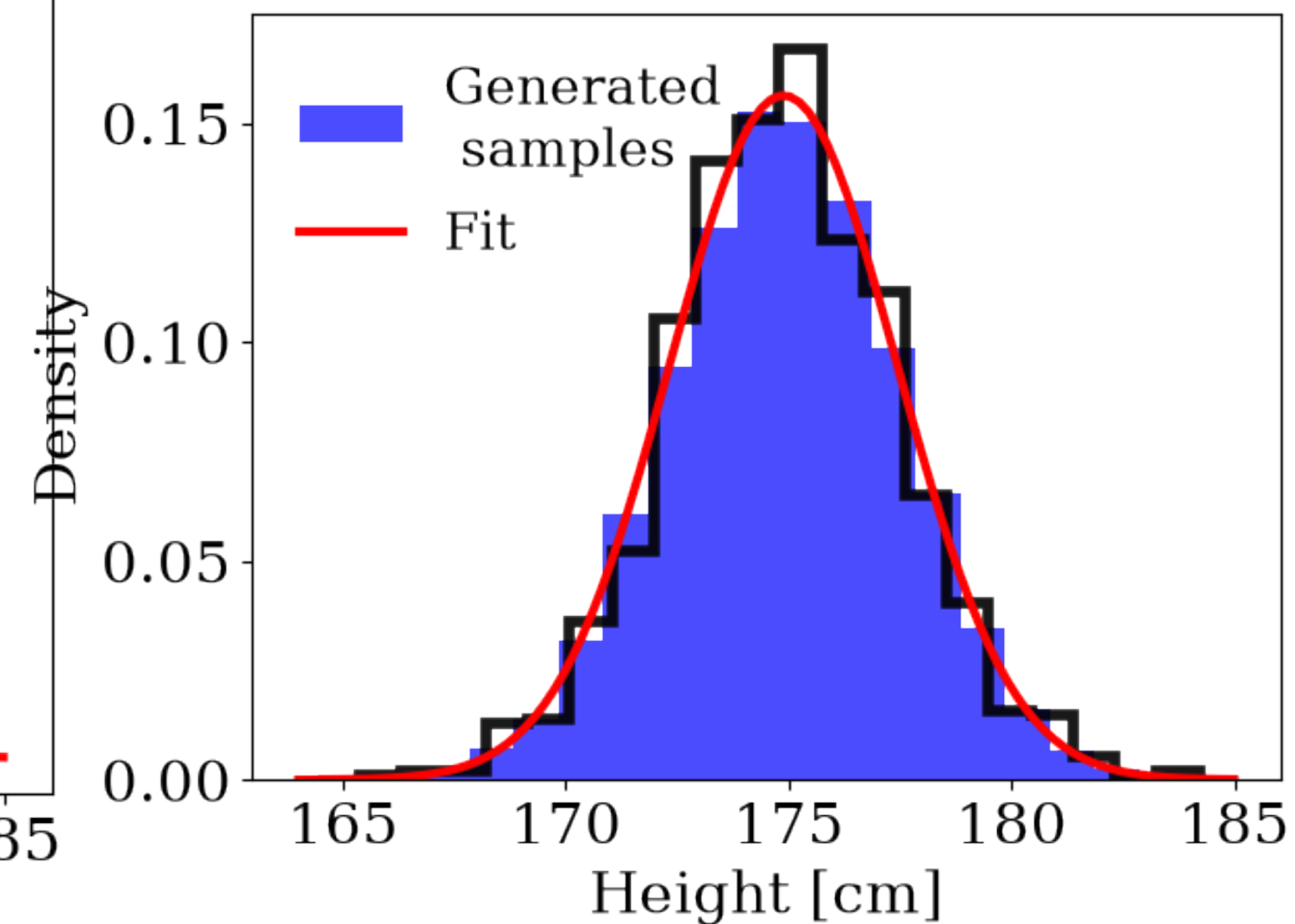
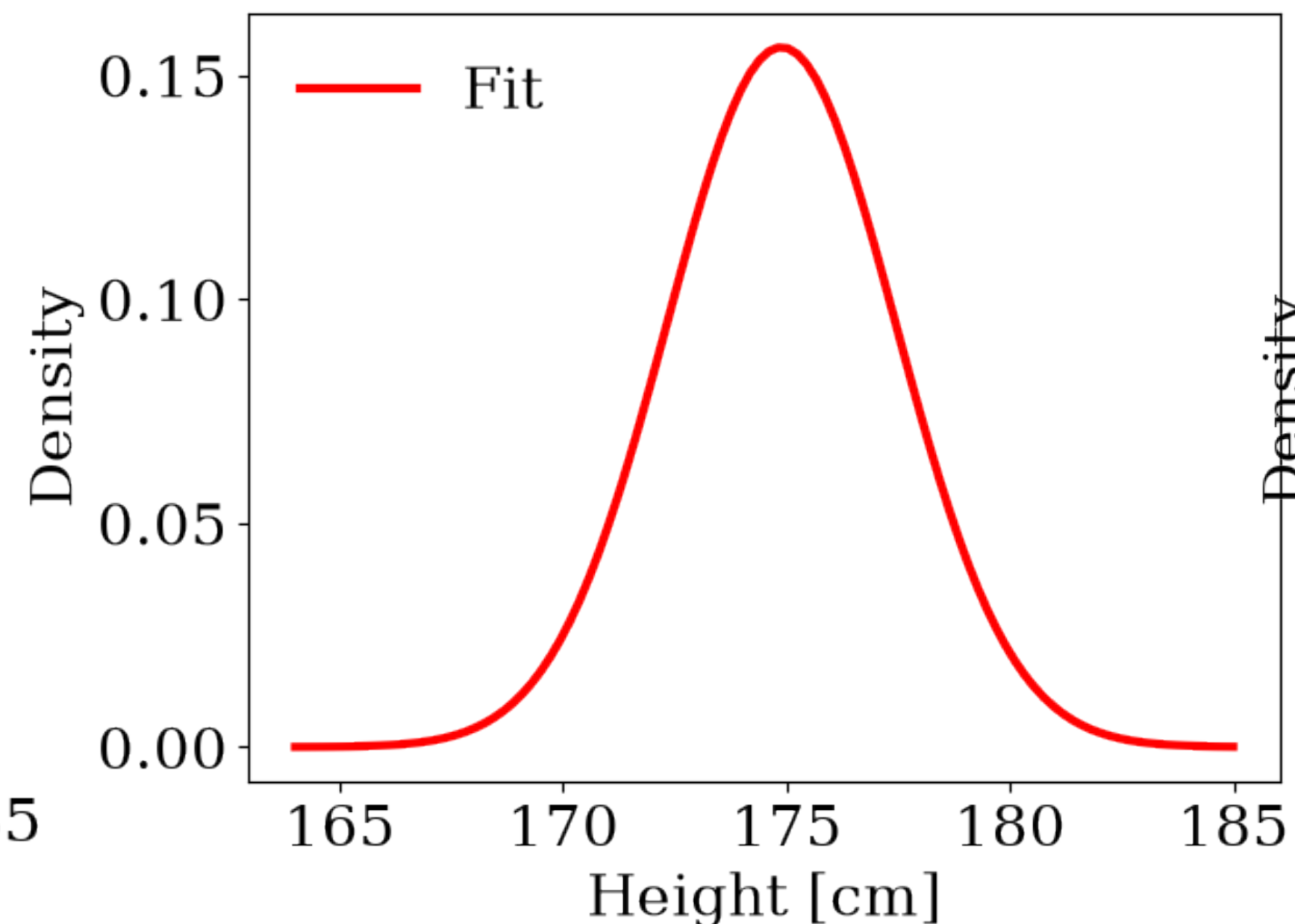
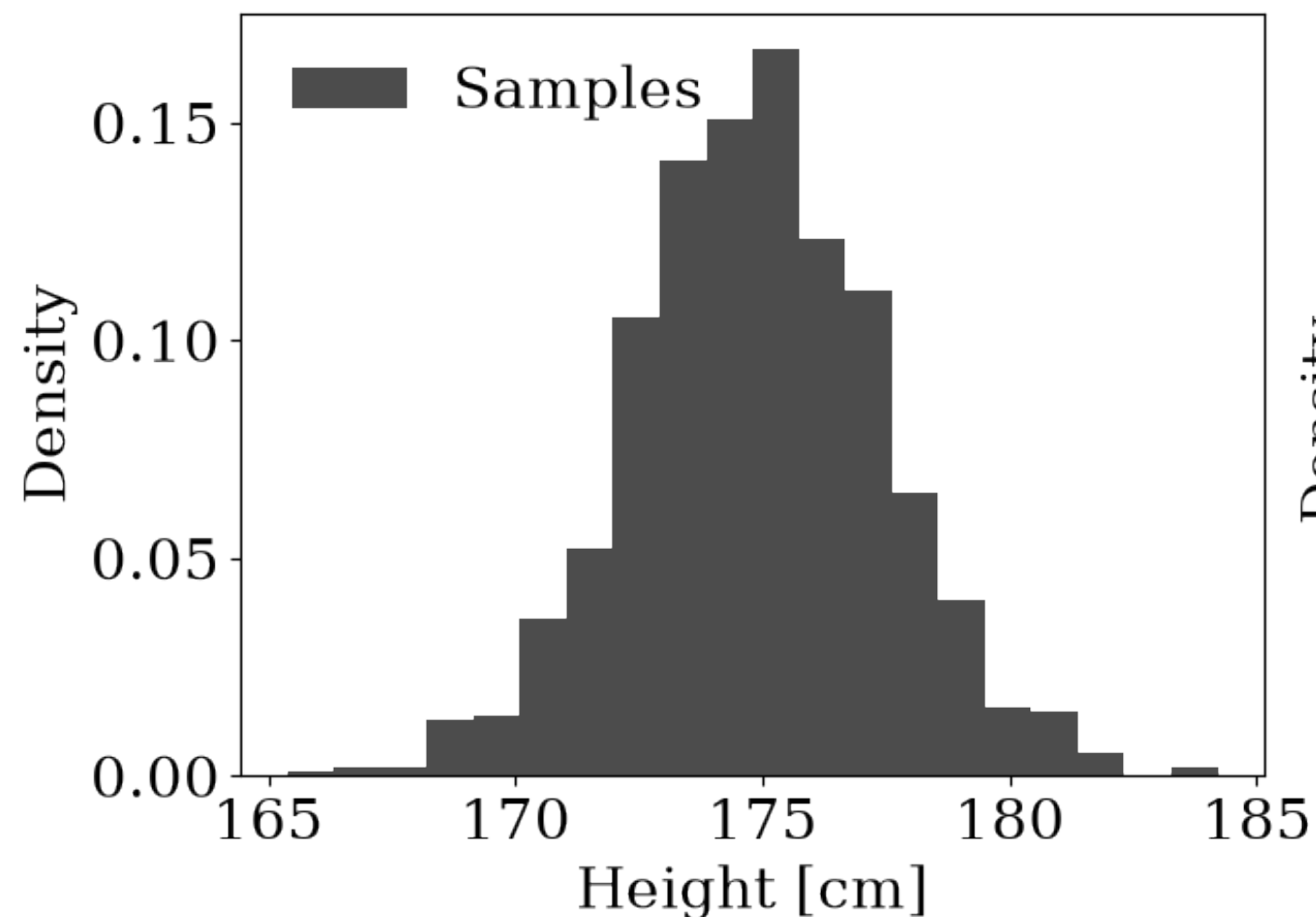
In 1D: $\mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$, where $\left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$



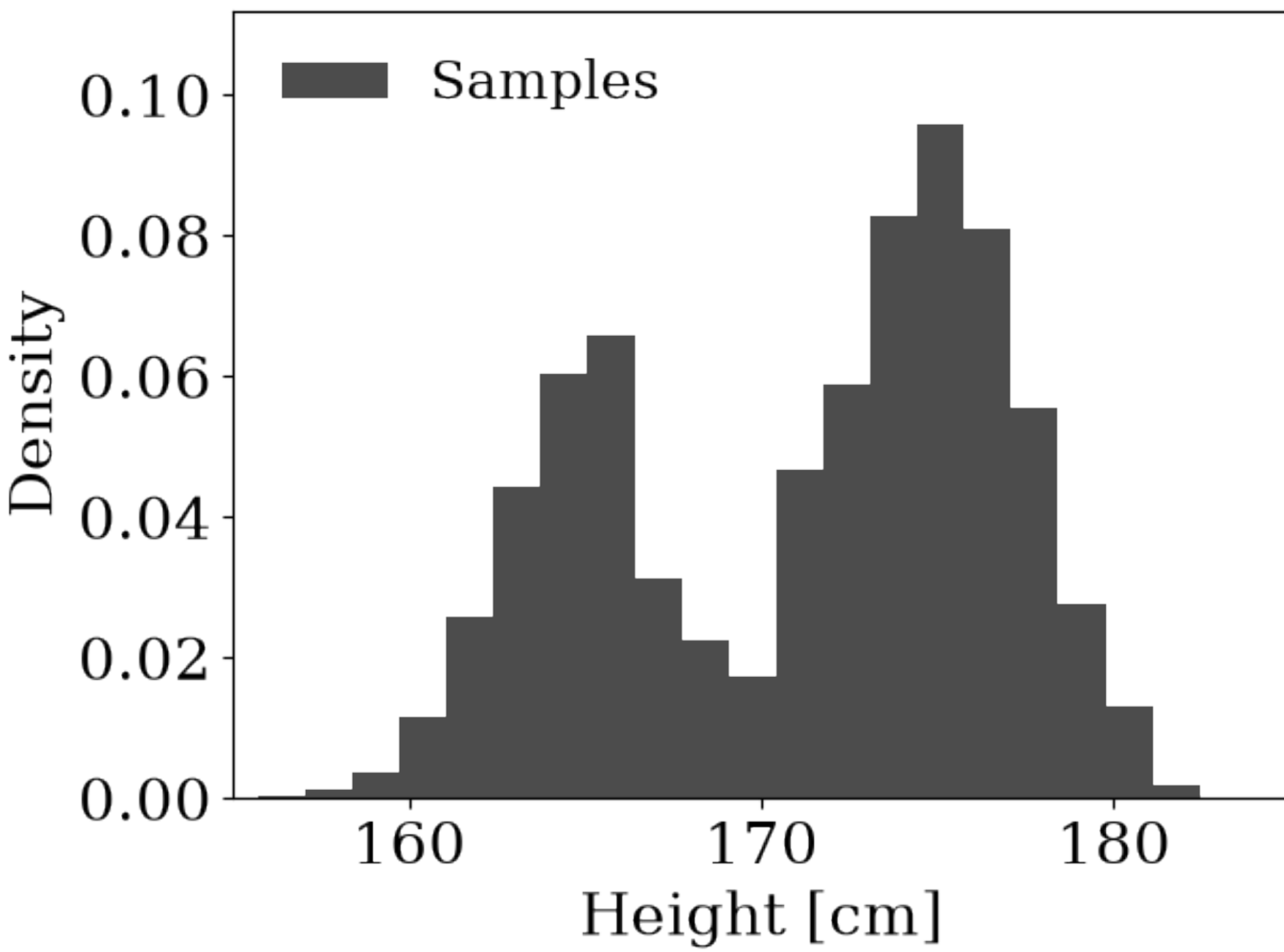
The simplest generative model - Gaussian distribution

Example: height of people

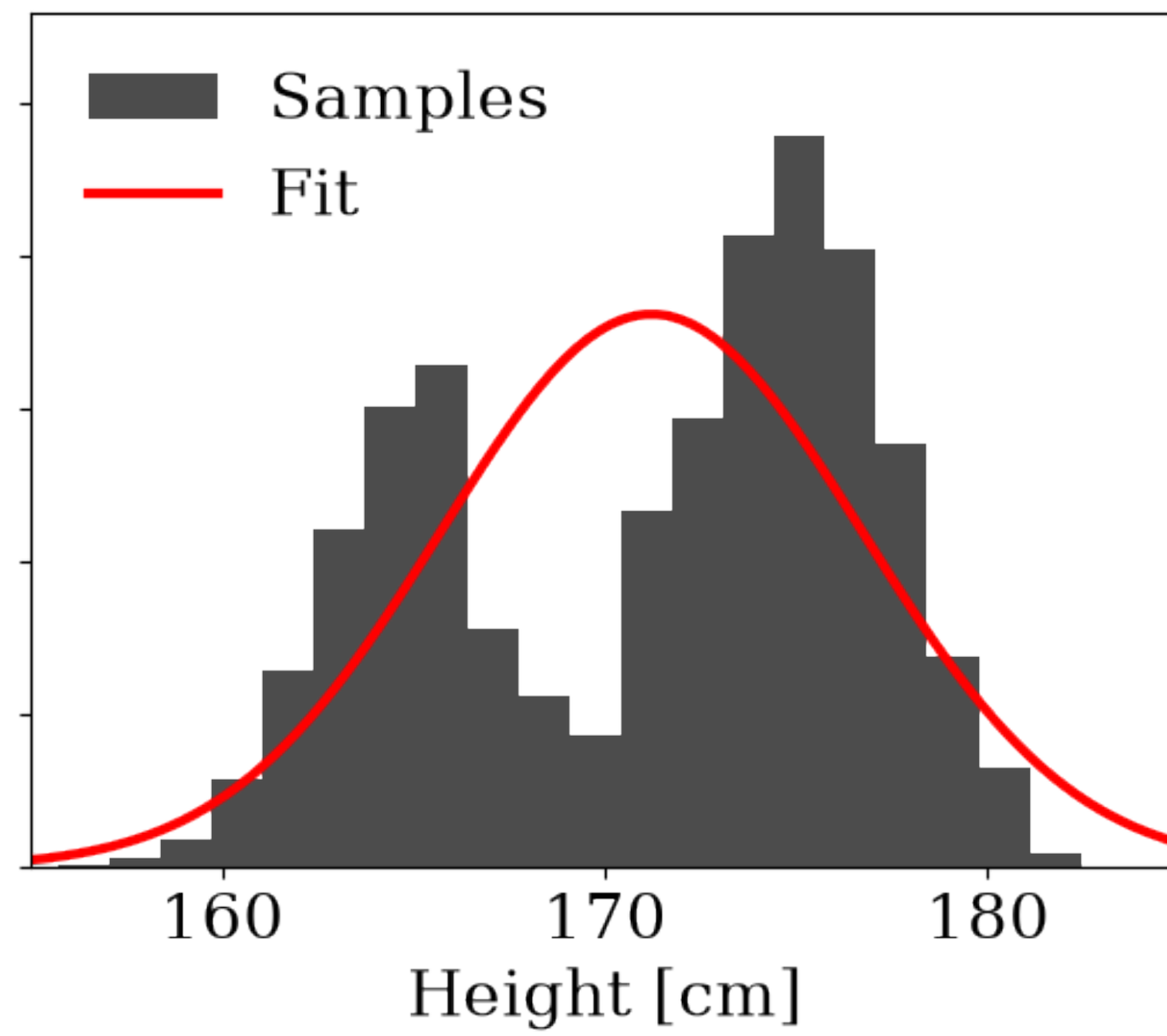
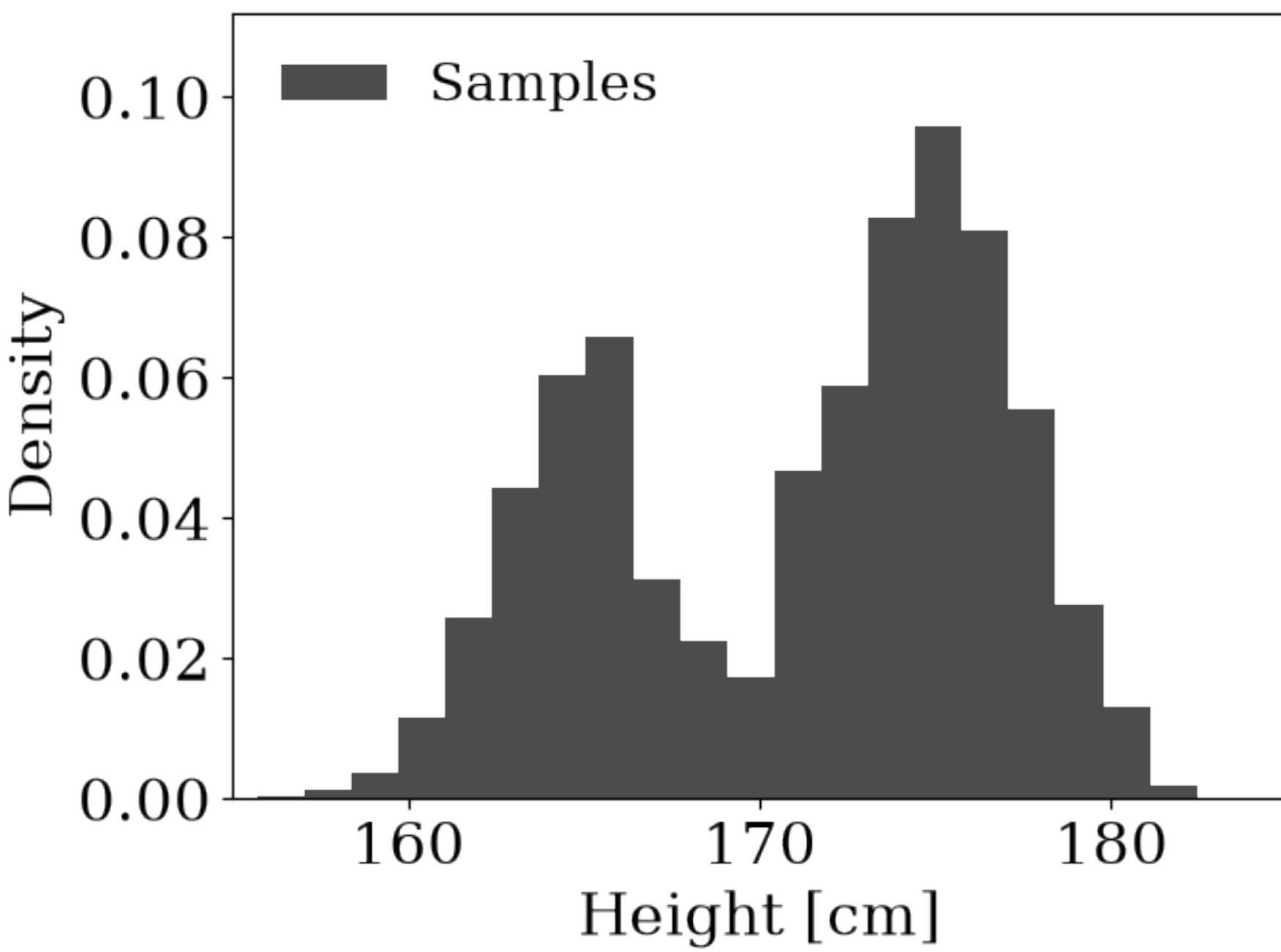
In 1D: $\mathcal{N}(x | \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$, where $\left\{ \begin{array}{l} \mu \approx \frac{1}{N} \sum_i x_i \\ \sigma^2 \approx \frac{1}{N} \sum_i (x_i - \mu)^2 \end{array} \right.$



What if the data is more complex?

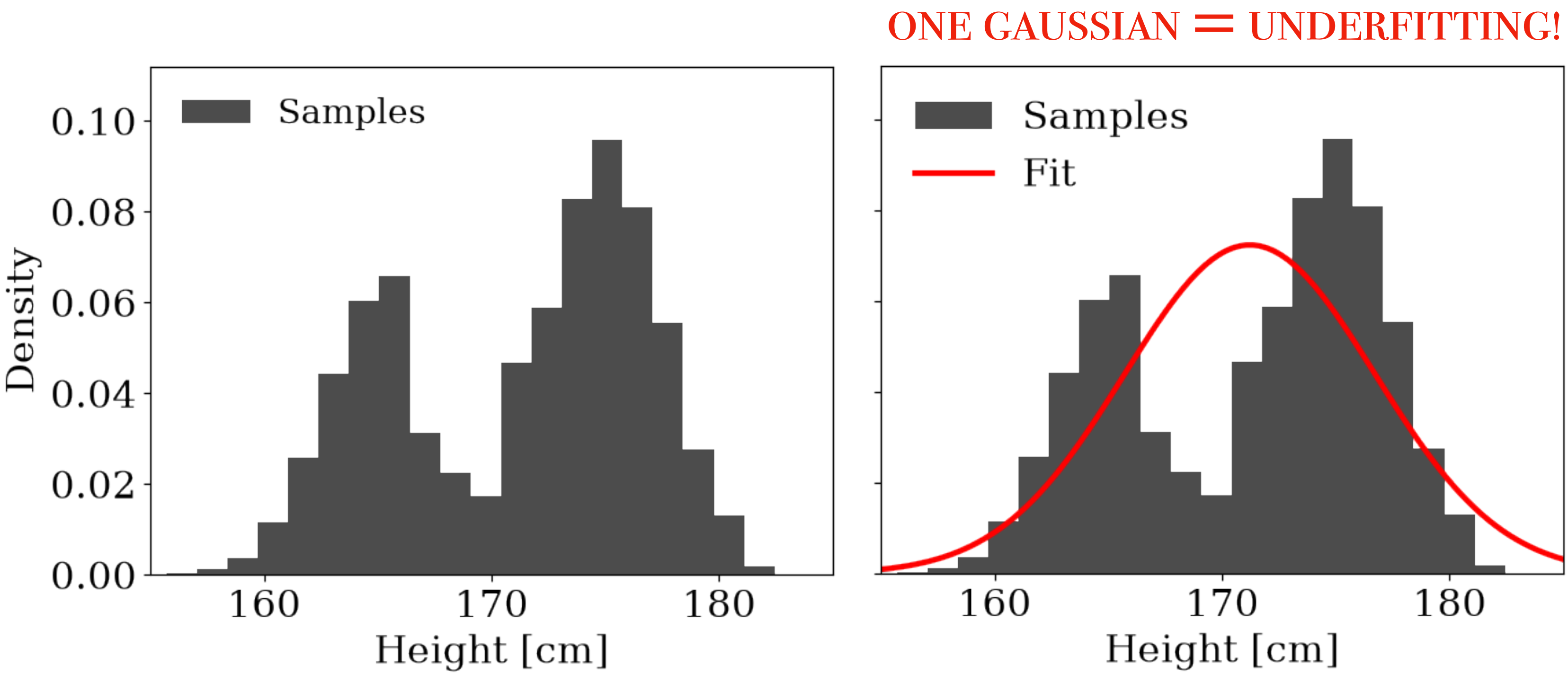


What if the data is more complex?



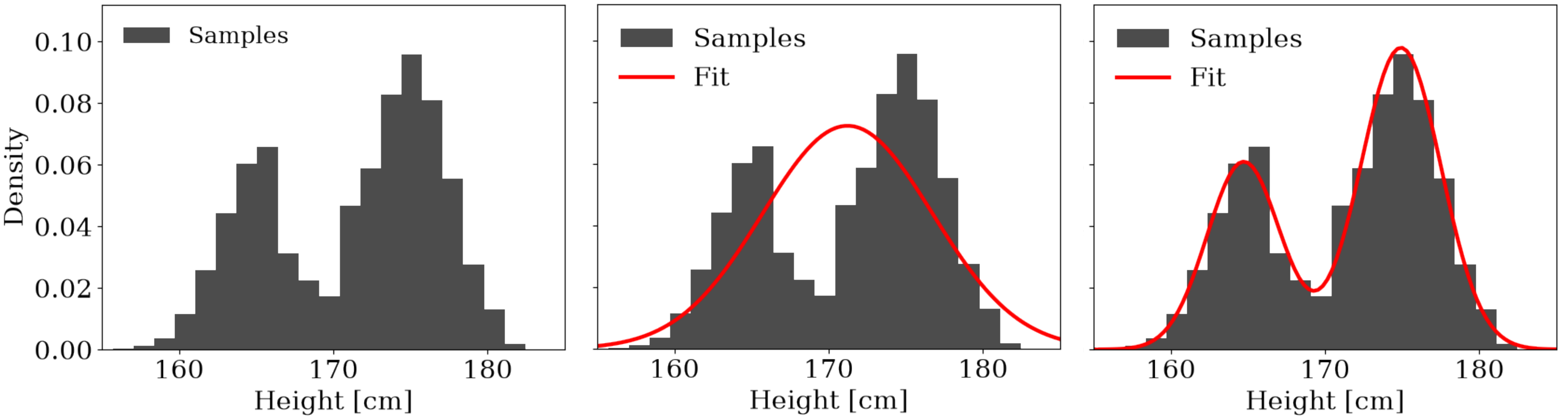
What if the data is more complex?

Is the value at the mean actually typical?



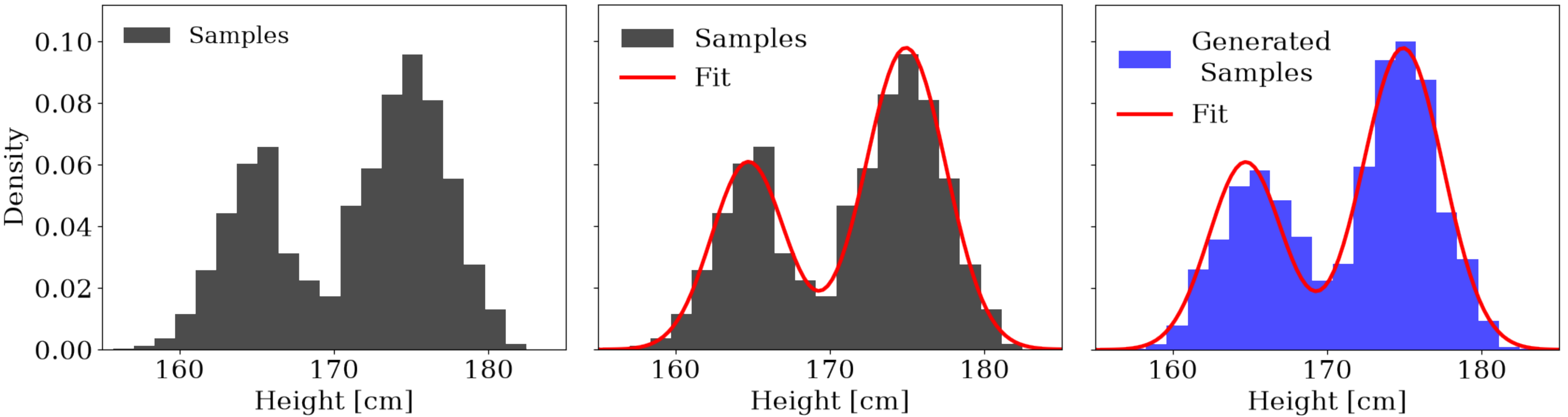
Gaussian mixture model (GMM)

$$P(x) = \sum_{k=1}^K a_k \mathcal{N}(x | \mu_k, \Sigma_k), \text{ where } a_k \geq 0 \text{ and } \sum_{k=1}^K a_k = 1$$



Gaussian mixture model (GMM)

$$P(x) = \sum_{k=1}^K a_k \mathcal{N}(x | \mu_k, \Sigma_k), \text{ where } a_k \geq 0 \text{ and } \sum_{k=1}^K a_k = 1$$



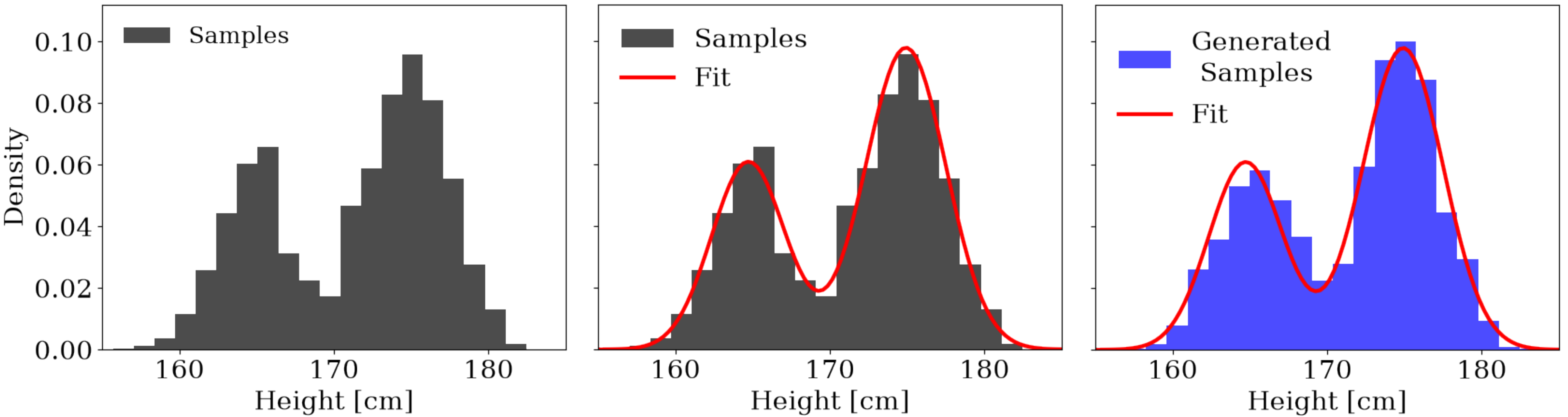
Gaussian mixture model (GMM)

$$P(x) = \sum_{k=1}^K a_k \mathcal{N}(x | \mu_k, \Sigma_k), \text{ where } a_k \geq 0 \text{ and } \sum_{k=1}^K a_k = 1$$

Sampling:

1. Choose component: $k \sim \text{Categorical}(a_1, \dots, a_K)$

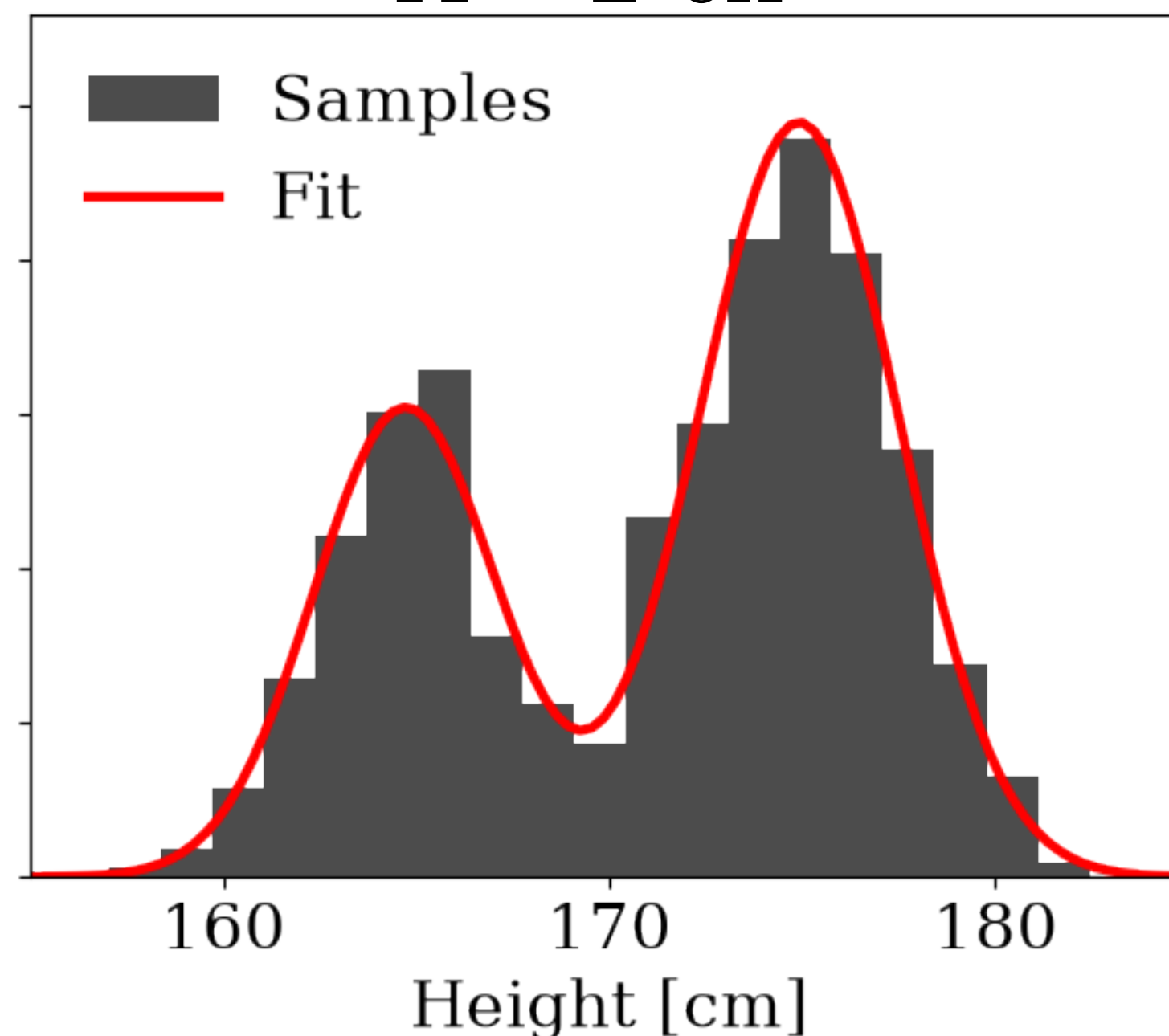
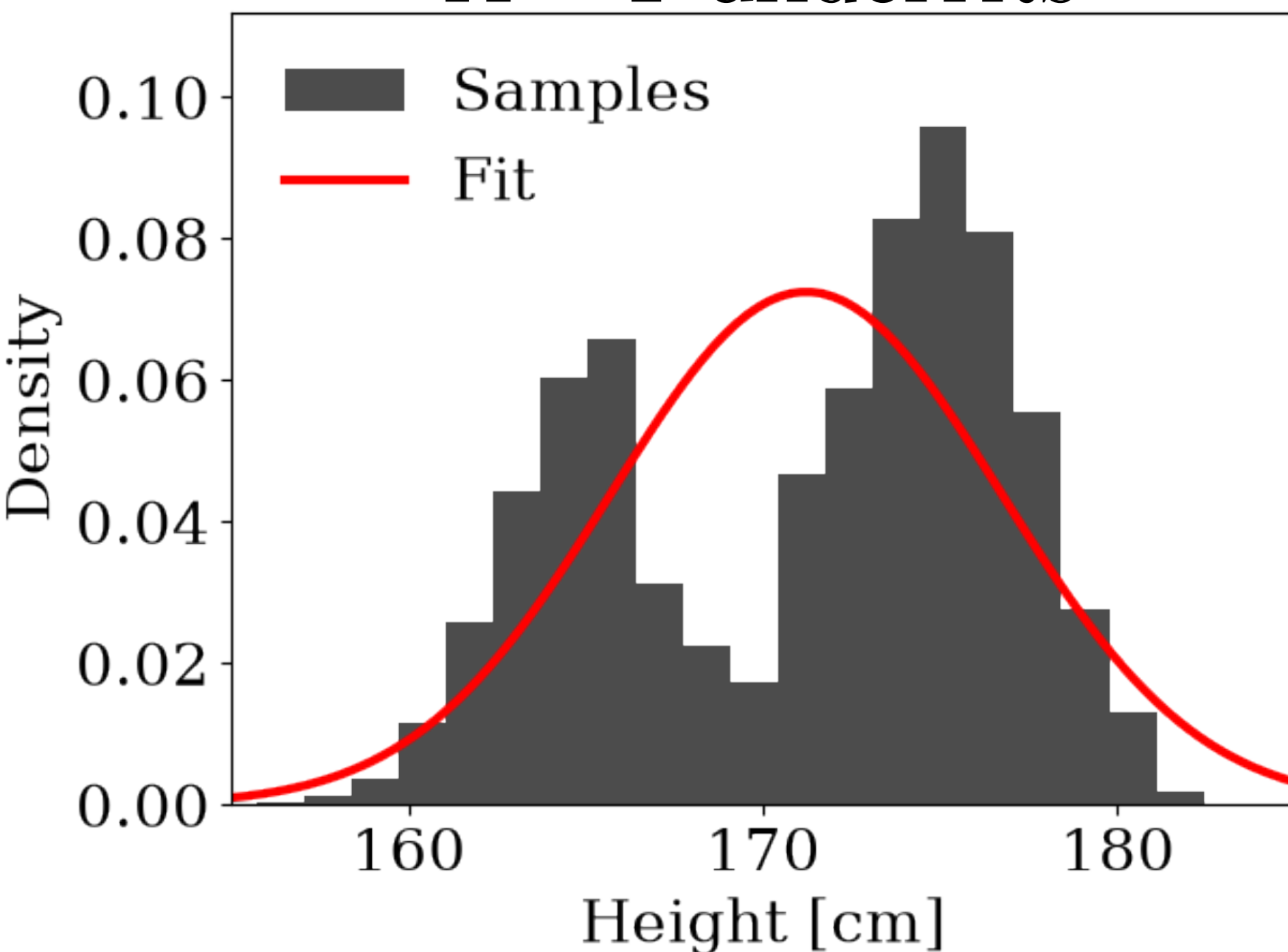
2. Generate a sample: $x \sim \mathcal{N}(\mu_k, \Sigma_k)$



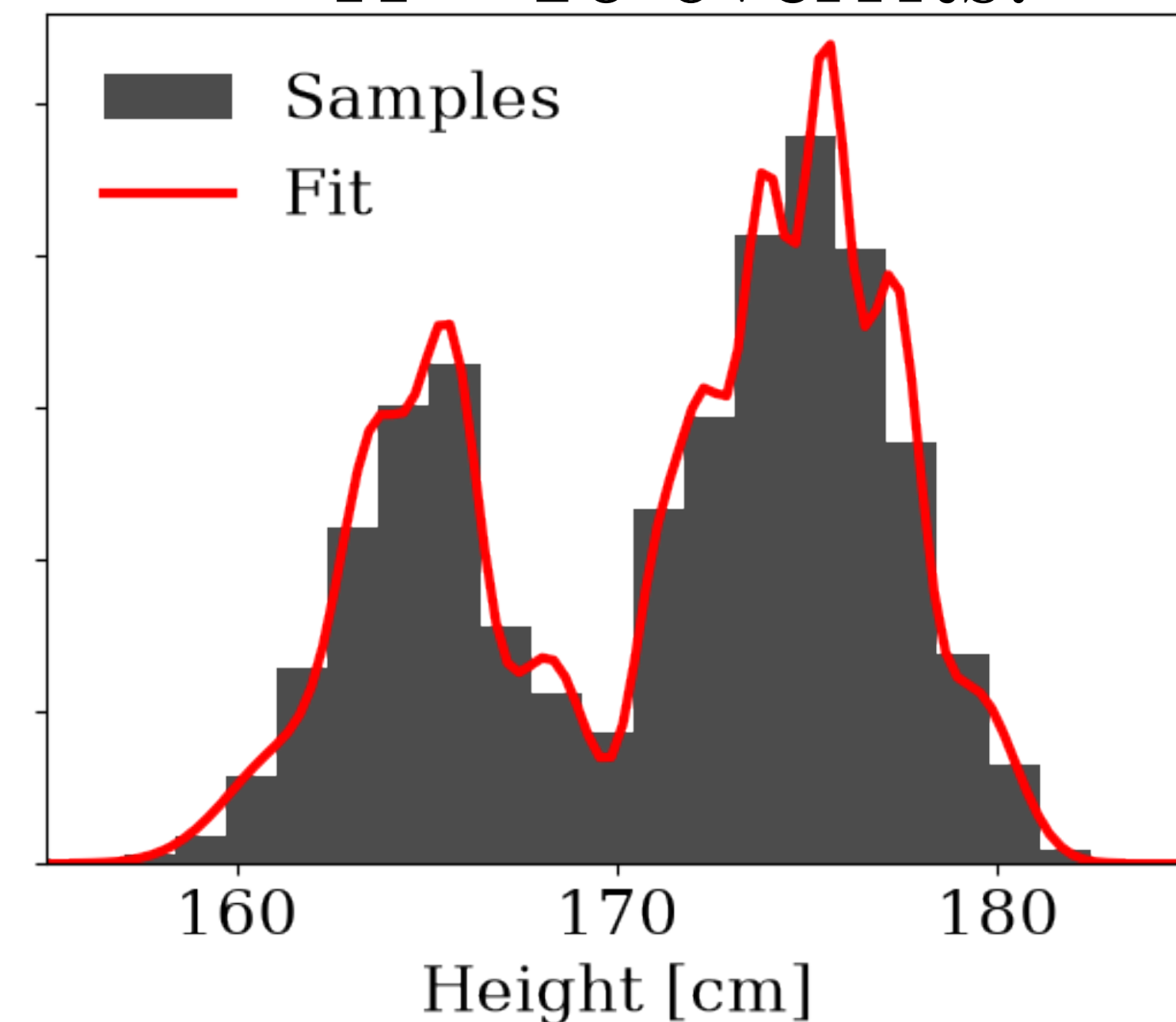
How many Gaussian components?

K is a hyperparameter

- Physical expectations and interpretability provide guidance e.g. for heights: men and women
- *Components are mathematical building blocks $\rightarrow$ may not correspond one-to-one with physical populations
- AIC/BIC $\rightarrow$ guidance for which K is “best”
 - K = 1 underfits
 - K = 2 ok
 - K = 10 overfits!



...



Enter ML...

GMM vs mixture density networks (MDN)

GMM

$$P(x) = \sum_{k=1}^K a_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

- One global distribution
- Unsupervised unconditional density estimation
- Typically fitted with Expectation-Maximization (EM) algorithm

MDN

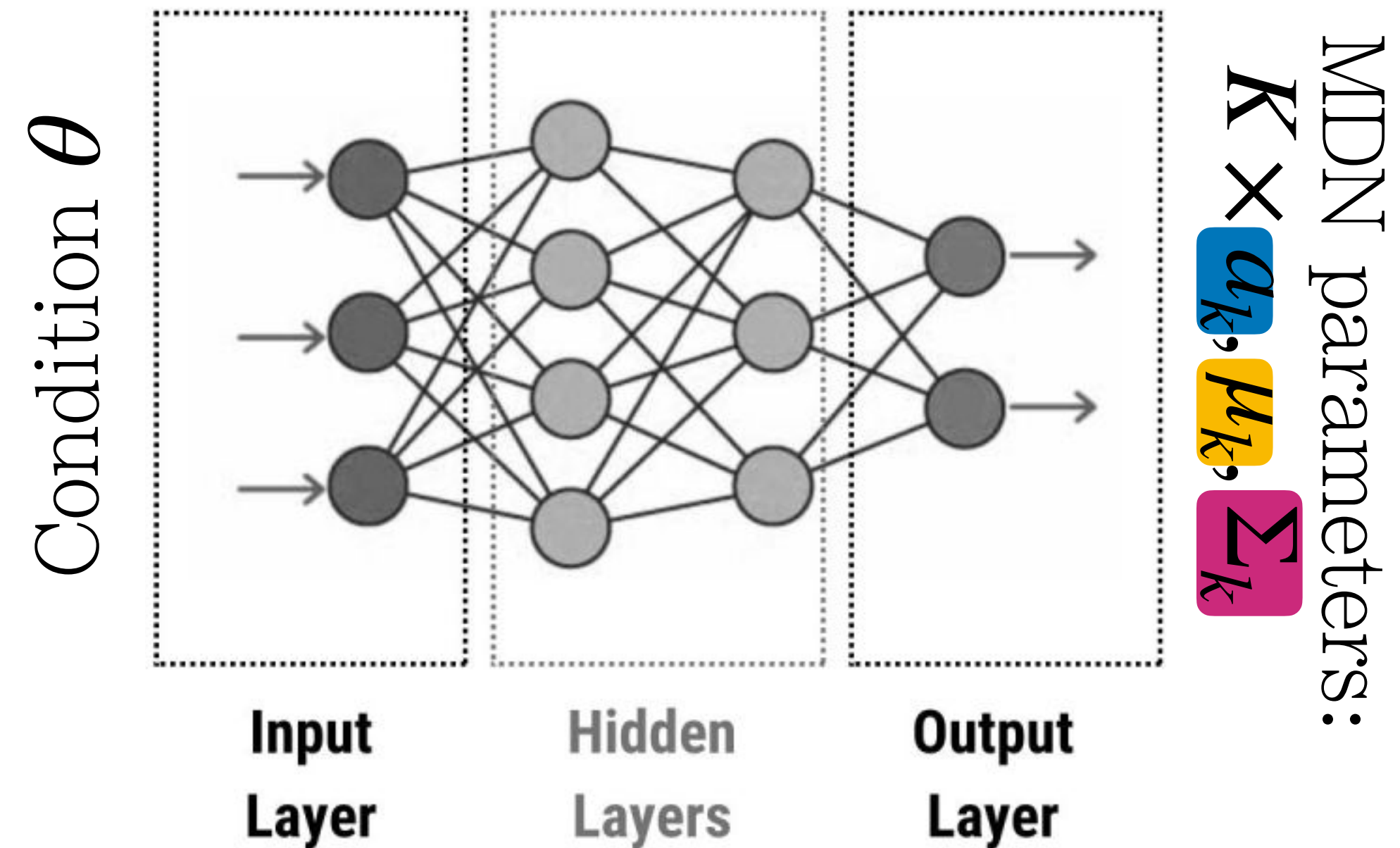
$$P_{\phi}(x | \theta) = \sum_{k=1}^K a_k(\theta) \mathcal{N}(x | \mu_k(\theta), \Sigma_k(\theta))$$

- Distribution changes with input θ
- Supervised conditional density estimation
- Typically trained with gradient descent

Building the MDN in PyTorch

```
n_features = theta.shape(-1)
self.backbone = nn.Sequential(
    nn.Linear(n_features, 128),
    nn.ReLU(),
    nn.Dropout(dropout), #optional
    nn.Linear(128, 128),
    nn.ReLU(),
)
```

```
self.head = nn.Linear(128, K * 3) #K components, each with  $a_k, \mu_k, \Sigma_k$ 
```



Turn raw output into a valid probability distribution

```
raw_a_k, mu_k, raw_sigma = split(raw_output)
```

```
# mu_k already satisfies  $\mu_k \in \mathbb{R}^D$ 
```

```
a_k = torch.softmax(raw_a_k, dim=-1)
```

```
# such that  $a_k \geq 0, \sum_k a_k = 1$ 
```

```
sigma_k = F.softplus(raw_sigma) + sigma_min
```

```
# such that  $\sigma_k > 0$  or positive definite
```

MDN training loss

Negative log-likelihood (NLL)

```
log_component = (
    torch.log(a_k + 1e-8)
    + Normal(mu_k, sigma_k).log_prob(x[..., None])
)

loss = -torch.logsumexp(
    log_component, dim=-1
).mean()
```

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{N} \sum_i \log \left[\sum_{k=1}^K a_k(\theta_i) \mathcal{N}(x_i | \mu_k(\theta_i), \Sigma_k(\theta_i)) \right]$$

LOWER IS BETTER!

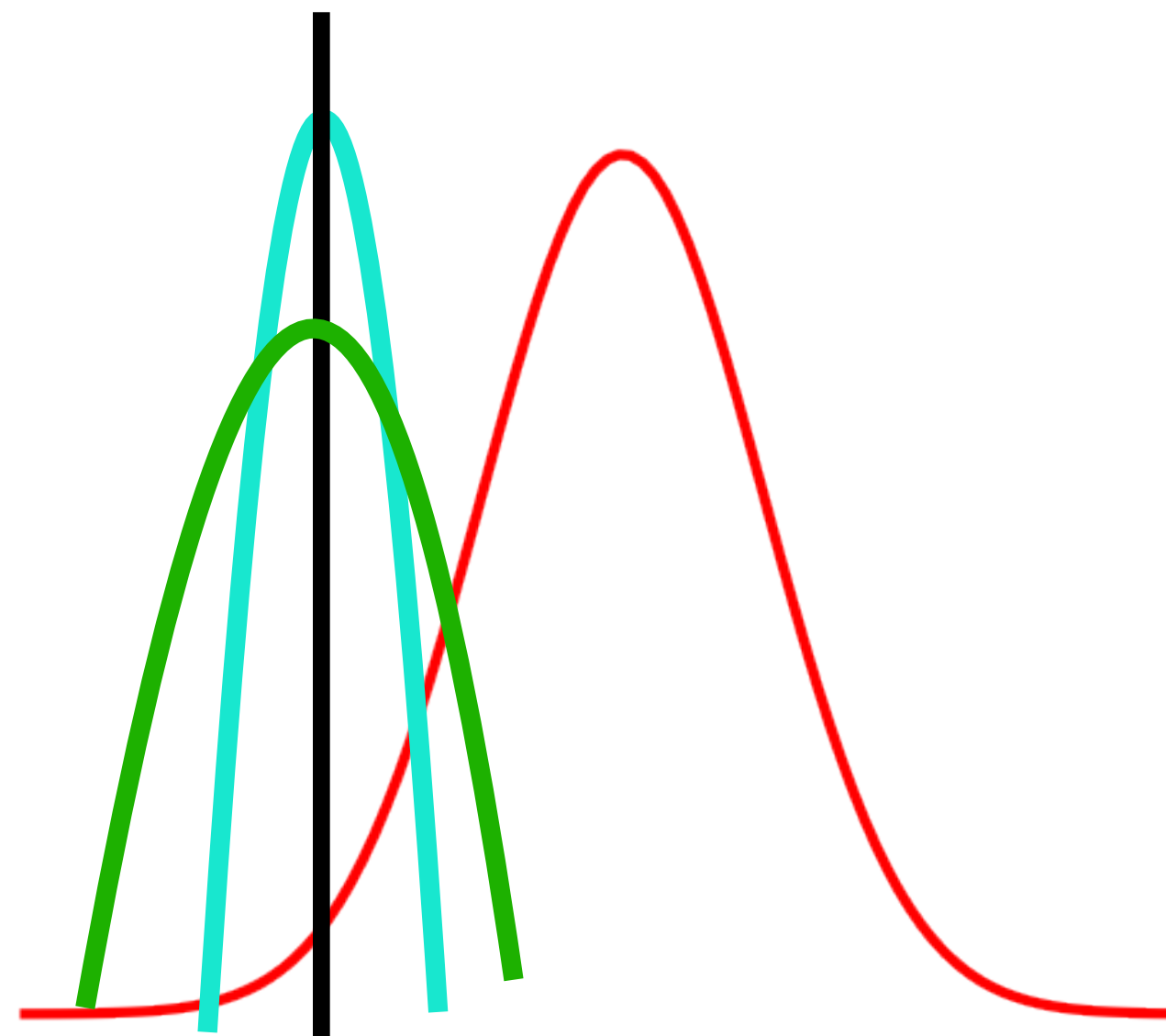
MDN training loss

Negative log-likelihood (NLL)

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{N} \sum_i \log p_{\phi}(y_i | x_i)$$

Suppose we have:

- A TARGET x_i
- PDF fit 1
- PDF fit 2
- PDF fit 3



Q: Rank them. Which PDF has a **BETTER** score for the given target?

Training loop

(see lectures 1-2)

```
start epoch
  sample batch
  predict
  calculate training loss
  update weights
end epoch
calculate validation loss
ask: do we stop training?
```

BATCH: examples used for one parameter update.

EPOCH: one complete pass through the training set.

VALIDATION LOSS: evaluated on examples not used to update the network.

EARLY STOPPING: stop when validation performance no longer improves.

Training loop

(see lectures 1-2)

start epoch

sample batch

predict

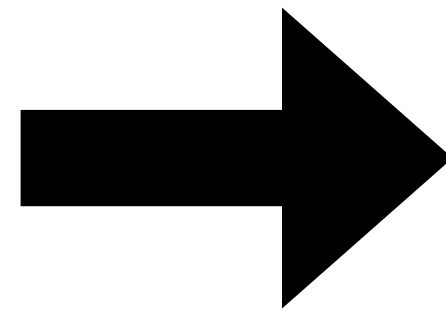
calculate training loss

update weights

end epoch

calculate validation loss

ask: do we stop training?



```
while epoch < max_epochs:  
    for x, y in train_loader:
```

```
        optimizer.zero_grad()
```

```
        loss = mdn_nll(model(x), y)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
    with torch.no_grad(): #no optimizer
```

```
        for x, y in valid_loader:
```

```
            valid_loss = mdn_nll(model(x), y)
```

```
        # stop or keep going?
```

```
    epoch += 1
```


What can we do with a trained MDN?

1. Evaluate the complete PDF
2. Draw samples
3. Calculate summaries e.g. mean, mode, etc.
4. Propagate uncertainty into downstream tasks

How do we validate probabilistic predictions?

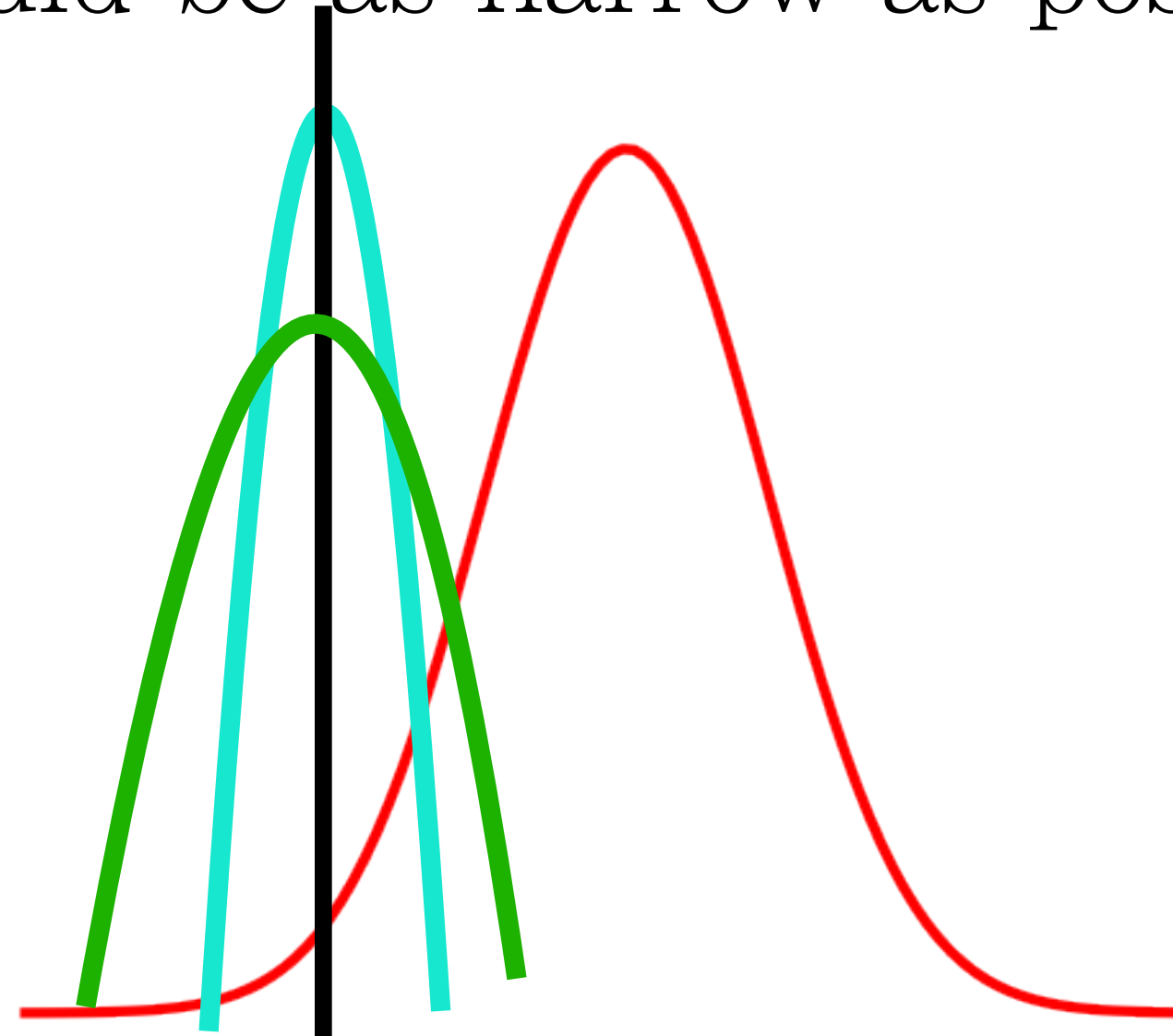
Density accuracy

- NLL: Does the model assign high probability density to outcomes that actually occur?
- Calibration: Among predictions within the 68% interval, approximately 68% of corresponding targets should fall inside.
- Sharpness: A useful model should be as narrow as possible **WHILE REMAINING CALIBRATED.**

How do we validate probabilistic predictions?

Density accuracy

- NLL: Does the model assign high probability density to outcomes that actually occur?
- Calibration: Among predictions within the 68% interval, approximately 68% of corresponding targets should fall inside.
- Sharpness: A useful model should be as narrow as possible **WHILE REMAINING CALIBRATED.**



Outline

PART I

Generative modelling
basics

- Intro to generation
- Simplest generative model: Gaussian
- Gaussian mixture model (GMM)
- Mixture density network (MDN)
- Training procedure

PART II

Image generation

- Variational autoencoder (VAE)
- Generative adversarial network (GAN)
- Diffusion
- Applications

HANDS-ON

Galaxy survey:
Spectroscopy vs
photometry

- SDSS galaxy survey
- Photometry vs spectroscopy
- Notebook setup
- Kaggle

From MDNs to image generation...

Could we predict a Gaussian mixture for every image pixel?

Why not model $p(\text{pixel} | x)$ with a few components?

From MDNs to image generation...

Could we predict a Gaussian mixture for every image pixel?

Why not model $p(\text{pixel} | x)$ with a few components?

K components x 3

From MDNs to image generation...

Could we predict a Gaussian mixture for every image pixel?

Why not model $p(\text{pixel} | x)$ with a few components?

K components x 3 x width x height

From MDNs to image generation...

Could we predict a Gaussian mixture for every image pixel?

Why not model $p(\text{pixel} | x)$ with a few components?

K components x 3 x width x height x 3 $\leftarrow$ (r,g,b)

From MDNs to image generation...

Could we predict a Gaussian mixture for every image pixel?

Why not model $p(\text{pixel} | x)$ with a few components?

K components x 3 x width x height x 3 $\leftarrow$ (r,g,b)



Small 64 x 64 image $\rightarrow$ over 36k parameters with K = 1, and no covariance

From MDNs to image generation...

Could we predict a Gaussian mixture for every image pixel?

Why not model $p(\text{pixel} | x)$ with a few components?

K components $\times 3 \times \text{width} \times \text{height} \times 3 \leftarrow (r, g, b)$

Small 64 x 64 image $\rightarrow$ over 36k parameters with $K = 1$, and no covariance



WHAT WOULD HAPPEN IF EVERY PIXEL WERE GENERATED INDEPENDENTLY?

From MDNs to image generation...

Could we predict a Gaussian mixture for every image pixel?

Why not model $p(\text{pixel} | x)$ with a few components?

K components $\times 3 \times \text{width} \times \text{height} \times 3 \leftarrow (r, g, b)$



Small 64 x 64 image $\rightarrow$ over 36k parameters with $K = 1$, and no covariance

Problems:

- Pixels are strongly correlated
- Independent pixel PDFs lose spatial structure
- A full covariance would require tens of millions of numbers per component
- The number of mixture outputs becomes enormous!

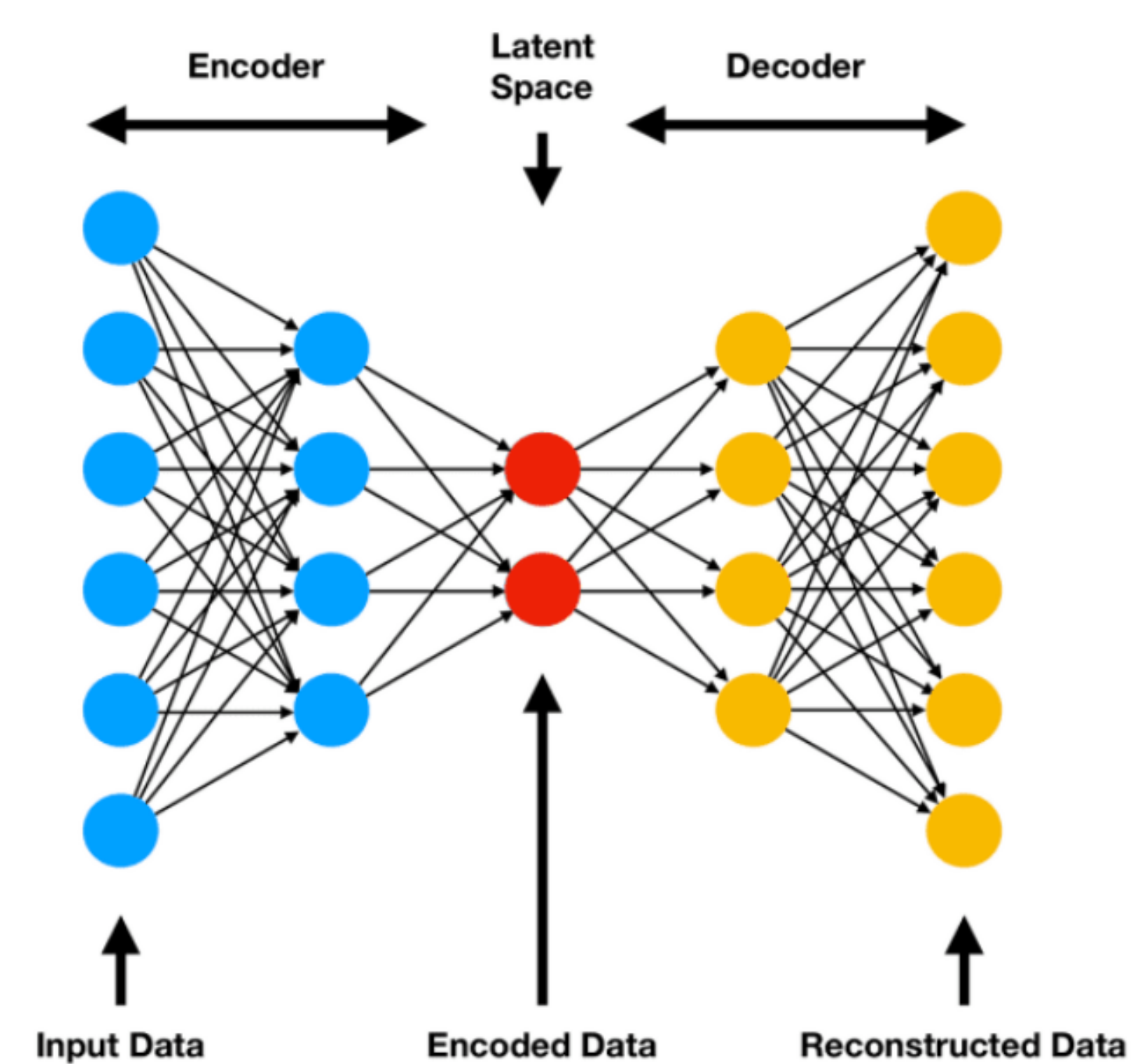
CAN WE LEARN A LOWER-DIMENSIONAL REPRESENTATION OF PLAUSIBLE IMAGES?

Variational autoencoder (VAE)

VAE: put the simple distribution in latent space

Training has two goals:

1. Reconstruction loss: reconstruct the input image
2. Latent regularization: impose a structure on the latent space z so that it can be sampled after training



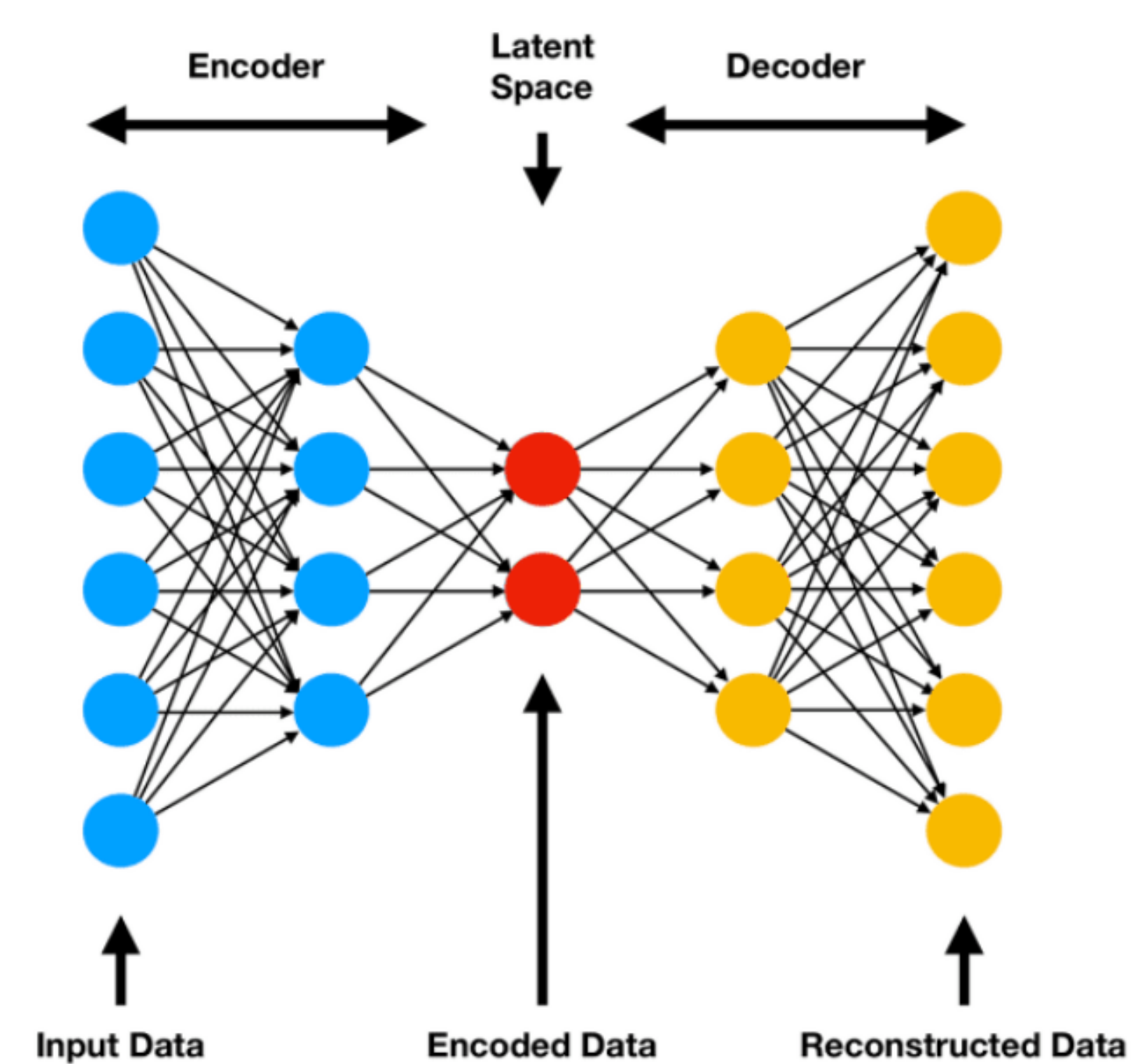
Variational autoencoder (VAE)

VAE: put the simple distribution in latent space

Training has two goals:

1. Reconstruction loss: reconstruct the input image
2. Latent regularization: impose a structure on the latent space z so that it can be sampled after training

WHERE IS THE GAUSSIAN IN THE VAE?



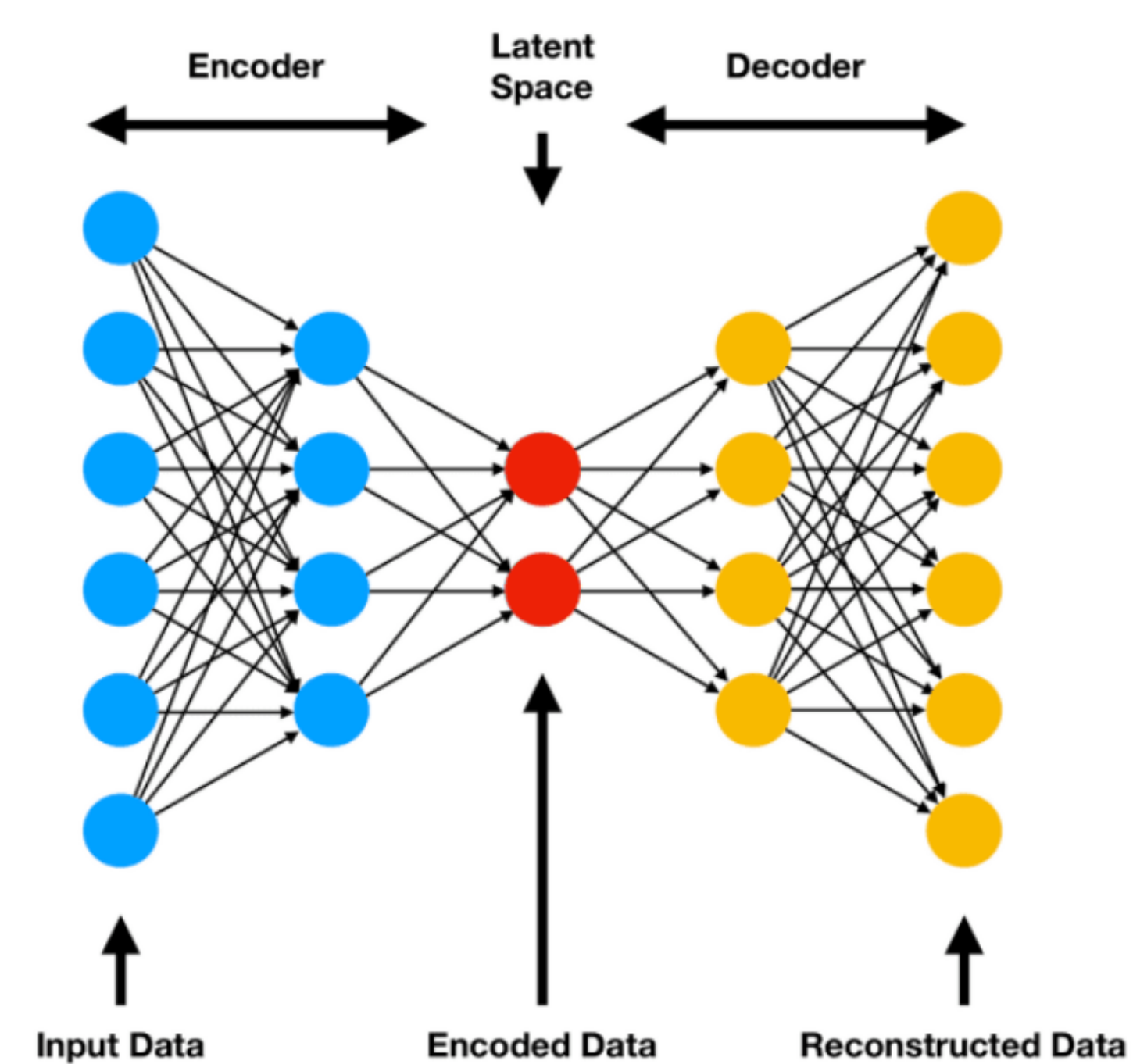
Variational autoencoder (VAE)

VAE: put the simple distribution in latent space

Training has two goals:

1. Reconstruction loss: reconstruct the input image
2. Latent regularization: impose a structure on the latent space z so that it can be sampled after training

Generation: $z \sim \mathcal{N}(0, I)$, $p_{\theta}(x | z)$



Variational autoencoder (VAE)

VAE: put the simple distribution in latent space

Training has two goals:

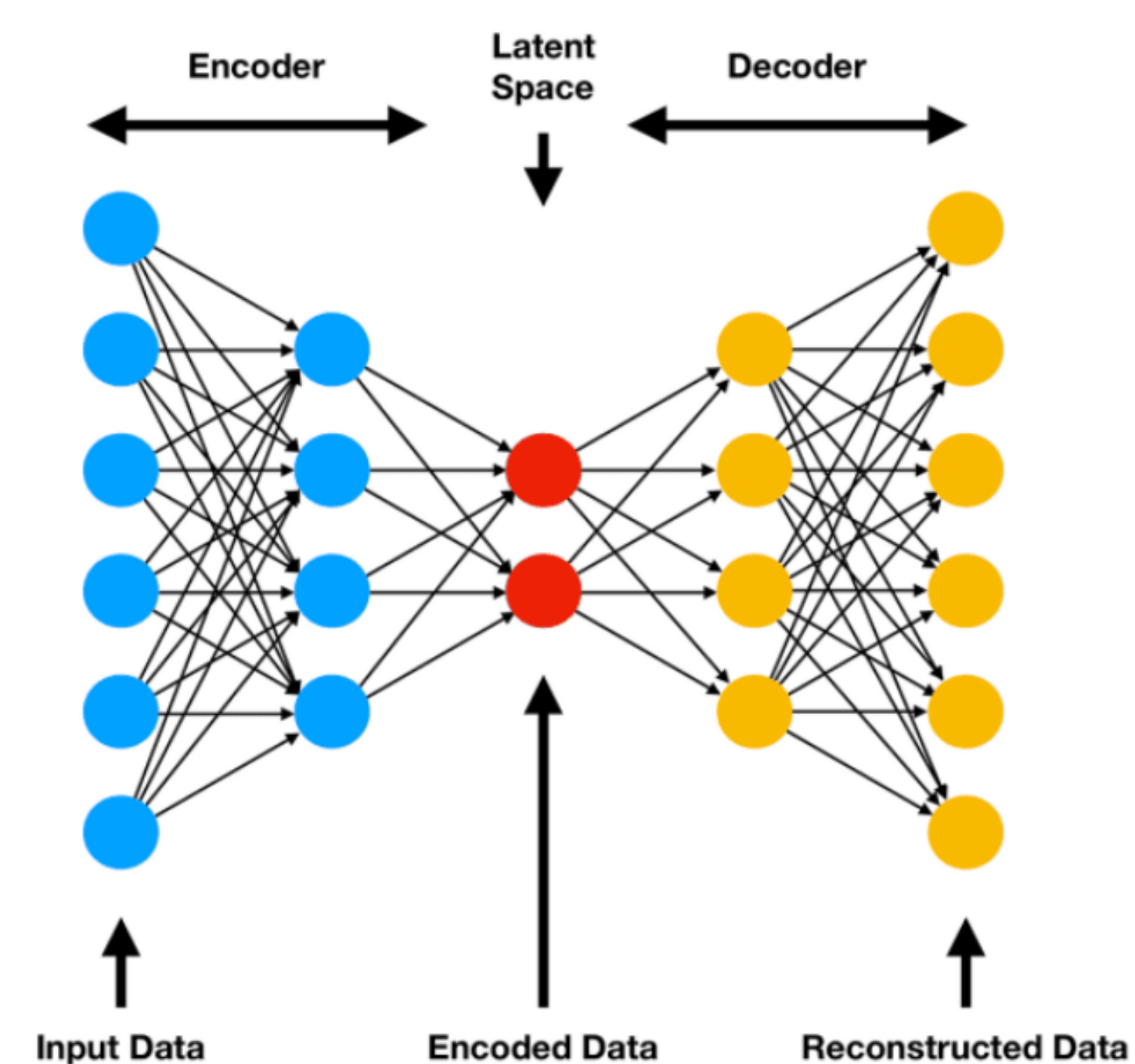
1. Reconstruction loss: reconstruct the input image
2. Latent regularization: impose a structure on the latent space z so that it can be sampled after training

Generation: $z \sim \mathcal{N}(0, I)$, $p_{\theta}(x | z)$

Strengths:

- Relatively stable training
- Direct image-to-latent encoder
- Useful latent representation (i.e. interpretable sometimes)

Limitation: simple VAEs can produce overly smooth images.

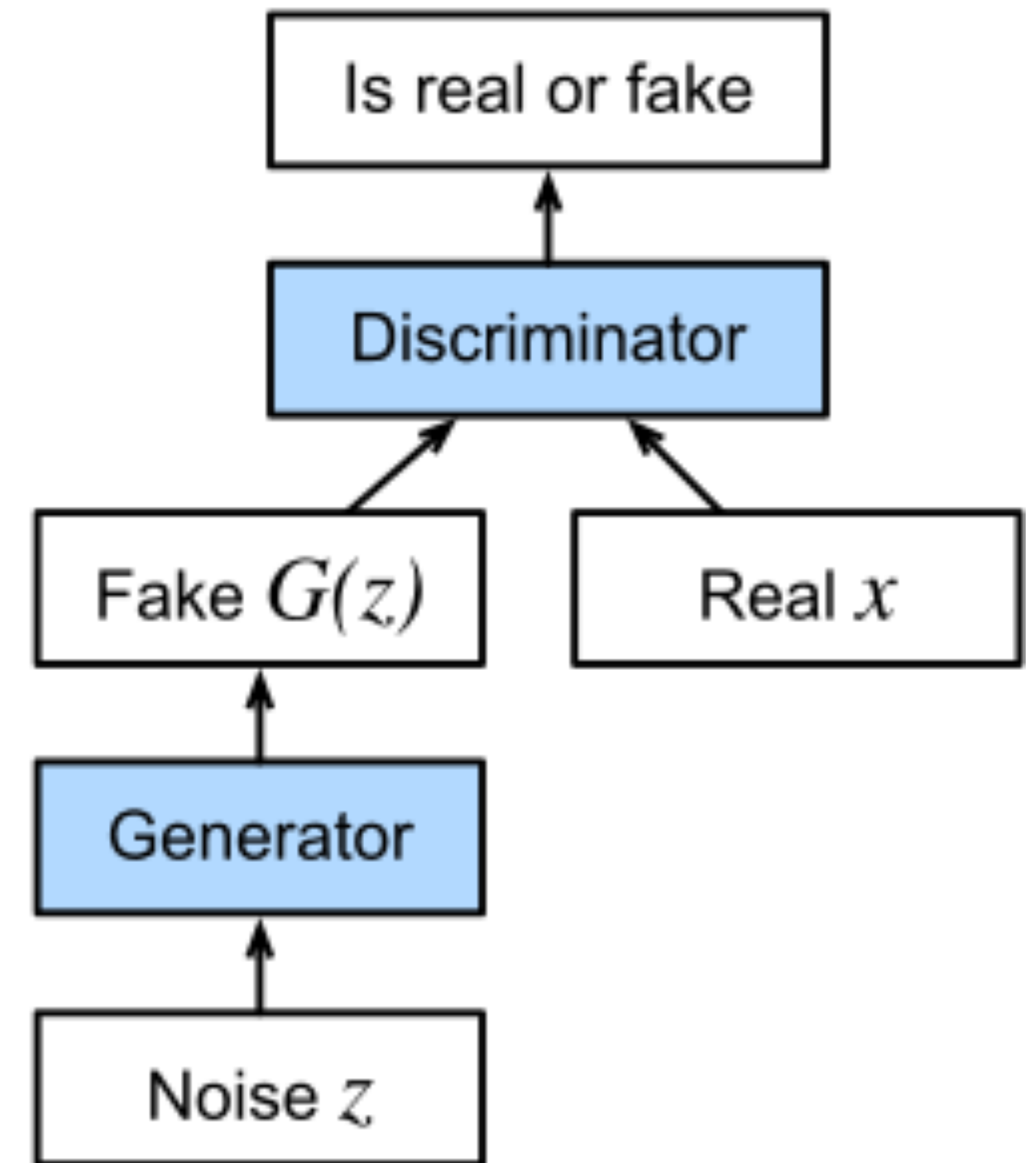


Generative adversarial networks (GANs)

Generator vs classifier

Learn the sampling process $\tilde{x} \sim p(x)$

- Generator net: $z \rightarrow G_{\phi}(z) \rightarrow \text{fake image}$
- Discriminator net: $D_{\phi}(x) \rightarrow P(\text{real})$



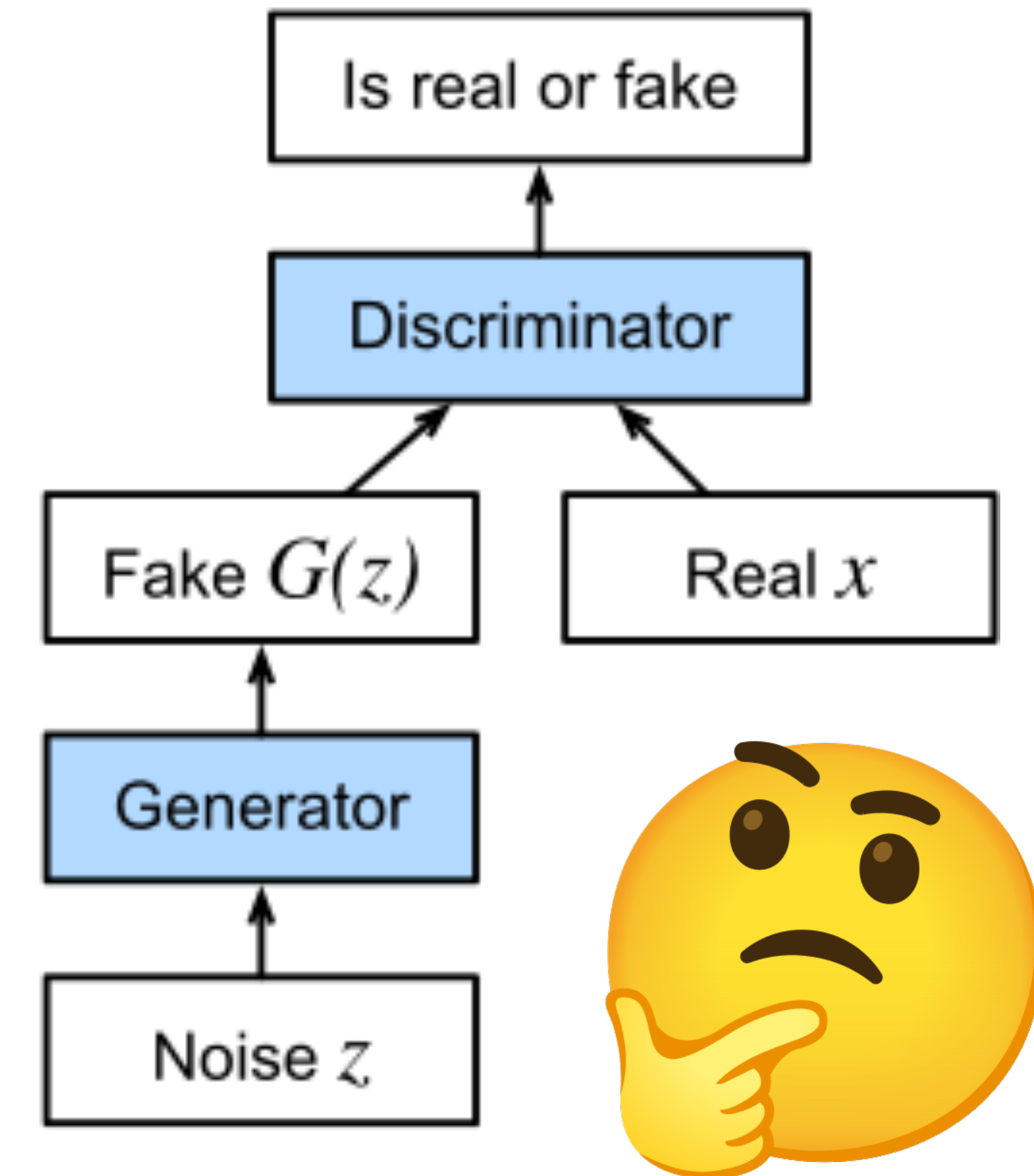
Generative adversarial networks (GANs)

Generator vs classifier

Learn the sampling process $\tilde{x} \sim p(x)$

- Generator net: $z \rightarrow G_\phi(z) \rightarrow \text{fake image}$
- Discriminator net: $D_\phi(x) \rightarrow P(\text{real})$

WHICH PART OF A GAN IS A CLASSIFIER?



Generative adversarial networks (GANs)

Generator vs classifier

Learn the sampling process $\tilde{x} \sim p(x)$

- Generator net: $z \rightarrow G_\phi(z) \rightarrow \text{fake image}$
- Discriminator net: $D_\phi(x) \rightarrow P(\text{real}) \rightarrow \text{binary classifier}$

Training: generator tries to fool the discriminator

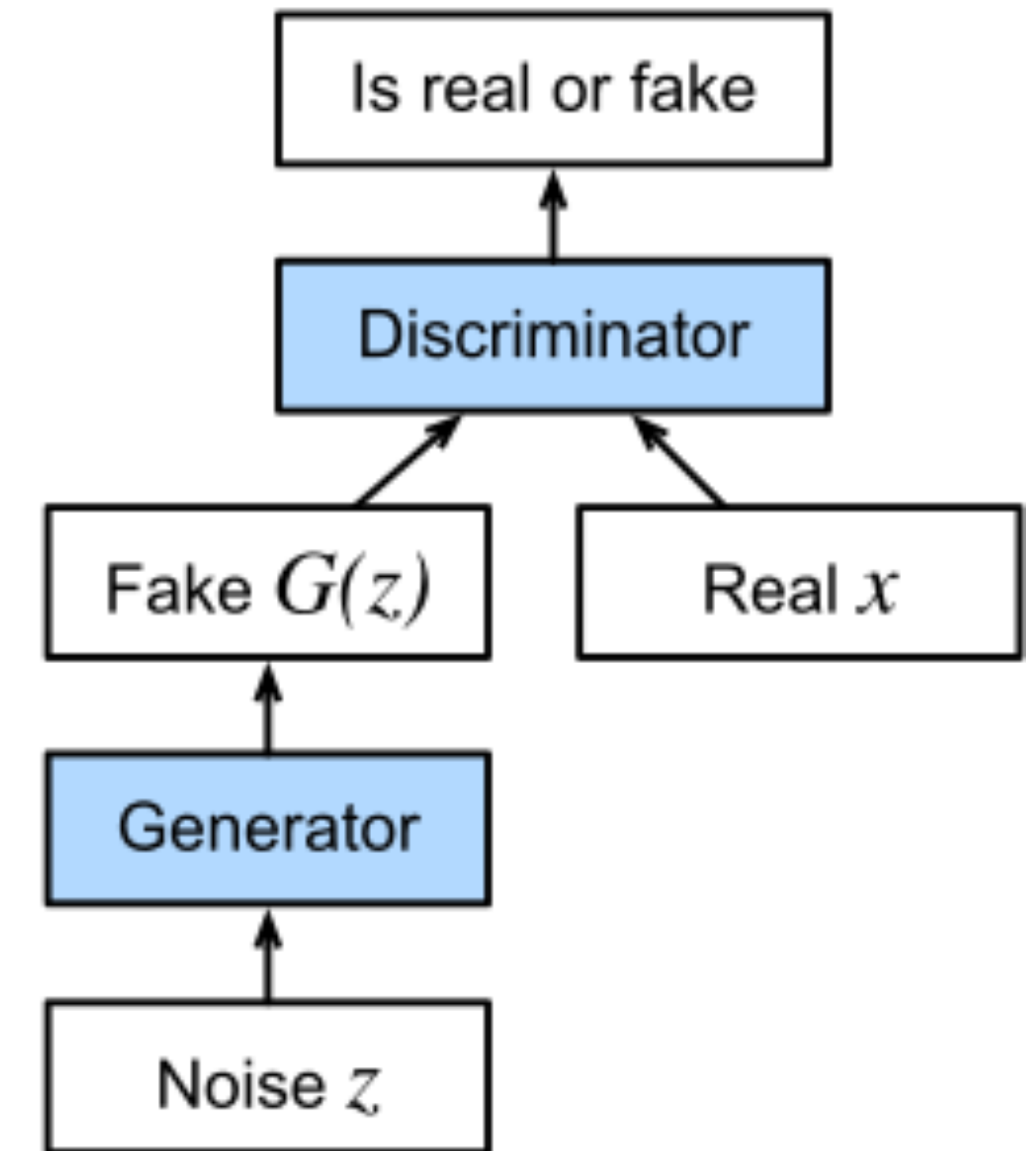
➡ Fast sampling

➡ Sharp images ($\neq$ correct distribution!)

* Can be highly unstable

* Mode collapse i.e. insufficient diversity

State-of-the-art in image generation for a long time



Different models learn through different targets

<u>MODEL</u>	<u>WHAT THE NETWORK PREDICTS</u>	<u>TRAINING LOSS</u>
MDN	Parameters of an explicit PDF	NLL
VAE	Latent representation and reconstruction	Reconstruction + latent regularization

Different models learn through different targets

<u>MODEL</u>	<u>WHAT THE NETWORK PREDICTS</u>	<u>TRAINING LOSS</u>
MDN	Parameters of an explicit PDF	NLL
VAE	Latent representation and reconstruction	Reconstruction + latent regularization

Different models learn through different targets

<u>MODEL</u>	<u>WHAT THE NETWORK PREDICTS</u>	<u>TRAINING LOSS</u>
MDN	Parameters of an explicit PDF	NLL
VAE	Latent representation and reconstruction	Reconstruction + latent regularization
GAN	Samples that fool a classifier	Adversarial loss

Different models learn through different targets

<u>MODEL</u>	<u>WHAT THE NETWORK PREDICTS</u>	<u>TRAINING LOSS</u>
MDN	Parameters of an explicit PDF	NLL
VAE	Latent representation and reconstruction	Reconstruction + latent regularization
GAN	Samples that fool a classifier	Adversarial loss

**CAN WE GENERATE COMPLEX DATA WITHOUT EXPLICITLY WRITING
DOWN THE FULL IMAGE DENSITY OR TRAINING A DISCRIMINATOR?**

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$



Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- The score can be parameterised using standard neural architectures without requiring an invertible network or an adversarial discriminator!
- No adversarial training needed
- Stable training



Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right]$$

WHY CAN WE NOT CALCULATE THIS LOSS?



Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \boxed{\nabla_x \log p_\theta(x)} \right\|_2^2 \right]$$

→ Score approximated by NN

WHY CAN WE NOT CALCULATE THIS LOSS?



Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right]$$

➡ Score of the data distribution is unknown!

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right]$$

➡ Goal: eliminate dependence on unknown data score.

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] =$$

Assume 1D RV

$$= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right]$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\begin{aligned} \frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] &= \\ \text{Assume 1D RV} &= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right] \\ \text{Definition of expected value} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx \end{aligned}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\begin{aligned} \frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] &= \\ \text{Assume 1D RV} &= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right] \\ \text{Definition of expected value} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx \\ \text{Do the square} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) \right)^2 dx + \frac{1}{2} \int p(x) \left(\nabla_x \log p_\theta(x) \right)^2 dx \\ &\quad - \int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx \end{aligned}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] =$$

Assume 1D RV

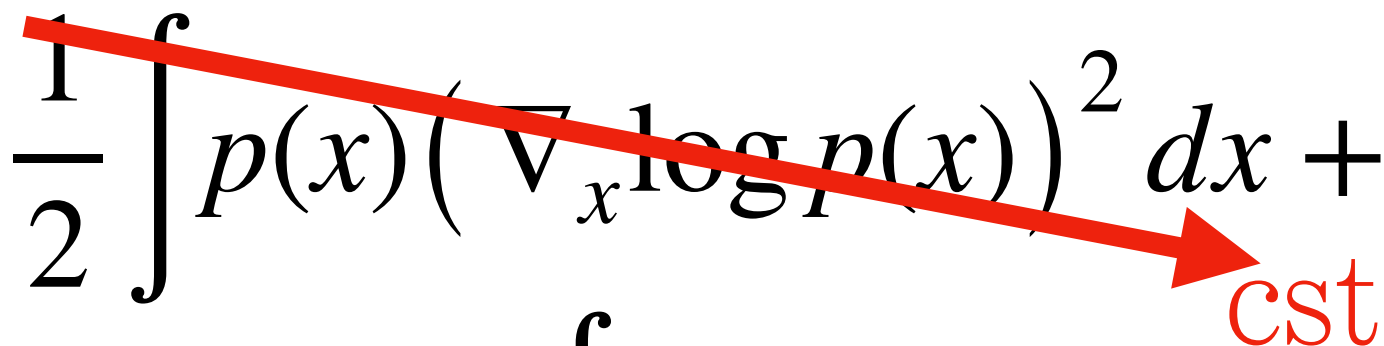
$$= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right]$$

Definition of expected value

$$= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx$$

Do the square

$$= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) \right)^2 dx + \frac{1}{2} \int p(x) \left(\nabla_x \log p_\theta(x) \right)^2 dx - \int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx$$

 cst

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\begin{aligned} \frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] &= \\ \text{Assume 1D RV} &= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right] \\ \text{Definition of expected value} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx \\ \text{Do the square} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) \right)^2 dx + \frac{1}{2} \int p(x) \left(\nabla_x \log p_\theta(x) \right)^2 dx \\ &\quad - \int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx \end{aligned}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\begin{aligned} \frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] &= \\ \text{Assume 1D RV} &= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right] \\ \text{Definition of expected value} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx \\ \text{Do the square} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) \right)^2 dx + \frac{1}{2} \int p(x) \left(\nabla_x \log p_\theta(x) \right)^2 dx \\ &\quad - \int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx \end{aligned}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$-\int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx =$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$-\int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx =$$

Chain rule: $d_x \log y = \frac{d_x y}{y}$

$$= -\int \cancel{p(x)} \frac{\nabla_x p(x)}{\cancel{p(x)}} \nabla_x \log p_\theta(x) dx$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$-\int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx =$$

$$\begin{aligned} \text{Chain rule: } d_x \log y &= \frac{d_x y}{y} &= -\int \cancel{p(x)} \frac{\nabla_x p(x)}{\cancel{p(x)}} \nabla_x \log p_\theta(x) dx \\ & &= -\int \nabla_x p(x) \nabla_x \log p_\theta(x) dx \end{aligned}$$

$$\text{Recall IBP: } \int u dv = uv - \int v du$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$-\int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx =$$

Chain rule: $d_x \log y = \frac{d_x y}{y}$

$$= -\int \cancel{p(x)} \frac{\nabla_x p(x)}{\cancel{p(x)}} \nabla_x \log p_\theta(x) dx$$

$$= -\int \nabla_x p(x) \nabla_x \log p_\theta(x) dx$$

Recall IBP: $\int u dv = uv - \int v du$

$$= -p(x) \nabla_x \log p_\theta \Big|_{-\infty}^{\infty} + \int p(x) \nabla_x^2 \log p_\theta dx$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$-\int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx =$$

Chain rule: $d_x \log y = \frac{d_x y}{y}$

$$= -\int \cancel{p(x)} \frac{\nabla_x p(x)}{\cancel{p(x)}} \nabla_x \log p_\theta(x) dx$$

$$= -\int \nabla_x p(x) \nabla_x \log p_\theta(x) dx$$

Recall IBP: $\int u dv = uv - \int v du$

$$= -p(x) \nabla_x \log p_\theta \Big|_{-\infty}^{\infty} + \int p(x) \nabla_x^2 \log p_\theta dx$$

$0 \because p(x) \rightarrow 0$ when $|x| \rightarrow \infty$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$-\int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx =$$

Chain rule: $d_x \log y = \frac{d_x y}{y}$

$$= -\int \cancel{p(x)} \frac{\nabla_x p(x)}{\cancel{p(x)}} \nabla_x \log p_\theta(x) dx$$

$$= -\int \nabla_x p(x) \nabla_x \log p_\theta(x) dx$$

Recall IBP: $\int u dv = uv - \int v du$

$$= -p(x) \nabla_x \log p_\theta \Big|_{-\infty}^{\infty} + \int p(x) \nabla_x^2 \log p_\theta dx$$

$$= \mathbb{E}_p \left[\nabla_x^2 \log p_\theta(x) \right]$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\begin{aligned} \frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] &= \\ \text{Assume 1D RV} &= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right] \\ \text{Definition of expected value} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx \\ \text{Do the square} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) \right)^2 dx + \frac{1}{2} \int p(x) \left(\nabla_x \log p_\theta(x) \right)^2 dx \\ &\quad - \int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx \end{aligned}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] =$$

Assume 1D RV

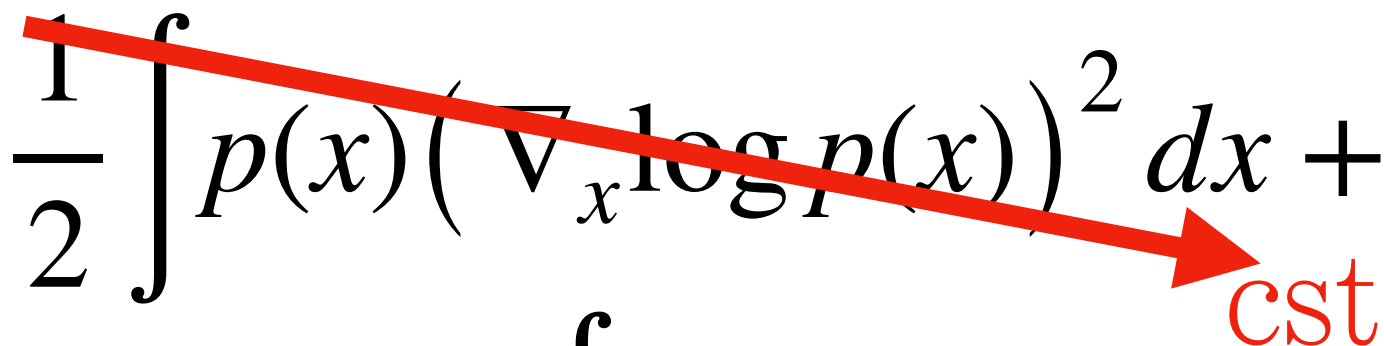
$$= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right]$$

Definition of expected value

$$= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx$$

Do the square

$$= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) \right)^2 dx + \frac{1}{2} \int p(x) \left(\nabla_x \log p_\theta(x) \right)^2 dx - \int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx$$

 cst

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] =$$

Assume 1D RV

$$= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right]$$

Definition of expected value

$$= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx$$

Do the square

$$= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) \right)^2 dx + \frac{1}{2} \int p(x) \left(\nabla_x \log p_\theta(x) \right)^2 dx$$

$$- \int p(x) \nabla_x \log p(x) \nabla_x \log p_\theta(x) dx$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\begin{aligned} \frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] &= \\ \text{Assume 1D RV} &= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right] \\ \text{Definition of expected value} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx \\ &= \mathbb{E}_p \left[\nabla_x^2 \log p_\theta(x) \right] + \frac{1}{2} \mathbb{E}_p \left[\nabla_x \log p_\theta(x) \right]^2 + \text{cst} \end{aligned}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] =$$

Assume 1D RV

$$= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right]$$

Definition of expected value

$$= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx$$

No more dependence on $p(x)$!

$$= \mathbb{E}_p \left[\nabla_x^2 \log p_\theta(x) \right] + \frac{1}{2} \mathbb{E}_p \left[\nabla_x \log p_\theta(x) \right]^2 + \text{cst}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

- Hyvärinen05: derives score matching training objective

$$\begin{aligned} \frac{1}{2} \mathbb{E}_p \left[\left\| \nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right\|_2^2 \right] &= \\ \text{Assume 1D RV} &= \frac{1}{2} \mathbb{E}_p \left[\left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 \right] \\ \text{Definition of expected value} &= \frac{1}{2} \int p(x) \left(\nabla_x \log p(x) - \nabla_x \log p_\theta(x) \right)^2 dx \\ &= \mathbb{E}_p \left[\nabla_x^2 \log p_\theta(x) \right] + \frac{1}{2} \mathbb{E}_p \left[\nabla_x \log p_\theta(x) \right]^2 + \text{cst} \\ \text{Generalize to N-D} &= \mathbb{E}_p \left[\text{tr} \left(\nabla_x^2 \log p_\theta(x) \right) + \frac{1}{2} \left\| \nabla_x \log p_\theta(x) \right\|_2^2 \right] \end{aligned}$$

Alternative: score-based generative models

Learn the SCORE FUNCTION $s(x) = \nabla_x \log p_t(x)$

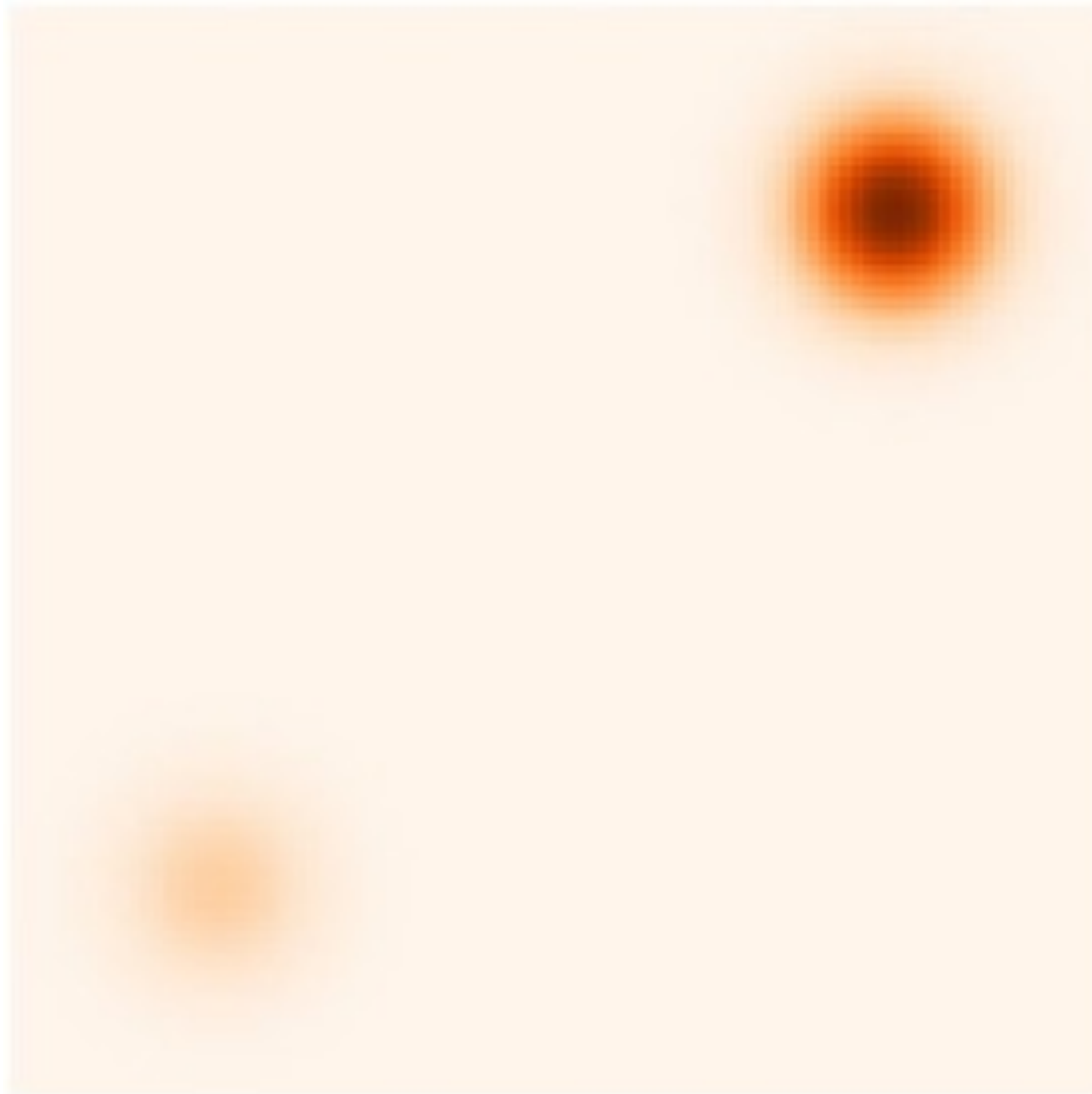
- Hyvärinen05: derives score matching training objective

$$\frac{1}{2} \mathbb{E}_p [||\nabla_x \log p(x) - \nabla_x \log p_\theta(x)||_2^2] = \mathbb{E}_p \left[\text{tr} (\nabla_x^2 \log p_\theta(x)) + \frac{1}{2} ||\nabla_x \log p_\theta(x)||_2^2 \right]$$

➡ The unknown data score has disappeared.
We only need samples from the data.

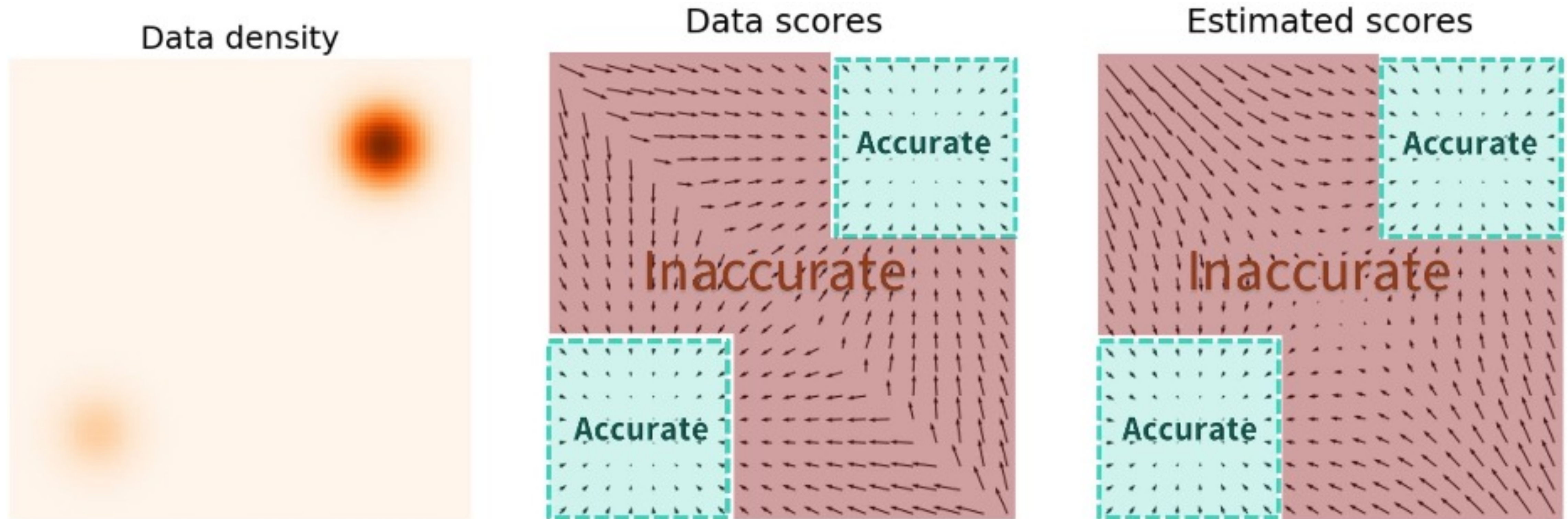
Score-based modelling with score matching

Data density



Score-based modelling with score matching

Naive score matching on clean high-dimensional data is insufficient for reliable generation...



Result: a score-based model only accurate in high density regions
(very small volume in a high dimensional problem)

➔ low quality generated samples

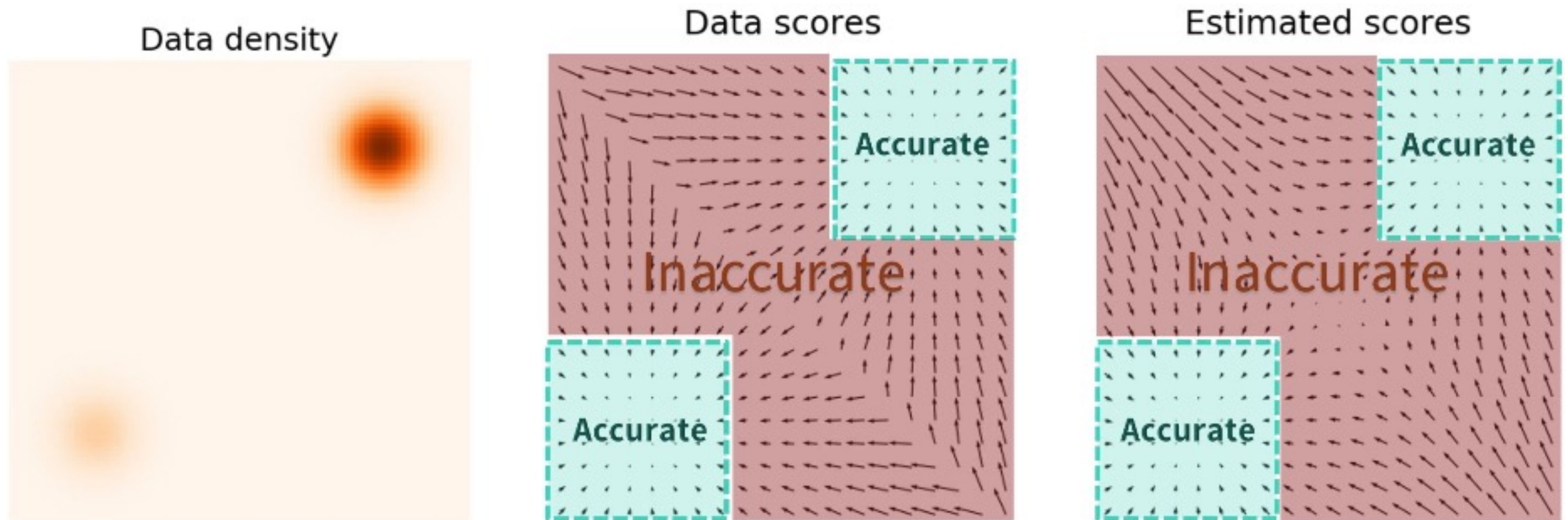


Score-based modelling with score matching

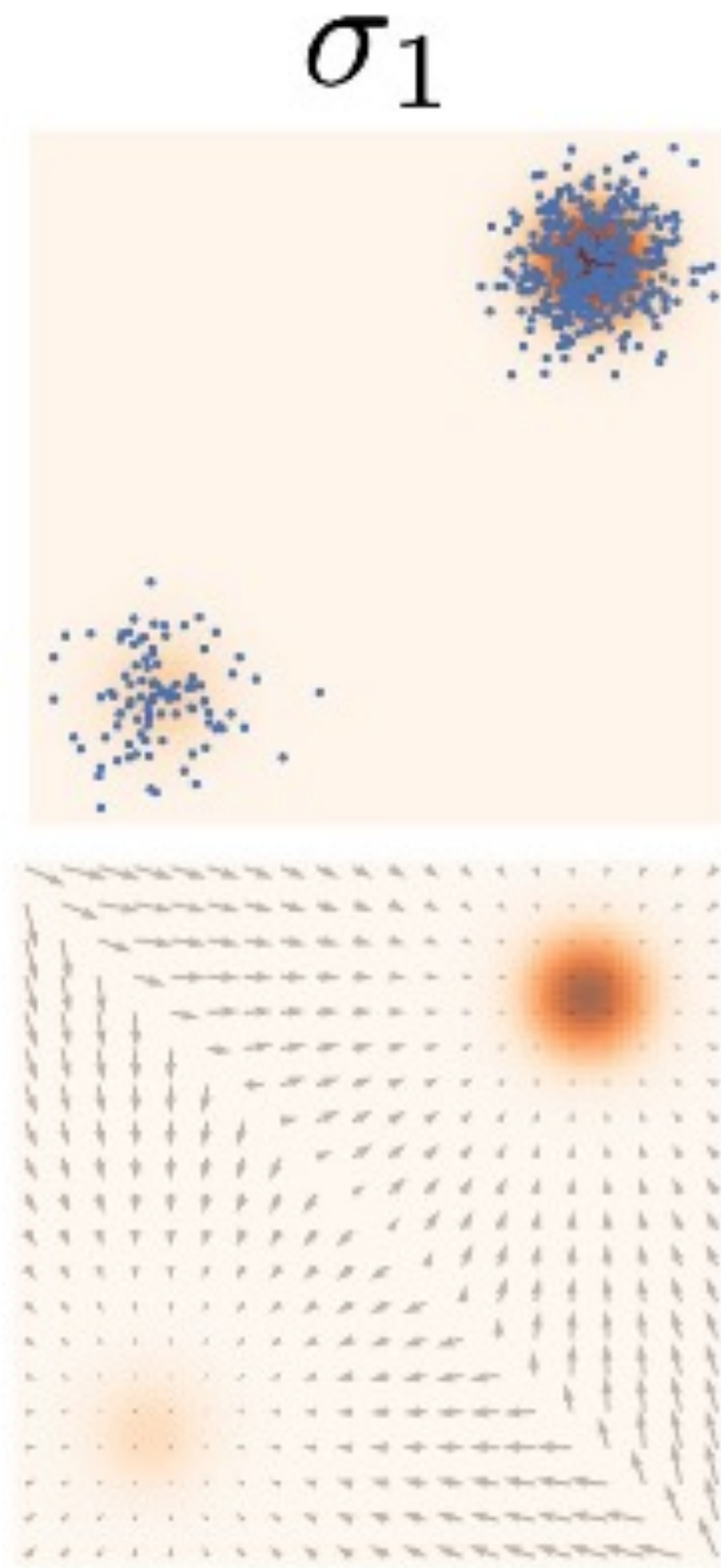
- Little interest in score-based models for over a decade...
- In 2019, Ermon & Song found a way to train a score-based model to be accurate everywhere!



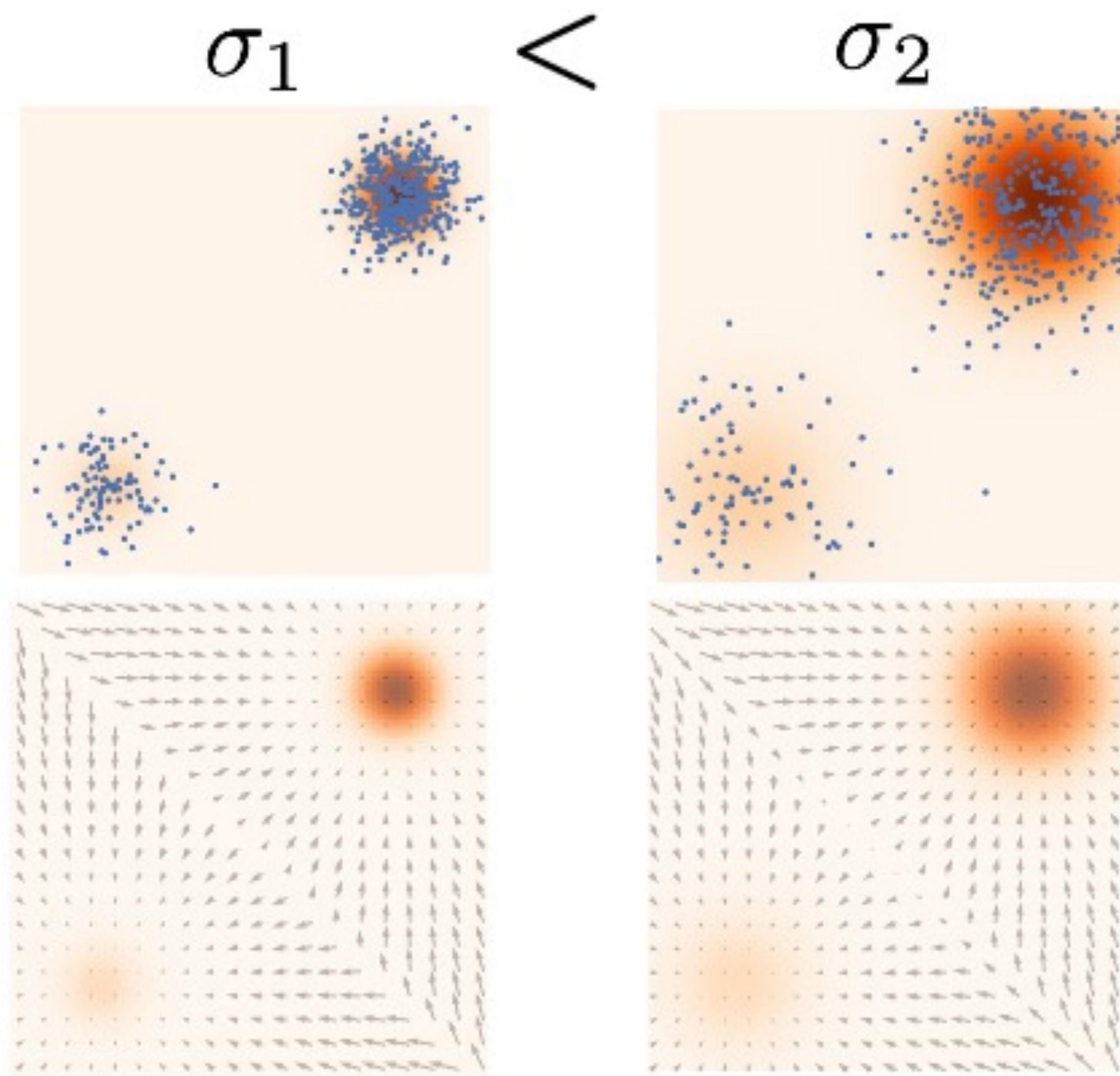
Noise Conditional Score Network (NCSN)



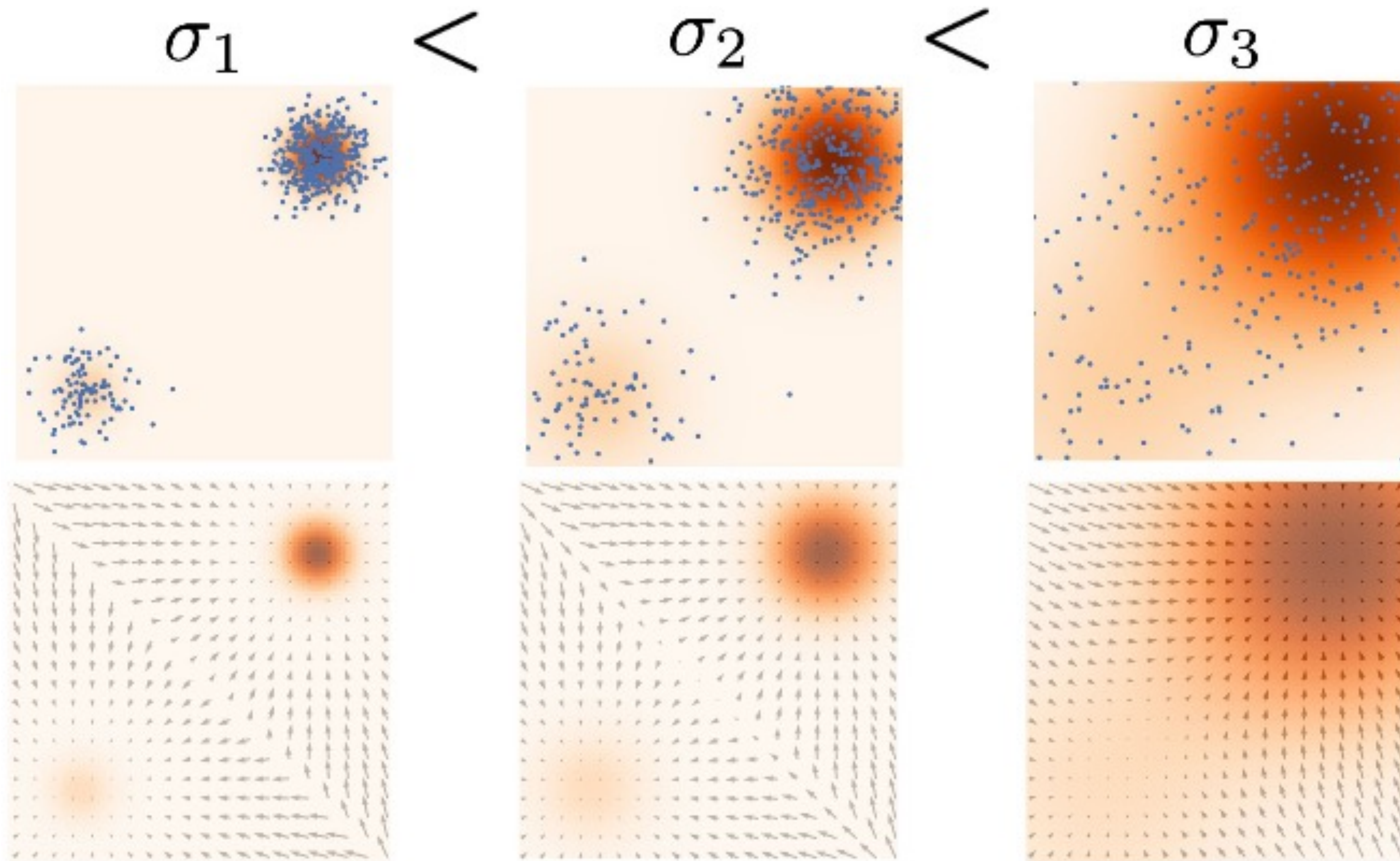
Noise Conditional Score Network (NCSN)



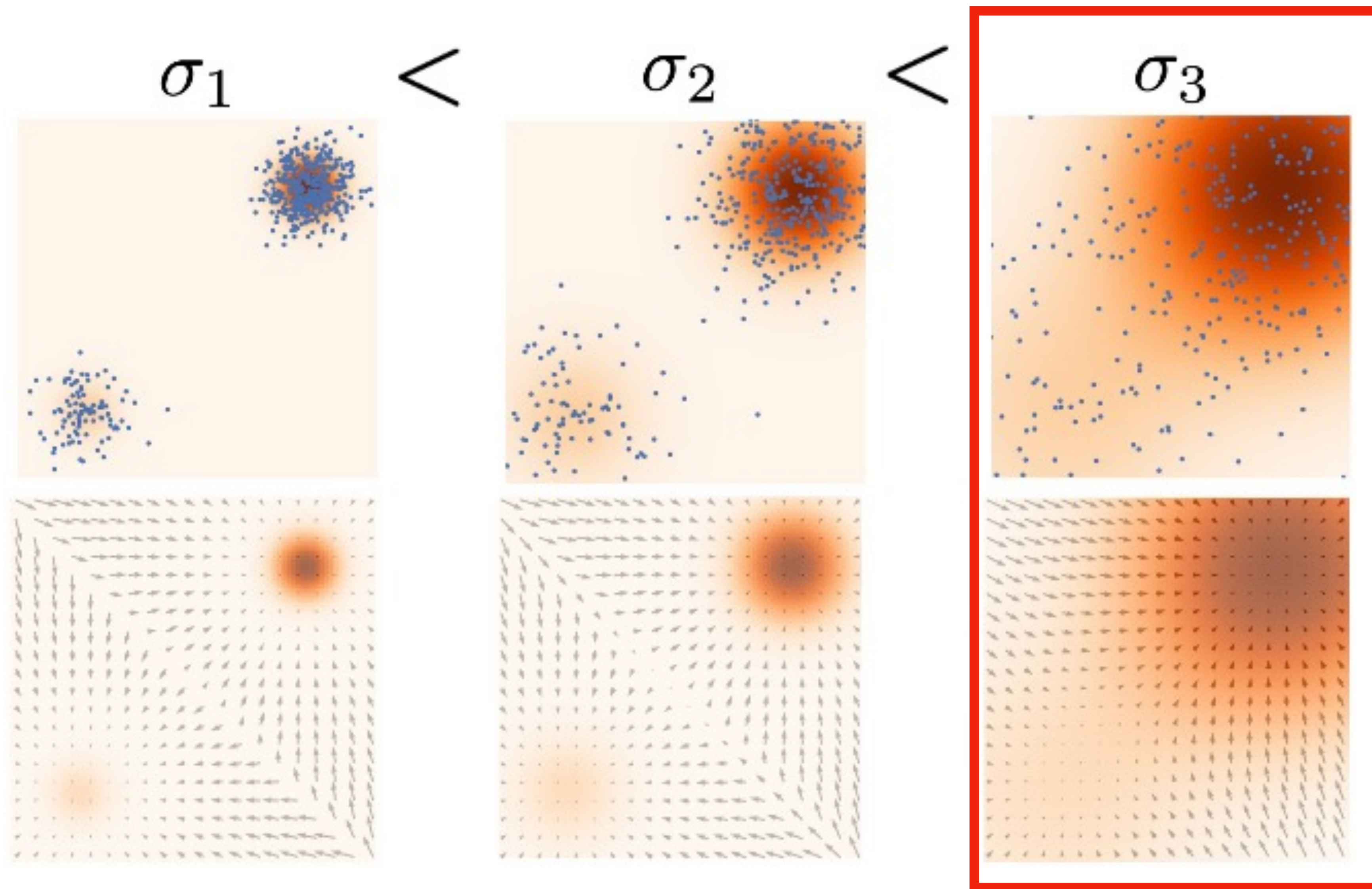
Noise Conditional Score Network (NCSN)



Noise Conditional Score Network (NCSN)

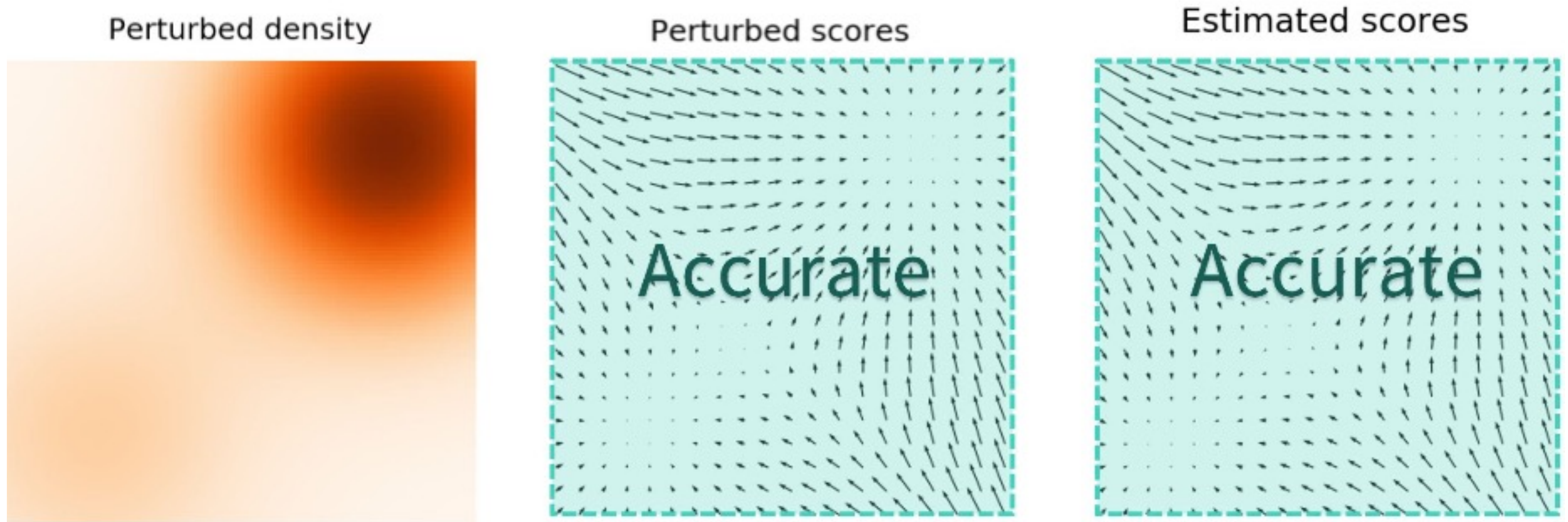


Noise Conditional Score Network (NCSN)



WHY NOT TRAIN USING ONLY THIS VERY LARGE NOISE LEVEL AND REMOVE THE OTHERS?

Noise Conditional Score Network (NCSN)

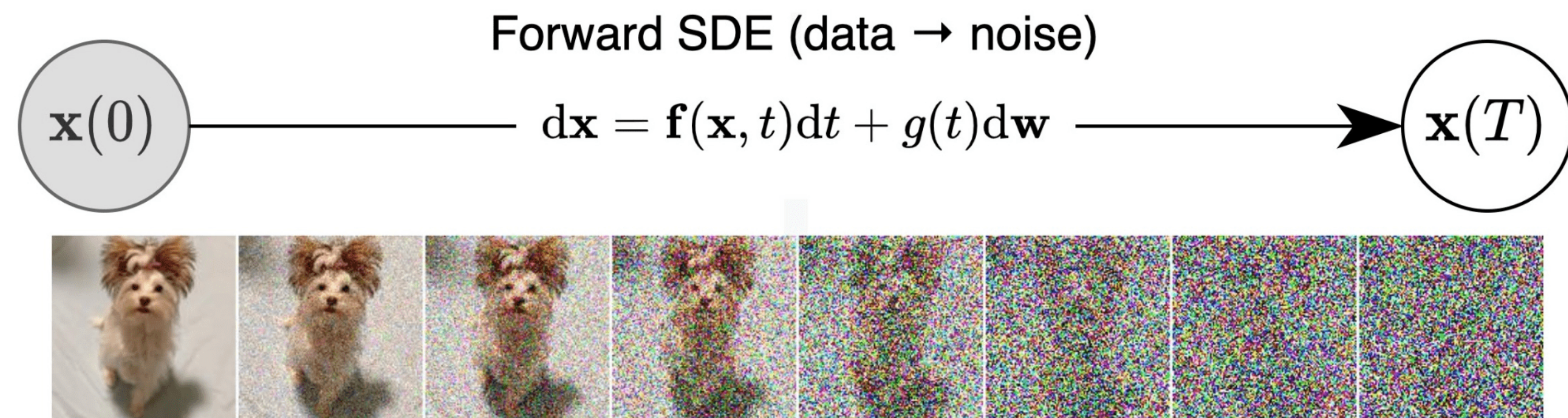


Result: NCSN trains the score across a much broader region of data space and across multiple noise levels!!

→ state-of-the-art in image generation, music generation, etc.
(+ much easier to train than GANs + architecture freedom)

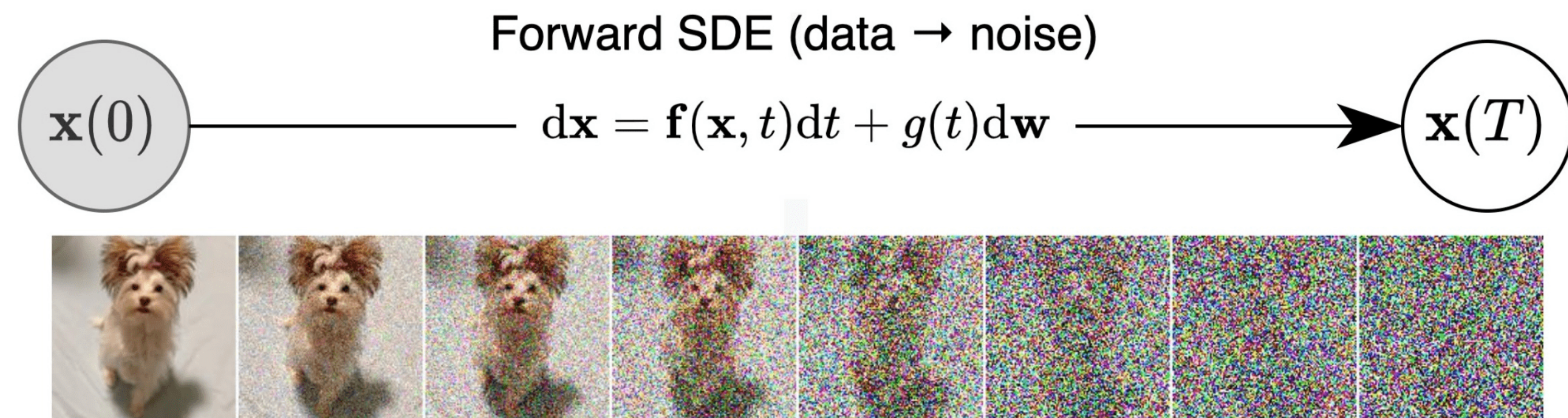


Forward process

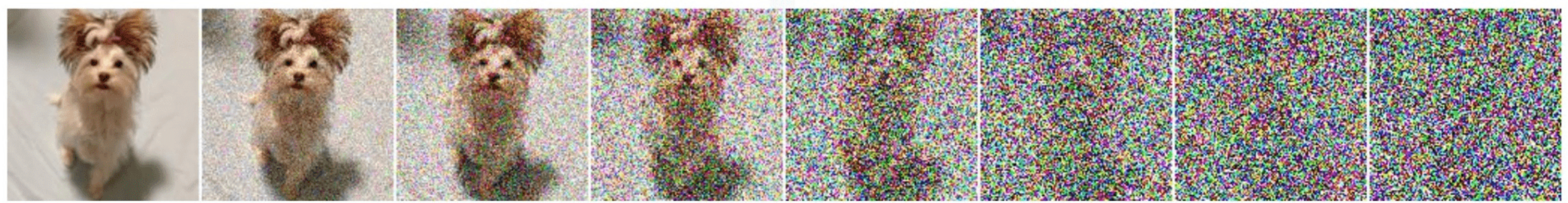


- The forward process is the key to the amazing performance of diffusion models [Song & Ermon 19, 20]
- Adding increasing levels of noise can be thought of as a continuous stochastic process
- Many stochastic processes are solutions to SDEs: $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$

Forward process



- The forward process is the key to the amazing performance of diffusion models [Song & Ermon 19, 20]
- Adding increasing levels of noise can be thought of as a continuous stochastic process
- Many stochastic processes are solutions to SDEs: $\text{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\text{d}t + g(t)\text{d}\mathbf{w}$
- Classic example is the VARIANCE PRESERVING (VP) SDE:
$$\text{d}\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}\text{d}t + \sqrt{\beta(t)}\text{d}\mathbf{w},$$
 where $\beta(t)$ is the noise scale as a function of time index t



Reverse process

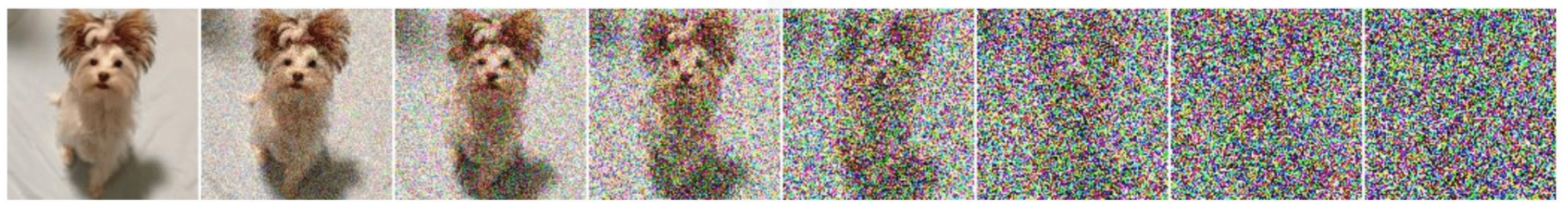
$$\mathbf{x}(0) \longleftarrow d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - g^2(t) \overbrace{\nabla_{\mathbf{x}} \log p_t(\mathbf{x})}^{\text{score function}} \right] dt + g(t) d\bar{\mathbf{w}} \longrightarrow \mathbf{x}(T)$$

Reverse SDE (noise $\rightarrow$ data)

- The forward SDE has a corresponding **REVERSE** SDE [Anderson 1982]
 - The only unknown is the **SCORE FUNCTION**
 - The NN learns to estimate the score function
- ➡ We can solve the reverse SDE to generate new data samples! How?
- Any SDE solver e.g. Euler-Maruyama
 - Song+20: the diffusion SDE has an associated probability-flow ODE with the same time-dependent marginal distributions

• Probability-flow ODE:
$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt$$

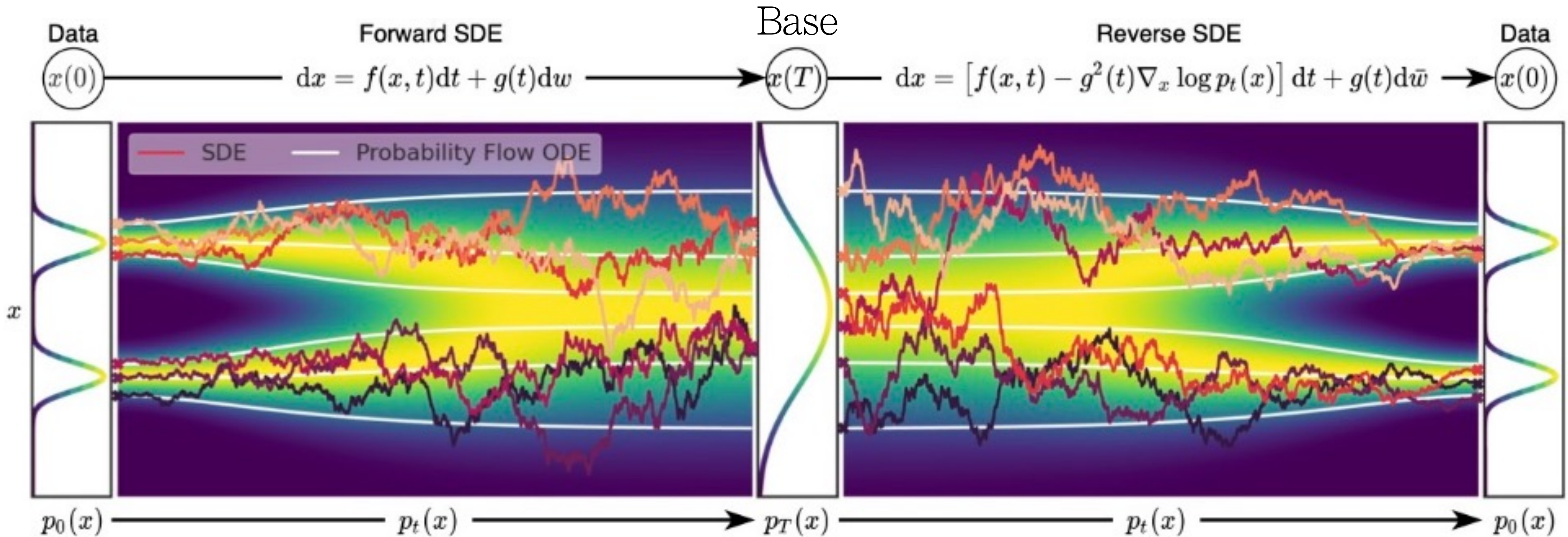
➡ Goes into any black box ODE solver!



Reverse process

$$\mathbf{x}(0) \longleftarrow d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt + g(t) d\bar{\mathbf{w}} \longrightarrow \mathbf{x}(T)$$

Reverse SDE (noise $\rightarrow$ data)



➡ Same target distribution, different sampling dynamics.

From probability-flow ODEs to flow matching

Learn the motion or “flow” directly

SCORE-BASED DIFFUSION

Learn the score then construct a deterministic velocity field:

$$s_{\phi}(x, t) \approx \nabla_x \log p_t(x)$$

$$\mathbf{v}_t^{\text{PF}}(\mathbf{x}) = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g(t)^2 s_{\phi}(\mathbf{x}, t) \right] dt$$

FLOW MATCHING

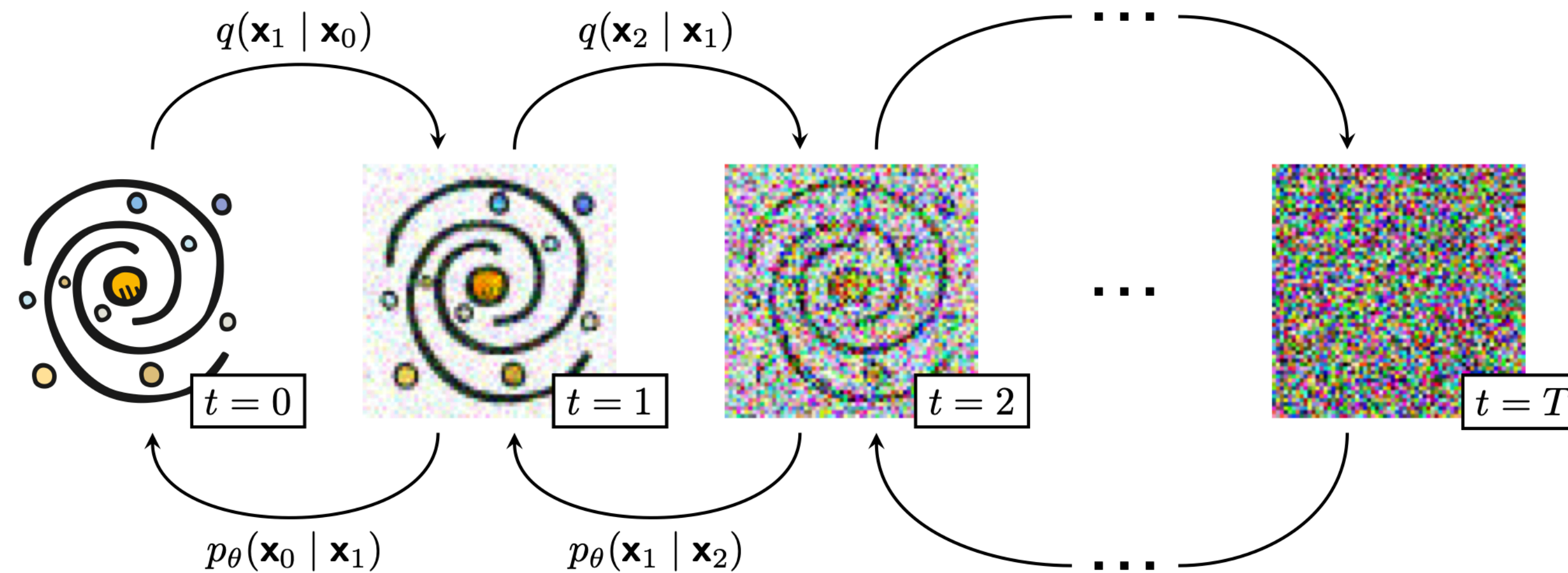
Choose a path from noise to data, then learn the velocity directly:

$$\mathbf{v}_{\phi}(\mathbf{x}, t) \approx \frac{d\mathbf{x}_t}{dt}$$

**BOTH ULTIMATELY GENERATE BY FOLLOWING DYNAMICS
FROM A SIMPLE BASE DISTRIBUTION TO THE DATA DISTRIBUTION.**

Diffusion: applications

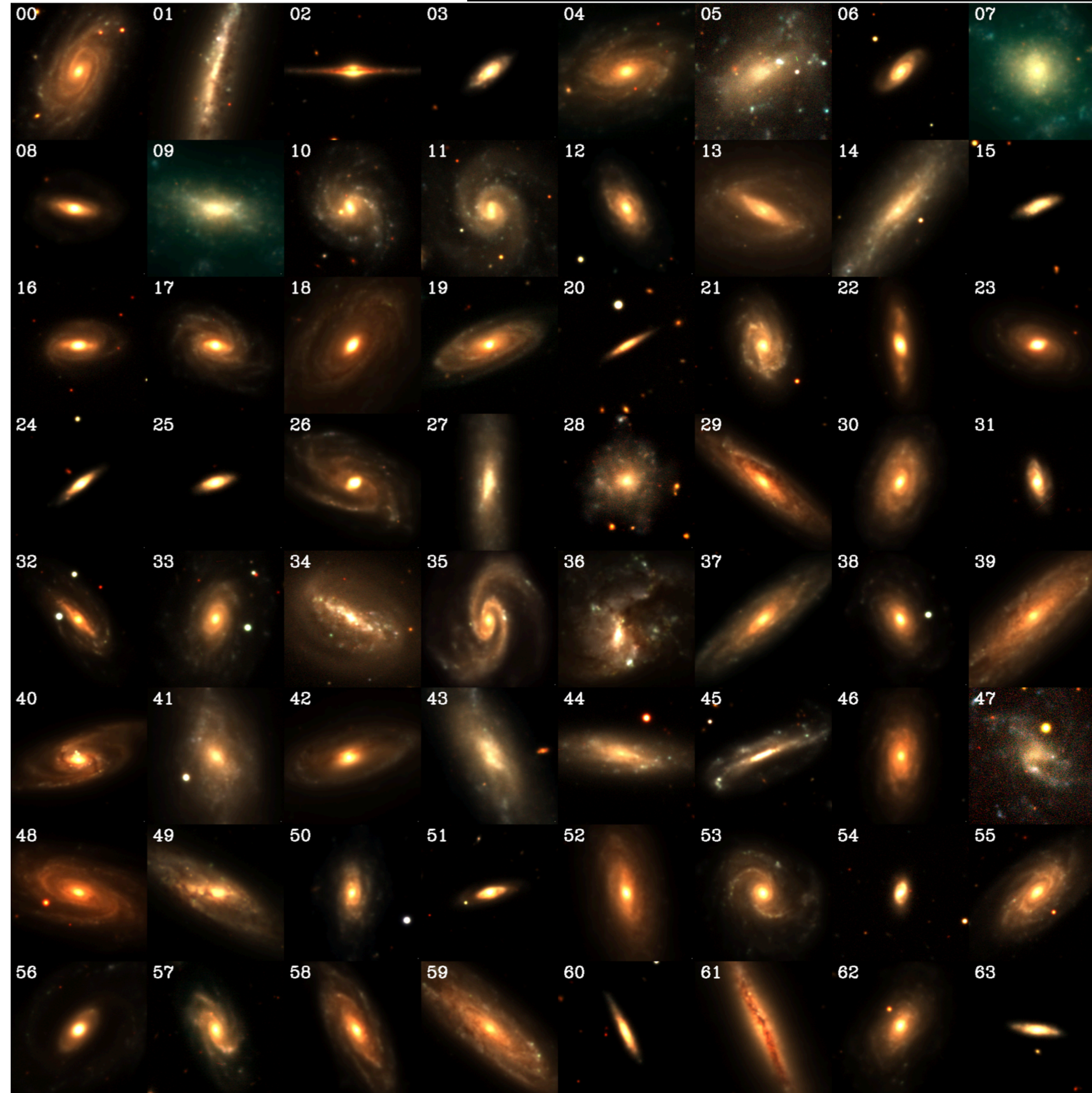
Smith M., Geach J. et al 2022 - <http://m.jjsmith.com/thisisnotagalaxy>



Diffusion: applications

Smith M., Geach J. et al 2022 - <http://m.jjsmith.com/thisisnotagalaxy>

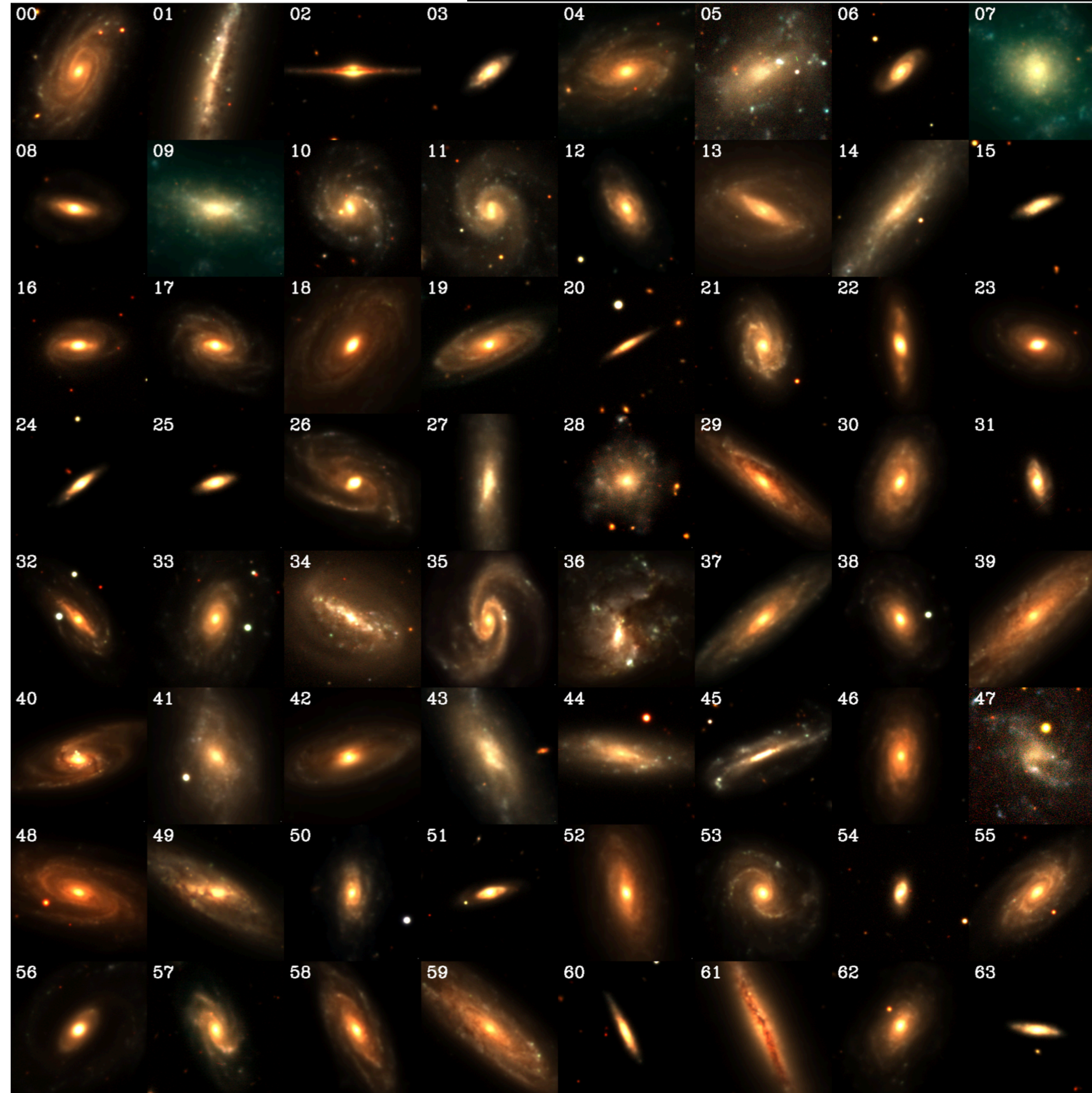
CAN YOU SPOT THE
GENERATED GALAXIES?



Diffusion: applications

Smith M., Geach J. et al 2022 - <http://m.jjsmith.com/thisisnotagalaxy>

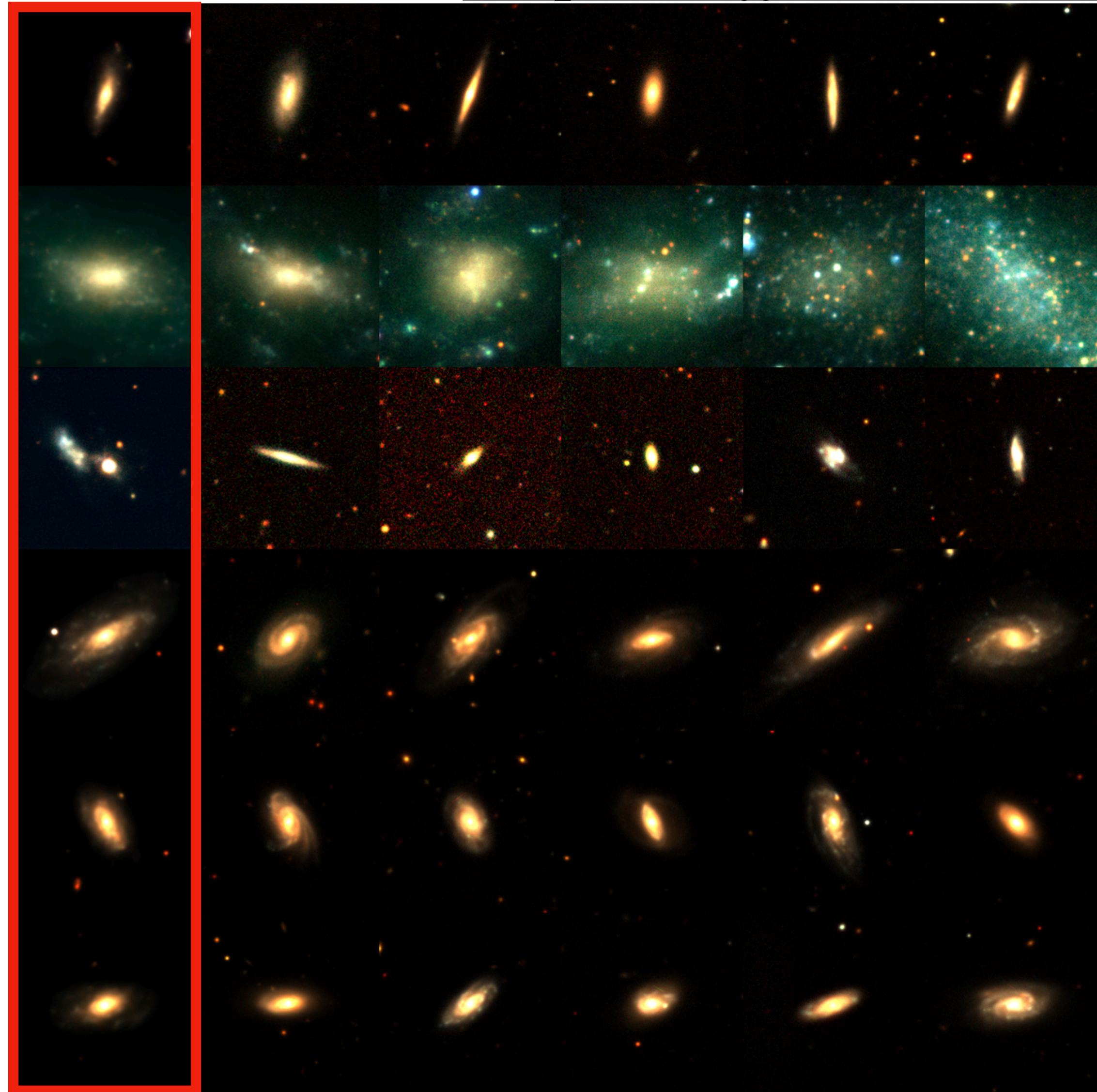
CAN YOU SPOT THE
GENERATED GALAXIES?



Diffusion: applications

Smith M., Geach J. et al 2022 - <http://m.jjsmith.com/thisisnotagalaxy>

GENERATED GALAXIES

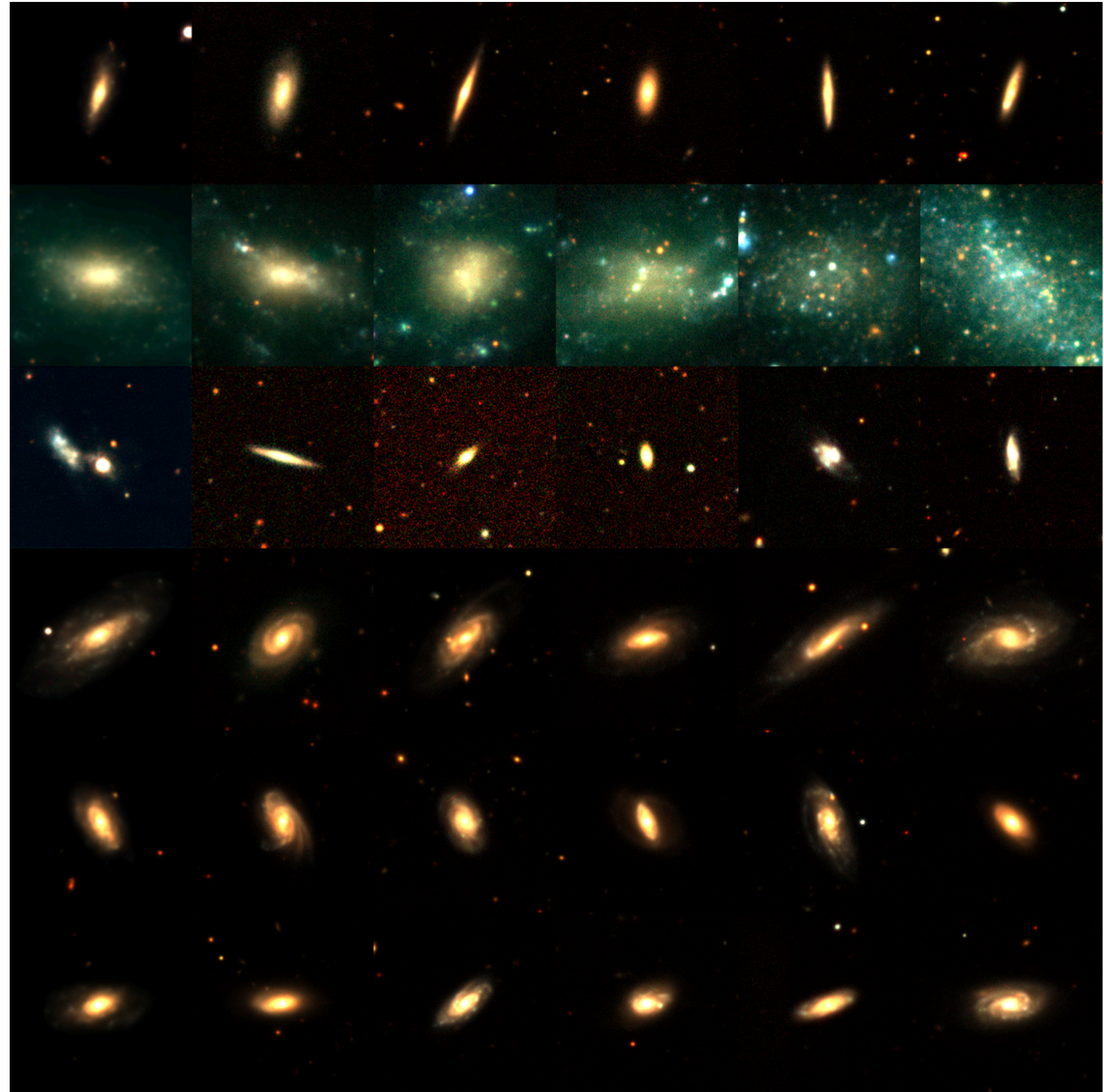


CLOSEST REAL GALAXIES

Diffusion: applications

Smith M., Geach J. et al 2022 - <http://m.jjsmith.com/thisisnotagalaxy>

VISUAL REALISM IS USEFUL, BUT WHAT SCIENTIFIC
PROPERTIES WOULD WE ALSO HAVE TO CHECK?



Key takeaways

Five questions for any generative model

1. What distribution are we learning?
 - $p(x), p(y|x), p(x|c)$?
2. Where does randomness enter?
 - Mixture component, latent variable, etc.
3. What does the network predict?
 - PDF parameters, latent representation, samples, score, etc.
4. What is the training objective?
 - NLL, adversarial loss, score loss, etc.
5. How do we validate the distribution?
 - Calibration, sample diversity, etc.

START SIMPLE. USE A MORE COMPLEX MODEL ONLY WHEN THE SCIENTIFIC PROBLEM NEEDS IT.

Outline

PART I

Generative modelling
basics

- Intro to generation
- Simplest generative model: Gaussian
- Gaussian mixture model (GMM)
- Mixture density network (MDN)
- Training procedure

PART II

Image generation

- Variational autoencoder (VAE)
- Generative adversarial network (GAN)
- Diffusion
- Applications

HANDS-ON

Galaxy survey:
Spectroscopy vs
photometry

- SDSS galaxy survey
- Photometry vs spectroscopy
- Notebook setup
- Kaggle

The problem

Photometry vs spectroscopy

- imaging is cheap and broad-band
- spectroscopy is expensive but gives the training label
- colours contain incomplete and sometimes degenerate redshift information

$$u, g, r, i, z \rightarrow p(z_{\text{spec}} | u, g, r, i, z)$$

The data

INPUT

raw colours

corrected colours

r_mag

concentration_r

z_spec

MEANING

No Milky Way extinction correction

Foreground-extinction corrected

Overall apparent magnitude

Crude morphology

Target

Baselines and metric

MLP: $x \rightarrow z$ (counts as a single component Gaussian using validation residuals)

MDN: $x \rightarrow \{a_k, \mu_k, \sigma_k\}$

Kaggle score: $-\frac{1}{N} \sum_i \log p(z_i | x_i)$

- lower NLL is better
- can be negative

Rough schedule

- 10 min: run every cell and submit the baseline
- 15 min: inspect and clean the data
- 25 min: change features, tune network:
 - number of layers
 - number of nodes
 - number of MDN components K , etc.
- 5 min: choose and upload final submissions **DEADLINE IS 12:25 PM**